Xlib

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1. Introduction

- **The X Window System (X11)**: The X Window System, often called X11, is the general windowing system design and protocol. It defines things like:
  - how graphical programs communicate with a display server,
  - how windows are created, moved, resized, and drawn into,
  - how keyboard and mouse input is delivered,
  - how clients and the display server exchange messages.

  There is a distinction between a specification/standard and a concrete program that implements it.
- **X.Org display server**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **X clients**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **Windowing system**: A windowing system is the overall graphical environment model that lets programs use windows, input devices, and the screen in a coordinated way.
- **Display server**: A display server is the actual program that sits between graphical applications and the hardware/display system.
  - **Windowing system:** X Window System / X11
  - **Display server:** X.Org Server

  A display server is responsible for managing access to the screen, keyboard, mouse, touchpad, and sometimes GPU/display hardware. Graphical applications usually do not draw directly to the physical screen. Instead, they communicate with the display server.

  A windowing system is broader. It is the architecture that defines how graphical applications, windows, input, drawing, and the display server/compositor interact.

  It includes concepts such as:
  - windows;
  - graphical clients;
  - input events;
  - drawing surfaces;
  - focus;
  - stacking order;
  - protocols for communication between apps and the graphical environment.
- **Intro**: The X Window System is a network-transparent window system that was designed at MIT. X display servers run on computers with either monochrome or color bitmap display hardware. The server distributes user input to and accepts output requests from various client programs located either on the same machine or elsewhere in the network. Xlib is a C subroutine library that application programs (clients) use to interface with the window system by means of a stream connection. Although a client usually runs on the same machine as the X server it is talking to, this need not be the case.

Note: Monochrome means **one color**. It describes any visual, design, or medium that is constructed using only a single base hue along with its various tints, tones, and shades. While black-and-white is the most common example, a monochrome palette can be built entirely around any color

- **Network-transparent**: In general, network-transparent means that a system lets you use some resource over a network as if it were local, while hiding most of the networking details from the user or program.
- **Color bitmap**: A color bitmap is an image represented as a rectangular grid of pixels, where each pixel stores color information. A color bitmap uses more bits per pixel so that each pixel can represent many possible colors. In a common 24-bit RGB color bitmap, each pixel stores three color components:

  $$(R, G, B).$$

  Each component uses 8 bits, so each ranges from 0 to 255.
- **Screens and Display**: The X Window System supports one or more screens containing overlapping windows or subwindows. A screen is a physical monitor and hardware that can be color, grayscale, or monochrome. There can be multiple screens for each display or workstation. A single X server can provide display services for any number of screens. A set of screens for a single user with one keyboard and one pointer (usually a mouse) is called a **display**.
- **Window hierarchy**: All the windows in an X server are arranged in strict hierarchies. At the top of each hierarchy is a root window, which covers each of the display screens. Each root window is partially or completely covered by child windows. All windows, except for root windows, have parents. There is usually at least one window for each application program. Child windows may in turn have their own children. In this way, an application program can create an arbitrarily deep tree on each screen. X provides graphics, text, and raster operations for windows.

  A child window can be larger than its parent. That is, part or all of the child window can extend beyond the boundaries of the parent, but all output to a window is clipped by its parent.

  That means X will only actually display the part of the child window that lies inside the parent window. Anything outside the parent's visible rectangle is not drawn on the screen.

  Children of a window have overlapping locations, one of the children is considered to be on top of or raised over the others, thus obscuring them. Output to areas covered by other windows is suppressed by the window system unless the window has backing store.

  Child windows obscure their parents, and graphic operations in the parent window usually are clipped by the children
- **Border**: A window has a border zero or more pixels in width, which can be any pattern (pixmap) or solid color you like. A window usually but not always has a background pattern, which will be repainted by the window system when uncovered.
- **Coordinates**: Each window and pixmap has its own coordinate system. The coordinate system has the X axis horizontal and the Y axis vertical with the origin [0, 0] at the upper-left corner. Coordinates are integral, in terms of pixels, and coincide with pixel centers. For a window, the origin is inside the border at the inside, upper-left corner.

- **Expose event**: X does not guarantee to preserve the contents of windows. When part or all of a window is hidden and then brought back onto the screen, its contents may be lost. The server then sends the client program an Expose event to notify it that part or all of the window needs to be repainted. Programs must be prepared to regenerate the contents of windows on demand.
- **Drawables**: X also provides off-screen storage of graphics objects, called pixmaps. Single plane (depth 1) pixmaps are sometimes referred to as bitmaps. Pixmaps can be used in most graphics functions interchangeably with windows and are used in various graphics operations to define patterns or tiles. Windows and pixmaps together are referred to as drawables.
- **Xlib function requests**: Most of the functions in Xlib just add requests to an output buffer. These requests later execute asynchronously on the X server. Functions that return values of information stored in the server do not return (that is, they block) until an explicit reply is received or an error occurs. You can provide an error handler, which will be called when the error is reported.
- **XSync**: If a client does not want a request to execute asynchronously, it can follow the request with a call to XSync, which blocks until all previously buffered asynchronous events have been sent and acted on. As an important side effect, the output buffer in Xlib is always flushed by a call to any function that returns a value from the server or waits for input.
- **Resource ID**: Many Xlib functions will return an integer resource ID, which allows you to refer to objects stored on the X server. These can be of type **Window, Font, Pixmap, Colormap, Cursor, and GContext**, as defined in the file **<X11/X.h>**. These resources are created by requests and are destroyed (or freed) by requests or when connections are closed. Most of these resources are potentially sharable between applications, and in fact, windows are manipulated explicitly by window manager programs. Fonts and cursors are shared automatically across multiple screens. Fonts are loaded and unloaded as needed and are shared by multiple clients. Fonts are often cached in the server. Xlib provides no support for sharing graphics contexts between applications
- **Client programs and events**: Client programs are informed of events. Events may either be side effects of a request (for example, restacking windows generates Expose ev ents) or completely asynchronous (for example, from the keyboard). A client program asks to be informed of events. Because other applications can send events to your application, programs must be prepared to handle (or ignore) events of all types.

  Input events (for example, a key pressed or the pointer moved) arrive asynchronously from the server and are queued until they are requested by an explicit call (for example, **XNextEvent** or **XWindowEvent**). In addition, some library functions (for example, **XRaiseWindow**) generate Expose and **ConfigureRequest** events. These events also arrive asynchronously, but the client may wish to explicitly wait for them by calling **XSync** after calling a function that can cause the server to generate events.
- **Errors**: Some functions return **Status**, an integer error indication. If the function fails, it returns a zero. If the function returns a status of zero, it has not updated the return arguments. Because C does not provide multiple return values, many functions must return their results by writing into clientpassed storage. By default, errors are handled either by a standard library function or by one that you provide. Functions that return pointers to strings return NULL pointers if the string does not exist.

  The X server reports **protocol errors** at the time that it detects them. If more than one error could be generated for a given request, the server can report any of them.

Because Xlib usually does not transmit requests to the server immediately (that is, it buffers them), errors can be reported much later than they actually occur. For debugging purposes, howev er, Xlib provides a mechanism for forcing synchronous behavior. When synchronization is enabled, errors are reported as they are generated.

When Xlib detects an error, it calls an error handler, which your program can provide. If you do not provide an error handler, the error is printed, and your program terminates.

- **Standard Header Files**: The following include files are part of the Xlib standard:
  - **<X11/Xlib.h>**: This is the main header file for Xlib. The majority of all Xlib symbols are declared by including this file. This file also contains the preprocessor symbol XlibSpecificationRelease. This symbol is defined to have the 6 in this release of the standard. (Release 5 of Xlib was the first release to have this symbol.)
  - **<X11/X.h>**: This file declares types and constants for the X protocol that are to be used by applications. It is included automatically from <X11/Xlib.h>, so application code should never need to reference this file directly.
  - **<X11/Xcms.h>**: This file contains symbols for much of the color management facilities. All functions, types, and symbols with the prefix ''Xcms'', plus the Color Conversion Contexts macros, are declared in this file. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xutil.h>**: This file declares various functions, types, and symbols used for inter-client communication and application utility functions, which are described in chapters 14 and 16. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xresource.h>**: This file declares all functions, types, and symbols for the resource manager facilities. **<X11/Xlib.h>** must be included before including this file.
  - **<X11/Xatom.h>**: This file declares all predefined atoms, which are symbols with the prefix ''XA_''.
  - **<X11/cursorfont.h>**: This file declares the cursor symbols for the standard cursor font. All cursor symbols have the prefix ''XC_''.
  - **<X11/keysymdef.h>**: This file declares all standard KeySym values, which are symbols with the prefix ''XK_''. The KeySyms are arranged in groups, and a preprocessor symbol controls inclusion of each group. The preprocessor symbol must be defined prior to inclusion of the file to obtain the associated values. The preprocessor symbols are
    - XK_MISCELLANY
    - XK_XKB_KEYS
    - XK_3270
    - XK_LATIN1
    - XK_LATIN2
    - XK_LATIN3
    - XK_LATIN4
    - XK_KATAKANA
    - XK_ARABIC
    - XK_CYRILLIC
    - XK_GREEK

    - XK_TECHNICAL
    - XK_SPECIAL
    - XK_PUBLISHING
    - XK_APL
    - XK_HEBREW
    - XK_THAI
    - XK_KOREAN
  - **<X11/keysym.h>**: This file defines the preprocessor symbols listed above and then includes **<X11/keysymdef.h>**.
  - **<X11/Xlibint.h>**: This file declares all the functions, types, and symbols used for extensions. This file automatically includes **<X11/Xlib.h>**.
  - **<X11/Xproto.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xlibint.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/Xprotostr.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xproto.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/X10.h>**: This file declares all the functions, types, and symbols used for the X10 compatibility functions.

- **KeySyms**: A KeySym is X11's symbolic meaning for a keyboard key.

  A keyboard event gives you a **KeyCode**, which is closer to "which physical/logical key was pressed." A **KeySym** is closer to "what symbol does that key currently mean?" The Xlib docs define a KeyCode as a physical or logical key code and a KeySym as the encoding of the symbol on the keycap.

  A keycode by itself is not enough, because the same physical key can mean different things depending on keyboard layout and modifiers.

- **Atoms**: An Atom is an integer ID that stands for a string name known to the X server.

  X11 uses atoms heavily for **window properties, property types, selections**, and **ClientMessage** protocols. Xlib's documentation describes a window property as named, typed data; the property name has a unique identifier called an atom, and property types are also represented using atoms.

  Instead of constantly sending strings like:

```
"WM_DELETE_WINDOW"
"_NET_WM_NAME"
"UTF8_STRING"
"WM_PROTOCOLS"
```

  clients ask the server for an integer identifier:

```
Atom wm_delete = XInternAtom(display, "WM_DELETE_WINDOW",
    ↪ False);
Atom wm_protocols = XInternAtom(display, "WM_PROTOCOLS",
    ↪ False);
```

Then they use those Atom values in later protocol requests.

- **Extensions**: An **X extension** is an addition to the core X11 protocol. The core X protocol has a fixed set of requests, events, errors, and objects. Extensions allow the server and clients to support extra functionality without changing the original core protocol.

  Examples include

```
RANDR     resizing/rotating screens, monitor layout
XRender   improved 2D rendering/compositing primitives
XInput2   advanced input devices/events
XFixes    fixes and additions to older X behavior
Composite off-screen window compositing support
Shape     non-rectangular windows
GLX       OpenGL integration with X
```

  Before using an extension, a client usually checks whether the server supports it. Xlib provides functions such as:

```
XQueryExtension(display, "RANDR", &major, &first_event,
    ↪ &first_error);
```

- **Generic values and types**: The following symbols are defined by Xlib and used throughout the manual.
  - Xlib defines the type **Bool** and the Boolean values **True** and **False**.
  - **None** is the universal null resource ID or atom.
  - The type **XID** is used for generic resource IDs.
  - The type **XPointer** is defined to be char* and is used as a generic opaque pointer to data.
- **Naming and Argument Conventions within Xlib**: Xlib follows a number of conventions for the naming and syntax of the functions. Given that you remember what information the function requires, these conventions are intended to make the syntax of the functions more predictable.

  The major naming conventions are:

  - To differentiate the X symbols from the other symbols, the library uses mixed case for external symbols. It leaves lowercase for variables and all uppercase for user macros, as per existing convention.
  - All Xlib functions begin with a capital X.

- The beginnings of all function names and symbols are capitalized.
- All user-visible data structures begin with a capital X. More generally, anything that a user might dereference begins with a capital X.
- Macros and other symbols do not begin with a capital X. To distinguish them from all user symbols, each word in the macro is capitalized.
- All elements of or variables in a data structure are in lowercase. Compound words, where needed, are constructed with underscores (_).
- The display argument, where used, is always first in the argument list.
- All resource objects, where used, occur at the beginning of the argument list immediately after the display argument.
- When a graphics context is present together with another type of resource (most commonly,adrawable), the graphics context occurs in the argument list after the other resource. Drawables outrank all other resources.
- Source arguments always precede the destination arguments in the argument list.
- The x argument always precedes the y argument in the argument list.
- The width argument always precedes the height argument in the argument list.
- Where the x, y, width, and height arguments are used together, the x and y arguments
- always precede the width and height arguments.
- Where a mask is accompanied with a structure, the mask always precedes the pointer to the structure in the argument list.

- **Programming Considerations**: The major programming considerations are:
  - Coordinates and sizes in X are actually 16-bit quantities. This decision was made to minimize the bandwidth required for a given lev el of performance. Coordinates usually are declared as an int in the interface. Values larger than 16 bits are truncated silently. Sizes (width and height) are declared as unsigned quantities.
  - Keyboards are the greatest variable between different manufacturers' workstations. If you want your program to be portable, you should be particularly conservative here.
  - Many display systems have limited amounts of off-screen memory. If you can, you should minimize use of pixmaps and backing store.
  - The user should have control of his screen real estate. Therefore, you should write your applications to react to window management rather than presume control of the entire screen. What you do inside of your top-level window, however, is up to your application. For further information, see chapter 14 and the Inter-Client Communication Conventions

- **Character Sets and Encodings**: Some of the Xlib functions make reference to specific character sets and character encodings. The following are the most common:
  - **X Portable Character Set**: A basic set of 97 characters, which are assumed to exist in all locales supported by Xlib. This set contains the following characters:

a..z A..Z 0..9
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
<space>, <tab>, <newline>

This set is the left/lower half of the graphic character set of ISO8859-1 plus space, tab, and newline. It is also the set of graphic characters in 7-bit ASCII plus the same three control characters. The actual encoding of these characters on the host is system dependent.

- **Host Portable Character Encoding**: The encoding of the X Portable Character Set on the host. The encoding itself is not defined by this standard, but the encoding must be the same in all locales supported by Xlib on the host. If a string is said to be in the Host Portable Character Encoding, then it only contains characters from the X Portable Character Set, in the host encoding.
  - **Latin-1**: The coded character set defined by the ISO8859-1 standard.
  - **Latin Portable Character Encoding**: The encoding of the X Portable Character Set using the Latin-1 codepoints plus ASCII control characters. If a string is said to be in the Latin Portable Character Encoding, then it only contains characters from the X Portable Character Set, not all of Latin-1.
  - **STRING Encoding**: Latin-1, plus tab and newline.
  - **POSIX Portable Filename Character Set**: The set of 65 characters, which can be used in naming files on a POSIX-compliant host, that are correctly processed in all locales. The set is:

$$a..z \quad A..Z \quad 0..9 \quad ._- .$$

- **Formatting Conventions**:
  - Global symbols are printed in this special font. These can be either function names, symbols defined in include files, or structure names. When declared and defined, function arguments are printed in italics. In the explanatory text that follows, they usually are printed in regular type.
  - Each function is introduced by a general discussion that distinguishes it from other functions. The function declaration itself follows, and each argument is specifically explained. Although ANSI C function prototype syntax is not used, Xlib header files normally declare functions using function prototypes in ANSI C environments. General discussion of the function, if any is required, follows the arguments. Where applicable, the last paragraph of the explanation lists the possible Xlib error codes that the function can generate.
  - To eliminate any ambiguity between those arguments that you pass and those that a function returns to you, the explanations for all arguments that you pass start with the word **specifies** or, in the case of multiple arguments, the word **specify**. The explanations for all arguments that are returned to you start with the word **returns** or, in the case of multiple arguments, the word **return**. The explanations for all arguments that you can pass and are returned start with the words **specifies** and **returns**.
  - Any pointer to a structure that is used to return a value is designated as such by the _return suffix as part of its name. All other pointers passed to these functions are used for reading only. A few arguments use pointers to structures that are used for both input and output and are indicated by using the _in_out suffix.

- **Clients**: A client is any program that connects to the *X* server and sends it requests.

  The X server controls the display, keyboard, mouse, screens, windows, pixmaps, graphics contexts, fonts, cursors, and other X resources. A client is a program using that server.

  So in *x*, the "client" is not necessarily a remote machine. It is usually just a local process on your own computer.

2. Display functions

- **Intro**: Before your program can use a display, you must establish a connection to the X server. Once you have established a connection, you then can use the Xlib macros and functions discussed in this chapter to return information about the display. We will discuss how to
  - Open (connect to) the display
  - Obtain information about the display, image formats, or screens
  - Generate a **NoOperation** protocol request
  - Free client-created data
  - Close (disconnect from) a display
  - Use X Server connection close operations
  - Use Xlib with threads
  - Use internal connections
- **Opening the Display**: To open a connection to the X server that controls a display, use **XOpenDisplay**

```
Display *XOpenDisplay(char* display_name)
```

  - **display_name** Specifies the hardware display name, which determines the display and communications domain to be used. On a POSIX-conformant system, if the display_name is NULL, it defaults to the value of the DISPLAY environment variable.

  Note: The encoding and interpretation of the display name are implementation-dependent. Strings in the Host Portable Character Encoding are supported; support for other characters is implementation-dependent. On POSIX-conformant systems, the display name or DISPLAY environment variable can be a string in the format:

  protocol/hostname:number.screen_number

  - **protocol**: Specifies a protocol family or an alias for a protocol family. Supported protocol families are implementation dependent. The protocol entry is optional. If protocol is not specified, the / separating protocol and hostname must also not be specified.
  - **hostname**: Specifies the name of the host machine on which the display is physically attached. You follow the hostname with either a single colon (:) or a double colon (::).
  - **number**: Specifies the number of the display server on that host machine. You may optionally follow this display number with a period (.). A single CPU can have more than one display. Multiple displays are usually numbered starting with zero.
  - **screen_number**: Specifies the screen to be used on that server. Multiple screens can be controlled by a single X server. The screen_number sets an internal variable that can be accessed by using the **DefaultScreen** macro or the **XDefaultScreen** function if you are using languages other than C

  For example, the following would specify screen 1 of display 0 on the machine named ‘‘dualheaded’’:

The **XOpenDisplay** function returns a **Display** structure that serves as the connection to the X server and that contains all the information about that X server. **XOpenDisplay** connects your application to the X server through TCP or DECnet communications protocols, or through some local inter-process communication protocol. If the protocol is specified as "tcp", "inet", or "inet6", or if no protocol is specified and the hostname is a host machine name and a single colon (:) separates the hostname and display number, **XOpenDisplay** connects using **TCP streams**.

If the protocol is specified as "inet", TCP over IPv4 is used. If the protocol is specified as "inet6", TCP over IPv6 is used. Otherwise, the implementation determines which IP version is used. If the hostname and protocol are both not specified, Xlib uses whatever it believes is the fastest transport. If the hostname is a host machine name and a double colon (::) separates the hostname and display number, **XOpenDisplay** connects using DECnet. A single X server can support any or all of these transport mechanisms simultaneously. A particular Xlib implementation can support many more of these transport mechanisms.

If successful, **XOpenDisplay** returns a pointer to a **Display** structure, which is defined in **<X11/Xlib.h>.** If **XOpenDisplay** does not succeed, it returns NULL. After a successful call to **XOpenDisplay,** all of the screens in the display can be used by the client. The screen number specified in the display_name argument is returned by the **DefaultScreen** macro (or the **XDefaultScreen** function). You can access elements of the **Display** and **Screen** structures only by using the information macros or functions.

- **Obtaining Information about the Display, Image Formats, or Screens**: The Xlib library provides a number of useful macros and corresponding functions that return data from the Display structure. The macros are used for C programming, and their corresponding function equivalents are for other language bindings.

  All other members of the Display structure (that is, those for which no macros are defined) are private to Xlib and must not be used. Applications must never directly modify or inspect these private members of the Display structure.

- **Planes**: In the X11 graphics model, a pixel value is not treated only as a color; it is also a group of bits. Each bit position is a plane. For example, on a drawable with depth 24, a pixel has 24 meaningful bits, so conceptually there are 24 planes.

  Note that here **depth** is the number of meaningful bits used to represent one pixel's value in a drawable.

- **Plane mask**: A plane mask chooses which bit positions of the destination pixel value are affected.

  Wherever the mask has a 1, that pixel bit plane may be changed. Wherever the mask has a 0, that pixel bit is preserved.

- **Colormaps**: A colormap is an X server object that describes how pixel values map to actual colors. In Xlib, many drawing functions do not directly use RGB colors. They use pixel values. A colormap is what lets the X server interpret those pixel values as colors.

  Pixel value → colormap → actual display color

- **Graphics contexts**: A graphics context in Xlib is an object that stores the drawing state used by graphics operations.

Instead of passing every drawing option directly to functions like XDrawLine, XFillRectangle, or XDrawString, Xlib groups many of those options into a **GC (graphics context)**.

```
XDrawLine(display, window, gc, x1, y1, x2, y2);
```

The coordinates are passed directly, but things like the line color, line width, fill style, clipping region, drawing function, font, and plane mask are taken from the gc.

The Xlib documentation describes a graphics context as holding most graphics operation attributes, including line width, line style, plane mask, foreground, background, tile, stipple, clipping region, end style, and join style. The internal contents of a GC are private to Xlib.

- **Default screen**: The default screen in Xlib is the screen number that Xlib chooses as the main screen for a given `Display*`.
- **Visuals**: A visual describes how a pixel value should be interpreted as a color on a particular screen.

  The visual describes the format/rules for interpreting pixel values, while the colormap provides the actual color mapping data when that visual class uses a lookup table.

  - **Visual**: What kind of pixel-to-color interpretation is used
  - **Colormap**: The table/mapping used by that interpretation

  The core Xlib visual classes are:

  - StaticGray
  - GrayScale
  - StaticColor
  - PseudoColor
  - TrueColor
  - DirectColor

  A `Visual*` is an Xlib object describing one supported color interpretation scheme.

- **Display Macros**: Applications should not directly modify any part of the Display and Screen structures. The members should be considered read-only, although they may change as the result of other operations on the display.

```
AllPlanes
unsigned long XAllPlanes()
```

Both return a value with all bits set to 1 suitable for use in a plane argument to a procedure.

Both BlackPixel and WhitePixel can be used in implementing a monochrome application. These pixel values are for permanently allocated entries in the default colormap. The actual RGB (red, green, and blue) values are settable on some screens and, in any case, may not actually be black or white. The names are intended to convey the expected relative intensity of the colors.

```
BlackPixel(display, screen_number)
unsigned long XBlackPixel (display, screen_number)
```

- display Specifies the connection to the X server.
- screen_number Specifies the appropriate screen number on the host server.

Both return the black pixel value for the specified screen.

```
WhitePixel (display, screen_number)
unsigned long XWhitePixel (display, screen_number)
```

Both return the white pixel value for the specified screen.

```
ConnectionNumber(display)
int XConnectionNumber(display)
```

Both return a connection number for the specified display. On a POSIX-conformant system, this is the file descriptor of the connection.

Internally, Xlib is communicating with the X server over something like a Unix-domain socket or TCP socket, and ConnectionNumber exposes that descriptor.

```
DefaultColormap(display, screen_number)
Colormap XDefaultColormap(display, screen_number)
```

Both return the default colormap ID for allocation on the specified screen. Most routine allocations of color should be made out of this colormap.

```
DefaultDepth(display, screen_number)
int XDefaultDepth(display, screen_number)
```

Both return the depth (number of planes) of the default root window for the specified screen. Other depths may also be supported on this screen (see XMatchVisualInfo).

```
int* XListDepths(display, screen_number, count_return)
```

– count_return returns the number of depths

The **XListDepths** function returns the array of depths that are available on the specified screen. If the specified screen_number is valid and sufficient memory for the array can be allocated, **XListDepths** sets count_return to the number of available depths. Otherwise, it does not set count_return and returns NULL. To release the memory allocated for the array of depths, use **XFree.**

XListDepths ask the *X* server:

"*For this screen, what drawable/window depths are supported*"

```
DefaultGC(display, screen_number)
GC XDefaultGC(display, screen_number)
```

Both return the default graphics context for the root window of the specified screen. This GC is created for the convenience of simple applications and contains the default GC components with the foreground and background pixel values initialized to the black and white pixels for the screen, respectively. You can modify its contents freely because it is not used in any Xlib function. This GC should never be freed.

```
DefaultRootWindow(display)
Window XDefaultRootWindow(display)
```

Both return the root window for the default screen.

```
DefaultScreenOfDisplay(display)
Screen* XDefaultScreenOfDisplay(display)
```

Both return a pointer to the default screen.

```
ScreenOfDisplay(display, screen_number)
Screen* XScreenOfDisplay(display, screen_number)
```

Both return a pointer to the indicated screen

```
DefaultScreen(display)
int XDefaultScreen(display)
```

Both return the default screen number referenced by the **XOpenDisplay** function. This macro or function should be used to retrieve the screen number in applications that will use only a single screen.

```
DefaultVisual(display, screen_number)
Visual* XDefaultVisual(display, screen_number)
```

Both return the default visual type for the specified screen.

```
DisplayCells(display, screen_number)
int XDisplayCells(display, screen_number)
```

Both return the number of entries in the default colormap.

```
DisplayPlanes(display, screen_number)
int XDisplayPlanes(display, screen_number)
```

Both return the depth of the root window of the specified screen.

```
DisplayString(display)
char* XDisplayString(display)
```

Both return the string that was passed to XOpenDisplay when the current display was opened. On POSIX-conformant systems, if the passed string was NULL, these return the value of the DISPLAY environment variable when the current display was opened. These are useful to applications that invoke the fork system call and want to open a new connection to the same display from the child process as well as for printing error messages.

```
long XExtendedMaxRequestSize(display)
```

The **XExtendedMaxRequestSize** function returns zero if the specified display does not support an extended-length protocol encoding; otherwise, it returns the maximum request size (in 4-byte units) supported by the server using the extended-length encoding. The Xlib functions **XDrawLines, XDrawArcs, XFillPolygon, XChangeProperty, XSetClipRectangles,** and **XSetRegion** will use the extended-length encoding as necessary, if supported by the server. Use of the extended-length encoding in other Xlib functions (for example, **XDrawPoints, XDrawRectangles**, **XDrawSegments, XFillArcs, XFillRectangles, XPutImage)** is permitted but not required; an Xlib implementation may choose to split the data across multiple smaller requests instead.

```
long XMaxRequestSize(display)
```

The XMaxRequestSize function returns the maximum request size (in 4-byte units) supported by the server without using an extended-length protocol encoding. Single protocol requests to the server can be no larger than this size unless an extended-length protocol encoding is supported by the server. The protocol guarantees the size to be no smaller than 4096 units (16384 bytes). Xlib automatically breaks data up into multiple protocol requests as necessary for the following functions: **XDrawPoints, XDrawRectangles, XDrawSegments, XFillArcs, XFillRectangles,** and **XPutImage**

```
LastKnownRequestProcessed(display)
unsigned long XLastKnownRequestProcessed(display)
```

Both extract the full serial number of the last request known by Xlib to have been processed by the X server. Xlib automatically sets this number when replies, events, and errors are received.

```
NextRequest (display)
unsigned long XNextRequest (display)
```

Both extract the full serial number that is to be used for the next request. Serial numbers are maintained separately for each display connection.

Note: In Xlib, the **serial number** of a request is the sequence number assigned to each protocol request sent from your client to the X server.

```
ProtocolVersion(display)
int XProtocolVersion(display)
```

Both return the major version number (11) of the X protocol associated with the connected dis play.

```
ProtocolRevision (display)
int XProtocolRevision (display)
```

Both return the minor protocol revision number of the X server.

```
QLength (display)
int XQLength(display)
```

Both return the length of the event queue for the connected display. Note that there may be more events that have not been read into the queue yet (see **XEventsQueued).**

```
RootWindow(display, screen_number)
Window XRootWindow(display, screen_number)
```

Both return the root window. These are useful with functions that need a drawable of a particular screen and for creating top-level windows.

```
ScreenCount(display)
int XScreenCount(display)
```

Both return the number of available screens.

```
ServerVendor (display)
char* XServerVendor (display)
```

Both return a pointer to a null-terminated string that provides some identification of the owner of the X server implementation. If the data returned by the server is in the Latin Portable Character Encoding, then the string is in the Host Portable Character Encoding. Otherwise, the contents of the string are implementation-dependent.

```
VendorRelease (display)
int XVendorRelease (display)
```

Both return a number related to a vendor's release of the X server.

- **Image data formats**: In Xlib, **XY format** and **Z format** are two different memory layouts for image data. They mainly matter when using functions like:

```
XGetImage(...)
XCreateImage(...)
XPutImage(...)
```

and constants such as

```
XYBitmap
XYPixmap
ZPixmap
```

  - **Z format (ZPixmap)**: ZPixmap stores pixels pixel-by-pixel. Each pixel value contains all of its plane bits together. For a 24-bit image, one pixel might contain something like:

RRRRRRRR GGGGGGGG BBBBBBBB

So the image is stored as a normal packed array of pixel values. This is usually the easiest format to understand and use. If you want to read or write individual pixels with **XGetPixel / XPutPixel,** ZPixmap is the common choice.

- **XY format (XYPixmap)**: Instead of storing whole pixels together, it stores one bit-plane at a time. In a 4-bit-deep pixmap, each pixel has 4 bits. In XYPixmap, the image data is stored as separate planes.

  plane 0: bit 0 of every pixel
  plane 1: bit 1 of every pixel
  plane 2: bit 2 of every pixel
  plane 3: bit 3 of every pixel

  So,

  plane 0: pixel0_bit0 pixel1_bit0 pixel2_bit0 ...
  plane 1: pixel0_bit1 pixel1_bit1 pixel2_bit1 ...
  plane 2: pixel0_bit2 pixel1_bit2 pixel2_bit2 ...
  plane 3: pixel0_bit3 pixel1_bit3 pixel2_bit3 ...

  This is why it is called XY format: it stores bitmap-style x/y image planes, one plane after another.

- **XY format (XYBitmap)**: XYBitmap is a special case of XY format for depth 1 images. A bitmap has only one bit per pixel. So, there is only one plane. This is used for masks, stipples, cursors, icons, and other monochrome data.

- **Scanlines**: A scanline is one horizontal row of pixels in an image. If an image is

  $$\begin{aligned} \text{width} &= 640 \\ \text{height} &= 480 \end{aligned}$$

  then it has 480 scanlines, each containing 640 pixels.

  In Xlib, the scanline unit is the size, in bits, of the storage chunks used to represent a scanline, especially for bitmap/plane-style image data.

  Suppose you have a 1-bit bitmap image

  pixel: 0 1 2 3 4 5 6 7 8 9 ...
  value: 1 0 1 1 0 0 1 0 1 1 ...

  The pixels are individual bits, but memory is addressed in bytes or words, not individual bits. So X needs to know how the bits are grouped.

  If the scanline unit is 8, the scanline is grouped like this:

  bits 0-7 | bits 8-15 | bits 16-23 | ...

  That grouping affects how X interprets the bits in memory.

  If the image width is not a multiple of the scanline unit/padding requirement, the scanline is padded at the end. The extra bits are not pixels; they are just unused storage so that the next scanline starts at the required boundary.

- **Image format functions and macros**: Applications are required to present data to the X server in a format that the server demands. To help simplify applications, most of the work required to convert the data is provided by Xlib

The **XPixmapFormatValues** structure provides an interface to the pixmap format information that is returned at the time of a connection setup. It contains:

```
typedef struct {
    int depth;
    int bits_per_pixel;
    int scanline_pad;
} XPixmapFormatValues;
```

To obtain the pixmap format information for a given display, use **XListPixmapFormats**.

```
XPixmapFormatValues* XListPixmapFormats (display,
↪  count_return)
```

The **XListPixmapFormats** function returns an array of **XPixmapFormatValues** structures that describe the types of Z format images supported by the specified display. If insufficient memory is available, **XListPixmapFormats** returns NULL. To free the allocated storage for the **XPixmapFormatValues** structures, use **XFree.**

```
ImageByteOrder(display)
int XImageByteOrder(display)
```

Both specify the required byte order for images for each scanline unit in XY format (bitmap) or for each pixel value in Z format. The macro or function can return either **LSBFirst** or **MSBFirst.**

```
BitmapUnit (display)
int XBitmapUnit(display)
```

Both return the size of a bitmap's scanline unit in bits. The scanline is calculated in multiples of this value.

```
BitmapBitOrder (display)
int XBitmapBitOrder(display)
```

Within each bitmap unit, the left-most bit in the bitmap as displayed on the screen is either the least signifiant or most signifiant bit in the unit. This macro or function can return **LSBFirst** or **MSBFirst.**

```
BitmapPad (display)
int XBitmapPad (display)
```

Each scanline must be padded to a multiple of bits returned by this macro or function.

```
DisplayHeight(display, screen_number)
int XDisplayHeight(display, screen_number)
```

Both return an integer that describes the height of the screen in pixels.

```
DisplayHeightMM (display, screen_number)
int XDisplayHeightMM(display, screen_number)
```

Both return the height of the specified screen in millimeters.

```
DisplayWidth (display, screen_number)
int XDisplayWidth (display, screen_number)
```

Both return the width of the screen in pixels.

```
DisplayWidthMM (display, screen_number)
int XDisplayWidthMM (display, screen_number)
```

Both return the width of the specified screen in millimeters.

- **Backing stores**: A backing store is an optional server-side saved copy of a window's contents.

  Normally, if part of your window is covered and then uncovered, the X server may not remember what was there. Instead, it sends your program an Expose event telling you:

  "*this part of your window became visible again; redraw it.*"

  With backing store, the server may keep a copy of the obscured window contents. Then, when the window becomes visible again, the server can restore the pixels itself without requiring your client to redraw.

  Important: backing store is only a hint. Your program must still handle Expose events correctly. The X server is allowed not to preserve the contents, especially if it decides the backing store would be too expensive.

- **Save-unders**: A save-under is also a server-side pixel preservation hint, but it is used for a different purpose.

  Backing store preserves the contents of the window itself.

  Save-under preserves the contents of whatever is underneath a temporary window.

This is mainly for things like:

- menus
- tooltips
- popup windows
- drag boxes
- temporary overlays

Without save-under:

menu appears
↓
menu covers part of another window
↓
menu disappears
↓
the window underneath may need Expose events and redraws

With save-under:

menu appears
↓
server saves pixels underneath the menu
↓
menu disappears
↓
server restores the pixels underneath

- **Event masks**: An event mask tells the X server which events your client wants to receive for a window. X is event-driven. Your program usually waits for events like:
  - key press
  - mouse click
  - mouse movement
  - window exposure
  - resize
  - map/unmap
  - destroy
  - focus changes
  - property changes

  But the server does not automatically send every possible event to every client. You select the events you care about using an event mask.

  For example,

```
XSelectInput(
    display,
    window,
    ExposureMask  KeyPressMask  ButtonPressMask |
    ↪  StructureNotifyMask
);
```

If you want to redraw your window when it becomes visible or damaged, you must select:

```
ExposureMask
```

Then the server can send you **Expose** events.

- **Screen information macros**: The following lists the C language macros, their corresponding function equivalents that are for other language bindings, and what data they both can return. These macros or functions all take a pointer to the appropriate screen structure.

```
BlackPixelOfScreen (screen)
unsigned long XBlackPixelOfScreen (screen)
```

Both return the black pixel value of the specified screen.

```
WhitePixelOfScreen (screen)
unsigned long XWhitePixelOfScreen (screen)
```

Both return the white pixel value of the specified screen.

```
CellsOfScreen (screen)
int XCellsOfScreen(screen)
```

Both return the number of colormap cells in the default colormap of the specified screen.

```
DefaultColormapOfScreen (screen)
Colormap XDefaultColormapOfScreen (screen)
```

  - **Screen* screen**: screen Specifies the appropriate Screen structure.

Both return the default colormap of the specified screen.

```
DefaultDepthOfScreen (screen)
int XDefaultDepthOfScreen (screen)
```

Both return the depth of the root window.

```
DefaultGCOfScreen (screen)
GC XDefaultGCOfScreen (screen)
```

Both return a default graphics context (GC) of the specified screen, which has the same depth as the root window of the screen. The GC must never be freed.

```
DefaultVisualOfScreen (screen)
Visual *XDefaultVisualOfScreen (screen)
```

Both return the default visual of the specified screen. For information on visual types

```
DoesBackingStore (screen)
int XDoesBackingStore(screen)
```

Both return a value indicating whether the screen supports backing stores. The value returned can be one of **WhenMapped, NotUseful,** or Always

```
DoesSaveUnders (screen)
Bool XDoesSaveUnders (screen)
```

Both return a Boolean value indicating whether the screen supports save unders. If **True,** the screen supports save unders. If **False,** the screen does not support save unders (see section 3.2.5).

```
DisplayOfScreen (screen)
Display *XDisplayOfScreen(screen)
```

Both return the display of the specified screen.

```
int XScreenNumberOfScreen(screen)
```

The **XScreenNumberOfScreen** function returns the screen index number of the specified screen.

```
EventMaskOfScreen (screen)
long XEventMaskOfScreen (screen)
```

Both return the event mask of the root window for the specified screen at connection setup time.

```
WidthOfScreen (screen)
int XWidthOfScreen (screen)
```

Both return the width of the specified screen in pixels.

```
HeightOfScreen (screen)
int XHeightOfScreen(screen)
```

Both return the height of the specified screen in pixels.

```
WidthMMOfScreen (screen)
int XWidthMMOfScreen (screen)
```

Both return the width of the specified screen in millimeters.

```
HeightMMOfScreen (screen)
int XHeightMMOfScreen(screen)
```

Both return the height of the specified screen in millimeters.

```
MaxCmapsOfScreen (screen)
int XMaxCmapsOfScreen(screen)
```

Both return the maximum number of installed colormaps supported by the specified screen

```
MinCmapsOfScreen (screen)
int XMinCmapsOfScreen(screen)
```

Both return the minimum number of installed colormaps supported by the specified screen

```
PlanesOfScreen (screen)
int XPlanesOfScreen(screen)
```

Both return the depth of the root window.

```
RootWindowOfScreen (screen)
Window XRootWindowOfScreen (screen)
```

Both return the root window of the specified screen.

- **Generating a NoOperation Protocol Request**: To execute a **NoOperation** protocol request, use **XNoOp**

```
XNoOp(display)
```

The **XNoOp** function sends a **NoOperation** protocol request to the X server, thereby exercising the connection.

- **Freeing client-created data**: To free in-memory data that was created by an Xlib function, use **XFree**.

```
XFree(void* data)
```

The **XFree** function is a general-purpose Xlib routine that frees the specified data. You must use it to free any objects that were allocated by Xlib, unless an alternate function is explicitly specified for the object. A NULL pointer cannot be passed to this function.

- **Closing the Display**: To close a display or disconnect from the X server, use **XCloseDisplay**.

```
XCloseDisplay(Display* display)
```

The **XCloseDisplay** function closes the connection to the X server for the display specified in the **Display** structure and destroys all windows, resource IDs (**Window, Font, Pixmap, Colormap, Cursor,** and **GContext),** or other resources that the client has created on this display, unless the close-down mode of the resource has been changed (see **XSetCloseDownMode).** Therefore, these windows, resource IDs, and other resources should never be referenced again or an error will be generated. Before exiting, you should call **XCloseDisplay** explicitly so that any pending errors are reported as **XCloseDisplay** performs a final **XSync** operation.

XCloseDisplay can generate a **BadGC** error.

Xlib provides a function to permit the resources owned by a client to survive after the client's connection is closed. To change a client's close-down mode, use **XSetCloseDownMode**

```
XSetCloseDownMode (Display* display, int close_mode)
```

  - **close_mode**: close_mode Specifies the client close-down mode. You can pass **DestroyAll, RetainPermanent,** or **RetainTemporary.**

The **XSetCloseDownMode** defines what will happen to the client's resources at connection close. A connection starts in **DestroyAll** mode.

XSetCloseDownMode can generate a **BadValue** error.

- **Selections**: A selection is the X Window System's mechanism for clipboard-like data transfer.

  The important point is that the selected data is usually not stored in the X server. Instead, one client owns a selection, and another client requests that data from the owner.

  Common selections are

```
PRIMARY
SECONDARY
CLIPBOARD
```

  - **PRIMARY** usually the currently highlighted text
  - **CLIPBOARD** usually explicit copy/paste, like Ctrl+C / Ctrl+V
  - **SECONDARY** older, rarely used

  The selection itself is named by an Atom, for example

```
Atom clipboard = XInternAtom(display, "CLIPBOARD", False);
```

  A client becomes the owner of a selection with

```
XSetSelectionOwner(display, selection_atom, owner_window,
↪  timestamp);
```

  For example,

```
XSetSelectionOwner(display, clipboard, window, CurrentTime)
```

  You can ask who owns a selection with:

```
Window owner = XGetSelectionOwner(display, clipboard)
```

  Another client requests the selection using:

```
XConvertSelection(
    display,
    selection,
    target,
    property,
    requestor,
    time
);
```

This asks the selection owner:

"*Please convert your selection data to this target type and store it on this property of my window.*"

So the flow is

client A owns CLIPBOARD
↓
client B calls XConvertSelection
↓
client A receives SelectionRequest
↓
client A writes data to a property on client B's window
↓
client A sends SelectionNotify
↓
client B reads the property
.

This is why selections are more like negotiated inter-client data transfer than a simple global clipboard buffer.

The main events are

```
SelectionClear
SelectionRequest
SelectionNotify
```

  - **SelectionClear**: You lost ownership of the selection.
  - **SelectionRequest**: Another client is asking you for the selection data.
  - **SelectionNotify**: The selection conversion request completed.

A correct clipboard owner must handle **SelectionRequest.** If it exits, the selected data may disappear unless a clipboard manager has saved it.

- **Grabs**: A grab gives one client temporary exclusive control over some input.

There are two major kinds:

1. Pointer grabs
2. Keyboard grabs

A grab can be **active** or **passive**.

- **Active**: Happens immediately when you request it
- **Passive**: A passive grab does not take effect immediately. Instead, it activates when some input pattern occurs

For the pointer:

```
int XGrabPointer(
    Display* display,
    Window grab_window,
    bool owner_events,
    unsigned int event_mask,
    int pointer_mode,
    int keyboard_mode,
    Window confine_to,
    Cursor cursor,
    Time time
);
```

- display Specifies the connection to the X server.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.
- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync.**
- confine_to Specifies the window to confine the pointer in or None.
- cursor Specifies the cursor that is to be displayed during the grab or None.
- time Specifies the time. You can pass either a timestamp or CurrentTime

The XGrabPointer() function actively grabs control of the pointer and returns GrabSuccess if the grab was successful. Further pointer events are reported only to the grabbing client. XGrabPointer() can generate **BadCursor, BadValue,** and **BadWindow** errors.

For the keyboard:

```
int XGrabKeyboard(
    Display* display,
    Window grab_window,
    Bool owner_events,
    int pointer_mode,
    int keyboard_mode,
    Time time
);
```

- display Specifies the connection to the X server.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
- keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

The **XGrabKeyboard()** function actively grabs control of the keyboard and generates **FocusIn** and **FocusOut** events. Further key events are reported only to the grabbing client.

Once a client has an active pointer grab, pointer events are directed according to the grab rules rather than normal window picking. Typical uses are

- menus
- drag-and-drop
- resizing a window
- moving a window
- modal interactions
- screen lockers

For example, while dragging a scrollbar, you do not want pointer events to suddenly go to another window just because the pointer moved outside the scrollbar. A pointer grab lets the scrollbar continue receiving motion/release events until the drag finishes.

You release grabs with:

```
XUngrabPointer(display, time)
XUngrabKeyboard(display, time)
```

A passive grab does not take effect immediately. Instead, it activates when some input pattern occurs.

For keyboard shortcuts:

```
int XGrabKey(
    Display* display,
    int keycode,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    int pointer_mode,
    int keyboard_mode
);
```

- display Specifies the connection to the X server.
- keycode Specifies the KeyCode or **AnyKey.**
- modifiers Specifies the set of keymasks or **AnyModifier.** The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync**

The **XGrabKey()** function establishes a passive grab on the keyboard. **XGrabKey()** can generate **BadAccess** , **BadValue** , and **BadWindow** errors.

For mouse buttons:

```
int XGrabButton(
    Display* display,
    unsigned int button,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    unsigned int event_mask,
    int pointer_mode,
    int keyboard_mode,
    Window confine_to,
    Cursor cursor
);
```

- display Specifies the connection to the X server.
- button Specifies the pointer button that is to be grabbed or **AnyButton.**
- modifiers Specifies the set of keymasks or **AnyModifier.** The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.

- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync.**
- confine_to Specifies the window to confine the pointer in or None.
- cursor Specifies the cursor that is to be displayed or None.

The XGrabButton() function establishes a passive grab. **XGrabButton()** can generate **BadCursor, BadValue,** and **BadWindow** errors.

Xlib defines constants for the standard mouse buttons:

- **Button1**: Typically the left mouse button.
- **Button2**: Typically the middle mouse button (or scroll wheel click).
- **Button3**: Typically the right mouse button.
- **Button4 / Button5**: Often assigned to the scroll wheel up/down actions.
- **AnyButton**: A special constant used in functions like XGrabButton to indicate that the operation should apply to any mouse button

This is how a window manager can say:

"*When Mod1 + Button1 is pressed over this window, give me control.*"

The grab becomes active only when the specified key/button/modifier combination occurs.

- **Keymask modifiers**: In Xlib, keymask modifiers are bitwise constants used to describe the state of modifier keys (like Shift, Control, or Alt) during an input event. These masks are commonly found in the state member of event structures such as XKeyEvent or used as arguments in functions like **XGrabKey.**
  - **ShiftMask**: The Shift key is pressed.
  - **LockMask**: The Caps Lock key is active.
  - **ControlMask**: The Control key is pressed.
  - **Mod1Mask**: Usually mapped to Alt or Meta.
  - **Mod2Mask**: Often mapped to Num Lock.
  - **Mod3Mask**: Sometimes mapped to AltGr.
  - **Mod4Mask**: Commonly mapped to the Super/Windows key.
  - **Mod5Mask**: Often mapped to Scroll Lock or other custom modifiers.
- **Synchronous vs asynchronous grabs**: Grabs can freeze or not freeze event processing. The modes are usually:

```
0 GrabModeAsync
1 GrabModeSync
```

With GrabModeAsync, events continue normally. With GrabModeSync, the server may freeze pointer or keyboard event processing until the grabbing client explicitly allows it to continue with:

```
int XAllowEvents(
    Display* display,
    int mode,
    Time time
);
```

- display Specifies the connection to the X server.
- event_mode Specifies the event mode. You can pass **AsyncPointer, SyncPointer, AsyncKeyboard, SyncKeyboard, ReplayPointer, ReplayKeyboard, AsyncBoth,** or **SyncBoth.**
- time Specifies the time. You can pass either a timestamp or CurrentTime.

The XAllowEvents() function releases some queued events if the client has caused a device to freeze. It has no effect if the specified time is earlier than the last-grab time of the most recent active grab for the client or if the specified time is later than the current X server time. Depending on the event_mode argument, the following occurs:

- **AsyncPointer**: If the pointer is frozen by the client, pointer event processing continues as usual. If the pointer is frozen twice by the client on behalf of two separate grabs, AsyncPointer thaws for both. AsyncPointer has no effect if the pointer is not frozen by the client, but the pointer need not be grabbed by the client.
- **SyncPointer**: If the pointer is frozen and actively grabbed by the client, pointer event processing continues as usual until the next ButtonPress or ButtonRelease event is reported to the client. At this time, the pointer again appears to freeze. However, if the reported event causes the pointer grab to be released, the pointer does not freeze. SyncPointer has no effect if the pointer is not frozen by the client or if the pointer is not grabbed by the client.
- **ReplayPointer**: If the pointer is actively grabbed by the client and is frozen as the result of an event having been sent to the client (either from the activation of a XGrabButton() or from a previous XAllowEvents() with mode SyncPointer but not from a XGrabPointer()), the pointer grab is released and that event is completely reprocessed. This time, however, the function ignores any passive grabs at or above (towards the root of) the grab_window of the grab just released. The request has no effect if the pointer is not grabbed by the client or if the pointer is not frozen as the result of an event.
- **AsyncKeyboard**: If the keyboard is frozen by the client, keyboard event processing continues as usual. If the keyboard is frozen twice by the client on behalf of two separate grabs, AsyncKeyboard thaws for both. AsyncKeyboard has no effect if the keyboard is not frozen by the client, but the keyboard need not be grabbed by the client.
- **SyncKeyboard**: If the keyboard is frozen and actively grabbed by the client, keyboard event processing continues as usual until the next KeyPress or KeyRelease event is reported to the client. At this time, the keyboard again appears to freeze. However, if the reported event causes the keyboard grab to be released, the keyboard does not freeze. SyncKeyboard has no effect if the keyboard is not frozen by the client or if the keyboard is not grabbed by the client.

- **ReplayKeyboard**: If the keyboard is actively grabbed by the client and is frozen as the result of an event having been sent to the client (either from the activation of a XGrabKey() or from a previous XAllowEvents() with mode SyncKeyboard but not from a XGrabKeyboard()), the keyboard grab is released and that event is completely reprocessed. This time, however, the function ignores any passive grabs at or above (towards the root of) the grab_window of the grab just released. The request has no effect if the keyboard is not grabbed by the client or if the keyboard is not frozen as the result of an event.
- **SyncBoth**: If both pointer and keyboard are frozen by the client, event processing for both devices continues as usual until the next ButtonPress, ButtonRelease, KeyPress, or KeyRelease event is reported to the client for a grabbed device (button event for the pointer, key event for the keyboard), at which time the devices again appear to freeze. However, if the reported event causes the grab to be released, then the devices do not freeze (but if the other device is still grabbed, then a subsequent event for it will still cause both devices to freeze). SyncBoth has no effect unless both pointer and keyboard are frozen by the client. If the pointer or keyboard is frozen twice by the client on behalf of two separate grabs, SyncBoth thaws for both (but a subsequent freeze for SyncBoth will only freeze each device once).
- **AsyncBoth**: If the pointer and the keyboard are frozen by the client, event processing for both devices continues as usual. If a device is frozen twice by the client on behalf of two separate grabs, AsyncBoth thaws for both. AsyncBoth has no effect unless both pointer and keyboard are frozen by the client.

This is useful when a client wants to inspect an event before deciding whether it should be replayed or swallowed.

XAllowEvents() can generate a **BadValue** error.

- **Grab result codes**: A grab can fail, for example

```
int status = XGrabPointer(...);
```

Possible result values include:

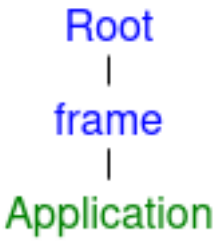
```
GrabSuccess
AlreadyGrabbed
GrabInvalidTime
GrabNotViewable
GrabFrozen
```

So a grab is not guaranteed.

- **Save-set**: A save-set is a mechanism used mainly by window managers to avoid destroying or orphaning client windows if the window manager exits. This matters because window managers often reparent client windows.

Normally, an application creates a top-level window as a child of the root window:

A reparenting window manager inserts a frame window around it:



The application window is now a child of a window owned by the window manager. Now, we have a problem...

If the window manager dies, what happens to the application windows inside its frame windows?

Without protection, those client windows could be destroyed or left in a bad state when the frame windows disappear. So the window manager adds application windows to its save-set:

```
XAddToSaveSet(display, client_window)
```

If the window manager exits or its connection to the X server closes, the server automatically reparents those save-set windows, usually back to a safe ancestor such as the root, and maps them if appropriate.

You can remove a window from the save-set with

```
XRemoveFromSaveSet(display, client_window);
```

- **Superior and inferior**: A window's **inferiors** are its descendants. A window's **superiors** are its ancestors.
- **Using X Server Connection Close Operations**: When the X server's connection to a client is closed either by an explicit call to **XCloseDisplay** or by a process that exits, the X server performs the following automatic operations:
  - It disowns all selections owned by the client (see **XSetSelectionOwner).**

- It performs an **XUngrabPointer** and **XUngrabKeyboard** if the client has actively grabbed the pointer or the keyboard.
- It performs an **XUngrabServer** if the client has grabbed the server.
- It releases all passive grabs made by the client.
- It marks all resources (including colormap entries) allocated by the client either as permanent or temporary, depending on whether the close-down mode is **RetainPermanent** or **RetainTemporary.** However, this does not prevent other client applications from explicitly destroying the resources (see **XSetCloseDownMode).**

When the close-down mode is **DestroyAll,** the X server destroys all of a client's resources as follows:

- It examines each window in the client's save-set to determine if it is an inferior (subwindow) of a window created by the client. (The save-set is a list of other clients' windows that are referred to as save-set windows.) If so, the X server reparents the save-set window to the closest ancestor so that the save-set window is not an inferior of a window created by the client. The reparenting leaves unchanged the absolute coordinates (with respect to the root window) of the upper-left outer corner of the save-set window.
- It performs a **MapWindow** request on the save-set window if the save-set window is unmapped. The X server does this even if the save-set window was not an inferior of a window created by the client.
- It destroys all windows created by the client.
- It performs the appropriate free request on each nonwindow resource created by the client in the server (for example, **Font, Pixmap, Cursor, Colormap,** and **GContext).**
- It frees all colors and colormap entries allocated by a client application

Additional processing occurs when the last connection to the X server closes. An X server goes through a cycle of having no connections and having some connections. When the last connection to the X server closes as a result of a connection closing with the close_mode of **DestroyAll,** the X server does the following:

- It resets its state as if it had just been started. The X server begins by destroying all lingering resources from clients that have terminated in **RetainPermanent** or **RetainTemporary** mode.
- It deletes all but the predefined atom identifiers.
- It deletes all properties on all root windows (see section 4.3).
- It resets all device maps and attributes (for example, key click, bell volume, and acceleration) as well as the access control list.
- It restores the standard root tiles and cursors.
- It restores the default font path.
- It restores the input focus to state **PointerRoot**

However, the X server does not reset if you close a connection with a close-down mode set to **RetainPermanent** or **RetainTemporary**

- **Locking a display**: To lock a display in Xlib means take a mutex-like lock on the `Display*` so only one thread in your process uses that X connection at a time.

```
XLockDisplay(display)
```

While the display is locked, other threads in the same process that try to use the same Display * will block until it is unlocked.

```
XUnlockDisplay(display)
```

- **Using Xlib with threads**: To initialize support for concurrent threads, use **XInitThreads**

```
Status XInitThreads();
```

The **XInitThreads** function initializes Xlib support for concurrent threads. This function must be the first Xlib function a multi-threaded program calls, and it must complete before any other Xlib call is made. This function returns a nonzero status if initialization was successful; otherwise, it returns zero. On systems that do not support threads, this function always returns zero.

It is only necessary to call this function if multiple threads might use Xlib concurrently. If all calls to Xlib functions are protected by some other access mechanism (for example, a mutual exclusion lock in a toolkit or through explicit client programming), Xlib thread initialization is not required. It is recommended that single-threaded programs not call this function.

To lock a display across several Xlib calls, use **XLockDisplay**

```
void XLockDisplay(display)
```

The **XLockDisplay** function locks out all other threads from using the specified display. Other threads attempting to use the display will block until the display is unlocked by this thread. Nested calls to **XLockDisplay** work correctly; the display will not actually be unlocked until **XUnlockDisplay** has been called the same number of times as **XLockDisplay.** This function has no effect unless Xlib was successfully initialized for threads using **XInitThreads.**

To unlock a display, use XUnlockDisplay

```
void XUnlockDisplay(display)
```

The **XUnlockDisplay** function allows other threads to use the specified display again. Any threads that have blocked on the display are allowed to continue. Nested locking works correctly; if **XLockDisplay** has been called multiple times by a thread, then **XUnlockDisplay** must be called an equal number of times before the display is actually unlocked. This function has no effect unless Xlib was successfully initialized for threads using **XInitThreads.**

- **Using Internal Connections**: In addition to the connection to the X server, an Xlib implementation may require connections to other kinds of servers. Toolkits and clients that use multiple displays, or that use displays in combination with other inputs, need to obtain these additional connections to correctly block until input is available and need to process that input when it is available. Simple clients that use a single display and block for input in an Xlib event function do not need to use these facilities.

  To track internal connections for a display, use **XAddConnectionWatch.**

```
typedef void (*XConnectionWatchProc) (Display* display,
    ↪ XPointer client_data, int fd, Bool opening, XPointer
    ↪ watch_data)

Status XAddConnectionWatch (Display* display,
    ↪ XConnectionWatchProc procedure, XPointer client_data)
```

  – procedure Specifies the procedure to be called.
  – client_data Specifies the additional client data.

  The **XAddConnectionWatch** function registers a procedure to be called each time Xlib opens or closes an internal connection for the specified display. The procedure is passed the display, the specified client_data, the file descriptor for the connection, a Boolean indicating whether the connection is being opened or closed, and a pointer to a location for private watch data. If opening is **True,** the procedure can store a pointer to private data in the location pointed to by watch_data; when the procedure is later called for this same connection and opening is **False**, the location pointed to by watch_data will hold this same private data pointer.

  This function can be called at any time after a display is opened. If internal connections already exist, the registered procedure will immediately be called for each of them, before **XAddConnectionWatch** returns. **XAddConnectionWatch** returns a nonzero status if the procedure is successfully registered; otherwise, it returns zero

  The registered procedure should not call any Xlib functions. If the procedure directly or indirectly causes the state of internal connections or watch procedures to change, the result is not defined. If Xlib has been initialized for threads, the procedure is called with the display locked and the result of a call by the procedure to any Xlib function that locks the display is not defined unless the executing thread has externally locked the display using **XLockDisplay**

  To stop tracking internal connections for a display, use **XRemoveConnectionWatch.**

```
Status XRemoveConnectionWatch (display, procedure,
    ↪ client_data)
```

The **XRemoveConnectionWatch** function removes a previously registered connection watch procedure. The client_data must match the client_data used when the procedure was initially registered.

To process input on an internal connection, use **XProcessInternalConnection**

```
void XProcessInternalConnection(Display* display, int fd)
```

The **XProcessInternalConnection** function processes input available on an internal connection. This function should be called for an internal connection only after an operating system facility (for example, select or poll) has indicated that input is available; otherwise, the effect is not defined.

To obtain all of the current internal connections for a display, use **XInternalConnectionNumbers.**

```
Status XInternalConnectionNumbers(Display* display, int**
↪  fd_return, int* count_return)
```

The **XInternalConnectionNumbers** function returns a list of the file descriptors for all internal connections currently open for the specified display. When the allocated list is no longer needed, free it by using **XFree.** This functions returns a nonzero status if the list is successfully allocated; otherwise, it returns zero.

3. Window functions

- **Intro**: In the *X* Window System, a window is a rectangular area on the screen that lets you view graphic output. Client applications can display overlapping and nested windows on one or more screens that are driven by *X* servers on one or more machines. Clients who want to create windows must first connect their program to the *X* server by calling **XOpenDisplay.** This chapter begins with a discussion of visual types and window attributes. The chapter continues with a discussion of the Xlib functions you can use to:
  - Create windows
  - Destroy windows
  - Map windows
  - Unmap windows
  - Configure windows
  - Change window stacking order
  - Change window attributes
- **ICCCM / EWMH conventions**:
  - **ICCCM (Inter-client communication conventions)**: The Inter-Client Communication Conventions Manual (ICCCM) is a foundational standard protocol for the X Window System. It establishes the rules that graphical applications (clients) must follow to cooperate, share resources (like cut and paste selections), and interact smoothly with window managers.

    The ICCCM focuses on a few core areas of client behavior to ensure a stable and predictable desktop experience:
    * **Selections and Cut Buffers**: Defines how clients share data across the global X server, enabling universal copy and paste operations between different applications.
    * **Window Manager Communication**: Establishes the rules for how applications request window sizes, positions, and decorations, ensuring that a window manager can successfully control top-level windows without relying on private protocols.
    * **Session Management**: Provides protocols for applications to interact with session managers, allowing users to safely save their workspace state, restart programs, or shut down without losing functionality.
    * **Shared Resources**: Details the conventions for sharing input focus and color maps, ensuring that apps don't compete or lock each other out of basic system resources.
  - **EWMH (Extended window manager hints)**: EWMH stands for Extended Window Manager Hints. It is a standard for the X Window System that dictates how desktop environments, window managers, and applications communicate with one another. It builds upon older standards to let modern apps request specific behaviors (like going fullscreen or creating custom docks).

    The foundational standard for X11 communication (ICCCM) is notoriously complex and lacks the features we expect from modern operating systems, such as virtual desktops, docks, and window tabbing. EWMH bridges this gap by defining properties and messages (called "hints") that allow programs and the window manager to understand each other perfectly.

Under EWMH, the window manager exposes specific pieces of information on the root window, and applications use those to request states or react to desktop events. Key functions include:

* **Window Types & States**: Apps can declare what they are (e.g., a normal app, a dock, a splash screen, or a utility tool) or request states like _NET_WM_STATE_FULLSCREEN or _NET_WM_STATE_MODAL.
* **Virtual Desktops**: It defines protocols for managing workspaces, including how many virtual screens exist and which desktop a specific window lives on.
* **Window Tabbing & Grouping**: It specifies how windows can be logically grouped together (often seen in advanced tiling or tabbed window managers).
* **Desktop Properties**: It standardizes the retrieval of desktop dimensions, work areas (reserving space for taskbars), and active window tracking.

Developers working directly with window operations rely on specific EWMH messages (denoted by the **_NET** prefix):

* **_NET_ACTIVE_WINDOW**: Tells the window manager to give focus to a specific window.
* **_NET_CLOSE_WINDOW**: Requests a graceful closure of an application window.
* **_NET_WM_PING**: Allows the window manager to check if an application has stopped responding.

Note: EWMH is not standalone in the strict compliance sense. The EWMH spec explicitly says it builds on ICCCM, and that window managers and clients that aim to fulfill EWMH must also adhere to ICCCM, except where EWMH explicitly modifies ICCCM behavior.

- **Opaque structures**: An opaque struct is a design pattern in C where a structure is declared but its internal fields are hidden from the user. The user only interacts with the struct via pointers and a provided set of API functions, making it the primary way to achieve encapsulation and object-oriented ”data hiding” in C.

  A **transparent (exposed) struct** is the opposite. The fields are fully visible to the user.

- **Visual types**: On some display hardware, it may be possible to deal with color resources in more than one way. For example, you may be able to deal with a screen of either 12-bit depth with arbitrary mapping of pixel to color (pseudo-color) or 24-bit depth with 8 bits of the pixel dedicated to each of red, green, and blue. These different ways of dealing with the visual aspects of the screen are called visuals. For each screen of the display, there may be a list of valid visual types supported at different depths of the screen. Because default windows and visual types are defined for each screen, most simple applications need not deal with this complexity.

  Xlib uses an opaque Visual structure that contains information about the possible color mapping. The visual utility functions use an **XVisualInfo** structure to return this information to an application. The members of this structure pertinent to this discussion are class, red_mask, green_mask, blue_mask, bits_per_rgb, and colormap_size. The class member specified one of the possible visual classes of the screen and can be **StaticGray, StaticColor, TrueColor, GrayScale, PseudoColor,** or **DirectColor.**

A pixel value in X11 is usually just an integer. The Visual tells Xlib and the X server what that integer means. For example, does the integer directly encode red/green/blue components? Or is it an index into a colormap? Are the colors fixed, or can the client modify them?

The **Visual** describes the color interpretation model available for windows and pixmaps of a certain depth.

```
typedef struct {
    Visual *visual;
    VisualID visualid;
    int screen;
    int depth;
    int class;
    unsigned long red_mask;
    unsigned long green_mask;
    unsigned long blue_mask;
    int colormap_size;
    int bits_per_rgb;
} XVisualInfo;
```

There is no direct "convert Visual* to XVisualInfo" function, but you can get the VisualID from the Visual *, then ask Xlib for the matching XVisualInfo.

```
VisualID XVisualIDFromVisual(Visual* visual);
XVisualInfo* XGetVisualInfo(
    Display* display,
    long vinfo_mask,
    XVisualInfo* vinfo_template,
    int* nitems_return
);
```

- vinfo_mask Specifies the visual mask value.
- vinfo_template Specifies the visual attributes that are to be used in matching the visual structures.
- nitems_return Returns the number of matching visual structures.

Regarding the vinfo_mask: In Xlib, vinfo_mask is a bitwise mask value used as an argument in the XGetVisualInfo function to specify which fields of an XVisualInfo template structure the X server should evaluate when searching for available window visuals. When you use XGetVisualInfo, you pass a template structure containing the display criteria you want. The vinfo_mask tells the function exactly which of those template fields matter for your query

- **VisualNoMask**: No fields are matched (returns all available visuals)
- **VisualIDMask**: Matches the explicit visualid
- **IDVisualScreenMask**: Matches the specified screen
- **numberVisualDepthMask**: Matches the pixel bit depth (e.g., 24-bit, 32-bit color)

- **VisualClassMask**: Matches the color class (e.g., TrueColor, PseudoColor)
- **VisualRedMaskMask**: Matches the red_mask bit
- **positionsVisualGreenMaskMask**: Matches the green_mask bit
- **positionsVisualBlueMaskMask**: Matches the blue_mask bit
- **positionsVisualColormapSizeMask**: Matches the total entries in the colormap_size
- **VisualBitsPerRGBMask**: Matches the bits_per_rgb value
- **VisualAllMask**: Combines all masks to check every field in the structure

For example,

```
XVisualInfo template;
template.screen = screen;
template.depth = 32;
template.class = TrueColor;

int nitems;
XVisualInfo *matches = XGetVisualInfo(
    dpy,
    VisualScreenMask  VisualDepthMask  VisualClassMask,
    &template,
    &nitems
);
```

- **Class**: The class field tells you what kind of color model the visual uses. There are six visual classes
  1. **StaticGray**: A StaticGray visual represents fixed grayscale colors. The pixel value selects a gray value from a read-only colormap.

     You cannot change what gray value a pixel maps to. This is mostly relevant to old or limited displays.
  2. **GrayScale**: A GrayScale visual is like StaticGray, except the colormap entries can be changed. The colors are grayscale, but the client may allocate or change entries.
  3. **StaticColor**: A StaticColor visual uses a fixed color colormap. The pixel value indexes into a predefined table of colors. The client cannot redefine those colors.
  4. **PseudoColor**: A PseudoColor visual uses a modifiable colormap. For example, pixel value 42 does not inherently mean red or blue. It means "look at entry 42 in the colormap."

     The client may allocate/change colormap entries, depending on availability. This was common on older 8-bit indexed-color displays.
  5. **TrueColor**: A TrueColor visual means the pixel value directly contains RGB components. For example, with a common 24-bit / 32-bit visual.

     [pixel bits]: [[red bits] [green bits] [blue bits]]

     The masks say where the components are stored. For example,

     ```
     red_mask   = 0x00ff0000
     green_mask = 0x0000ff00
     blue_mask  = 0x000000ff
     ```

So a pixel value like

$$0x00ff0000.$$

means full red, zero green, zero blue.

In TrueColor, the mapping from component value to actual intensity is fixed. You cannot redefine what "red component 255" means. This is the usual modern visual class.

6. **DirectColor**: DirectColor is similar to TrueColor because the pixel value contains separate red, green, and blue fields.

   But instead of those fields directly giving final intensities, each component indexes into a separate modifiable lookup table.

The main distinction is

* **Static* classes**: colormap entries are fixed/read-only
* **Non-static classes**: colormap entries can be changed by the client
* **TrueColor / DirectColor**: pixel value contains separate RGB component fields
* **PseudoColor / StaticColor / GrayScale / StaticGray**: pixel value indexes into a colormap

– **red_mask, green_mask, blue_mask**: Described above in the *class* explanation.

– **bits_per_rgb**: Tells you the number of significant bits used for each RGB value in the colormap. For example, if

```
bits_per_rgb = 8;
```

then each color component has 8 meaningful bits, giving values from 0 to 255.

– **colormap_size**: The number of entries available in the colormap for that visual.

For example, an 8-bit PseudoColor visual might have:

```
colormap_size = 256;
```

because 8-bit pixels can index 256 possible entries.

For TrueColor, the meaning is slightly different. It usually refers to the number of entries per primary color map, not the total number of possible RGB colors.

So for a 24-bit TrueColor visual with 8 bits per channel, you might see:

```
colormap_size = 256;
```

Even though the visual can represent approximately

$$256^3 = 16777216$$

colors.

Note: An X11 client cannot define a custom `Visual`. A Visual is advertised by the X server as part of the screen's capabilities. When your program connects to the X server, the server tells the client which visuals exist for each screen. Your program can then choose from those visuals, but it cannot create a new one.

- **Top-level windows**: A top-level window is any window that is a direct subwindow of your screen's root window and is independently managed by your desktop's window manager.
- **Window attributes**: All **InputOutput** windows have a border width of zero or more pixels, an optional background, an event suppression mask (which suppresses propagation of events from children), and a property list. The window border and background can be a solid color or a pattern, called a tile. All windows except the root have a parent and are clipped by their parent. If a window is stacked on top of another window, it obscures that other window for the purpose of input.

  If a window has a background (almost all do), it obscures the other window for purposes of output. Attempts to output to the obscured area do nothing, and no input events (for example, pointer motion) are generated for the obscured area.

  Both **InputOutput** and **InputOnly** windows have the following common attributes, which are the only attributes of an **InputOnly** window:

  – win-gravity
  – event-mask
  – do-not-propagate-mask
  – override-redirect
  – cursor

  If you specify any other attributes for an **InputOnly** window, a **BadMatch** error results.

  InputOnly windows are used for controlling input events in situations where **InputOutput** windows are unnecessary. **InputOnly** windows are invisible; can only be used to control such things as cursors, input event generation, and grabbing; and cannot be used in any graphics requests. Note that **InputOnly** windows cannot have **InputOutput** windows as inferiors

  Windows have borders of a programmable width and pattern as well as a background pattern or tile. Pixel values can be used for solid colors. The background and border pixmaps can be destroyed immediately after creating the window if no further explicit references to them are to be made. The pattern can either be relative to the parent or absolute. If **ParentRelative,** the parent's background is used.

  When windows are first created, they are not visible (not mapped) on the screen. Any output to a window that is not visible on the screen and that does not have backing store will be discarded. An application may wish to create a window long before it is mapped to the screen. When a window is eventually mapped to the screen (using **XMapWindow),** the X server generates an Expose event for the window if backing store has not been maintained.

A window manager can override your choice of size, border width, and position for a top-level window. Your program must be prepared to use the actual size and position of the top window. It is not acceptable for a client application to resize itself unless in direct response to a human command to do so. Instead, either your program should use the space given to it, or if the space is too small for any useful work, your program might ask the user to resize the window. The border of your top-level window is considered fair game for window managers.

To set an attribute of a window, set the appropriate member of the **XSetWindowAttributes** structure and OR in the corresponding value bitmask in your subsequent calls to **XCreateWindow** and **XChangeWindowAttributes,** or use one of the other convenience functions that set the appropriate attribute. The symbols for the value mask bits and the **XSetWindowAttributes** structure are:

```
0  /* Window attribute value mask bits */
1  #define CWBackPixmap        (1L<<0)
2  #define CWBackPixel         (1L<<1)
3  #define CWBorderPixmap      (1L<<2)
4  #define CWBorderPixel       (1L<<3)
5  #define CWBitGravity        (1L<<4)
6  #define CWWinGravity        (1L<<5)
7  #define CWBackingStore      (1L<<6)
8  #define CWBackingPlanes     (1L<<7)
9  #define CWBackingPixel      (1L<<8)
10 #define CWOverrideRedirect  (1L<<9)
11 #define CWSaveUnder         (1L<<10)
12 #define CWEventMask         (1L<<11)
13 #define CWDontPropagate     (1L<<12)
14 #define CWColormap          (1L<<13)
15 #define CWCursor            (1L<<14)
```

```c
typedef struct {
    Pixmap background_pixmap;       /* background, None, or
    ↪  ParentRelative */
    unsigned long background_pixel; /* background pixel */
    Pixmap border_pixmap;           /* border of the window
    ↪  or CopyFromParent */
    unsigned long border_pixel;     /* border pixel value */
    int bit_gravity;                /* one of bit gravity
    ↪  values */
    int win_gravity;                /* one of the window
    ↪  gravity values */
    int backing_store;              /* NotUseful, WhenMapped,
    ↪  Always */
    unsigned long backing_planes;   /* planes to be preserved
    ↪  if possible */
    unsigned long backing_pixel;    /* value to use in
    ↪  restoring planes */
    Bool save_under;                /* should bits under be
    ↪  saved? (popups) */
    long event_mask;                /* set of events that
    ↪  should be saved */
    long do_not_propagate_mask;     /* set of events that
    ↪  should not propagate */
    Bool override_redirect;         /* boolean value for
    ↪  override_redirect */
    Colormap colormap;              /* color map to be
    ↪  associated with window */
    Cursor cursor;                  /* cursor to be displayed
    ↪  (or None) */
} XSetWindowAttributes;
```

- **background_pixmap**: Background_pixmap is a pixmap resource used as a tiled background. It can be:

```
None
ParentRelative
some_pixmap
```

  - **None:** None means the window has no automatic background. X will not repaint the background for you.
  - **ParentRelative**: ParentRelative means the window uses the parent window's background. This is only valid for windows with the same depth as the parent.
  - **real pixmap**: A real pixmap means the server tiles that pixmap across the window background.
- **long background_pixel**: background_pixel is simpler: it fills the background with one pixel value. The meaning of that pixel value depends on the window's colormap and visual. In a TrueColor visual, it usually encodes RGB bits. In PseudoColor, it is an index into a colormap.
- **border_pixmap**: border_pixmap uses a pixmap as the border pattern. It can also be CopyFromParent in some creation contexts.
- **long border_pixel**: border_pixel uses a single pixel value for the border color.

The border is the X window border specified by the border_width argument to XCreateWindow. Many modern window managers ignore or replace client-side borders for top-level windows because they draw their own decorations.

– **bit_gravity**: Bit gravity controls what happens to the contents of the window when the window is resized.

For example, suppose your window gets larger. Should the old pixels stay in the upper-left corner? Center? Bottom-right? Or should X discard them?

Common values include

```
0  ForgetGravity
1  NorthWestGravity
2  NorthGravity
3  NorthEastGravity
4  WestGravity
5  CenterGravity
6  EastGravity
7  SouthWestGravity
8  SouthGravity
9  SouthEastGravity
10 StaticGravity
```

So,

```
0  attrs.bit_gravity = NorthWestGravity;
```

This means that when the window is resized, X tries to keep the old pixel contents attached to the upper-left corner.

ForgetGravity means X does not preserve the old bits. You should repaint when you receive an Expose event.

In modern applications, you usually repaint explicitly instead of relying heavily on bit gravity.

– **win_gravity**: Window gravity controls what happens to a child window's position when its parent is resized.

```
0  attrs.win_gravity = SouthEastGravity;
```

means that when the parent resizes, *X* tries to keep the child attached to the parent's bottom-right region.

This matters mostly for child-window layout. It is not usually used as the primary layout mechanism in modern GUI toolkits.

A win-gravity of **UnmapGravity** is like **NorthWestGravity** (the window is not moved), except the child is also unmapped when the parent is resized, and an **UnmapNotify** event is generated.

– **backing_store**: Backing store asks the *X* server to preserve obscured or unmapped window contents, so that when the window becomes visible again, the server may restore pixels without requiring the client to repaint.

Possible values are

```
0 NotUseful
1 WhenMapped
2 Always
```

* NotUseful means you do not request backing store.
* WhenMapped means the server may preserve contents while the window is mapped.
* Always means the server should try to preserve contents more aggressively.

However, this is only a hint. The server is not required to honor it. Modern systems often ignore backing store or implement it differently because compositors already keep window buffers.

– **backing_planes**: backing_planes specifies which bit planes should be preserved.

– **backing_pixel**: backing_pixel gives a fallback pixel value for restoring planes that were not preserved.

Most modern Xlib programs simply handle Expose events and repaint instead of relying on backing store.

– **save_under**: save_under is a hint to the server that it should save the pixels underneath this window while it is visible.

This was intended for temporary popup windows, menus, tooltips, and similar small transient windows.

Like backing store, this is only a hint. The server may ignore it.

– **event_mask**: This selects which events your client wants to receive for this window.

Some common masks are

```
0  ExposureMask
1  KeyPressMask
2  KeyReleaseMask
3  ButtonPressMask
4  ButtonReleaseMask
5  PointerMotionMask
6  EnterWindowMask
7  LeaveWindowMask
8  StructureNotifyMask
9  SubstructureNotifyMask
10 SubstructureRedirectMask
11 PropertyChangeMask
12 FocusChangeMask
```

For example, if you do not select **KeyPressMask,** then your client will not receive **KeyPress** events for that window.

Some event masks are exclusive. For example, only one client can select **SubstructureRedirectMask** on a given window. That is how a window manager claims control over the root window's child-window management.

- **do_not_propagate_mask**: This controls which input events should not propagate upward to ancestor windows.

  In x, some events can propagate from a child window to its parent if no client selected them on the child.

  For example, suppose a button press occurs in a child window. If nobody selected **ButtonPressMask** on that child, the event may propagate to the parent.

  The **do_not_propagate_mask** can stop that propagation.
- **override_redirect**: This tells the X server whether the window manager should be bypassed for this window. If true, then the window manager normally does not manage the window. That means no decorations, no placement policy, no focus policy, no normal reparenting, and no window-manager intervention.
- **colormap**: This is the colormap associated with the window.

  A colormap maps pixel values to actual colors, depending on the visual class.

  For common TrueColor visuals, you usually use the default colormap:

  But if you create a window using a non-default visual, you often need to create and assign a matching colormap:

```
0 Colormap cmap = XCreateColormap(display, root, visual,
  ↪  AllocNone);
1 attrs.colormap = cmap;
```

  This is especially important when creating windows with 32-bit ARGB visuals for transparency. The window's visual and colormap must be compatible.
- **cursor**: This specifies the cursor shown when the pointer is inside the window.

  You can use a cursor created with functions like:

```
0 XCreateFontCursor
1 XCreatePixmapCursor
2 XcursorLibraryLoadCursor
```

  If the cursor is None, the cursor is inherited from the parent window.

The following lists the defaults for each window attribute and indicates whether the attribute is applicable to **InputOutput** and **InputOnly** windows:

Attribute	Default	InputOutput	InputOnly
background-pixmap	None	Yes	No
background-pixel	Undefined	Yes	No
border-pixmap	CopyFromParent	Yes	No
border-pixel	Undefined	Yes	No
bit-gravity	ForgetGravity	Yes	No
win-gravity	NorthWestGravity	Yes	Yes
backing-store	NotUseful	Yes	No
backing-planes	All ones	Yes	No
backing-pixel	zero	Yes	No
save-under	False	Yes	No
event-mask	empty set	Yes	Yes
do-not-propagate-mask	empty set	Yes	Yes
override-redirect	False	Yes	Yes
colormap	CopyFromParent	Yes	No
cursor	None	Yes	Yes

- **Background Attribute**: Only **InputOutput** windows can have a background. You can set the background of an **InputOutput** window by using a pixel or a pixmap.
- **Pixel centers**: In graphics, to **coincide with pixel centers** means that some geometric coordinate, such as a line, point, or sample location, falls exactly at the center of a pixel, not on the boundary between pixels.
- **Creating Windows**: Xlib provides basic ways for creating windows, and toolkits often supply higher-level functions specifically for creating and placing top-level windows, which are discussed in the appropriate toolkit documentation. If you do not use a toolkit, however, you must provide some standard information or hints for the window manager by using the Xlib inter-client communication functions

  If you use Xlib to create your own top-level windows (direct children of the root window), you must observe the following rules so that all applications interact reasonably across the different styles of window management:

  – You must never fight with the window manager for the size or placement of your top-level window.

  – You must be able to deal with whatever size window you get, even if this means that your application just prints a message like ‘‘Please make me bigger’’ in its window.

  – You should only attempt to resize or move top-level windows in direct response to a user request. If a request to change the size of a top-level window fails, you must be prepared to live with what you get. You are free to resize or move

the children of top-level windows as necessary. (Toolkits often have facilities for automatic relayout.)

– If you do not use a toolkit that automatically sets standard window properties, you should set these properties for top-level windows before mapping them.

XCreateWindow is the more general function that allows you to set specific window attributes when you create a window. **XCreateSimpleWindow** creates a window that inherits its attributes from its parent window.

The X server acts as if **InputOnly** windows do not exist for the purposes of graphics requests, exposure processing, and **VisibilityNotify** events. An **InputOnly** window cannot be used as a drawable (that is, as a source or destination for graphics requests). **InputOnly** and **InputOutput** windows act identically in other respects (properties, grabs, input control, and so on). Extension packages can define other classes of windows.

To create an unmapped window and set its window attributes, use **XCreateWindow**

```
Window XCreateWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    int depth,
    unsigned int class,
    Visual* visual,
    unsigned long valuemask,
    XSetWindowAttributes* attributes
)
```

– **display**: Specifies the connection to the X server.

– **parent**: Specifies the parent window.

– **x, y**: Specify the x and y coordinates, which are the top-left outside corner of the created window's borders and are relative to the inside of the parent window's borders.

– **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.

– **border_width**: Specifies the width of the created window's border in pixels.

– **depth**: Specifies the window's depth. A depth of **CopyFromParent** means the depth is taken from the parent.

– **class**: Specifies the created window's class. You can pass **InputOutput, InputOnly,** or **CopyFromParent.** A class of **CopyFromParent** means the class is taken from the parent.

– **visual**: Specifies the visual type. A visual of **CopyFromParent** means the visual type is taken from the parent.

– **valuemask**: Specifies which window attributes are defined in the attributes argument. This mask is the bitwise inclusive OR of the valid attribute mask bits. If valuemask is zero, the attributes are ignored and are not referenced.

– **attributes**: Specifies the structure from which the values (as specified by the value mask) are to be taken. The value mask should have the appropriate bits set to indicate which attributes have been set in the structure.

The **XCreateWindow** function creates an unmapped subwindow for a specified parent window, returns the window ID of the created window, and causes the X server to generate a **CreateNotify** event. The created window is placed on top in the stacking order with respect to siblings.

The coordinate system has the X axis horizontal and the Y axis vertical with the origin [0, 0] at the upper-left corner. Coordinates are integral, in terms of pixels, and coincide with pixel centers. Each window and pixmap has its own coordinate system. For a window, the origin is inside the border at the inside, upper-left corner.

The border_width for an **InputOnly** window must be zero, or a **BadMatch** error results. For class **InputOutput,** the visual type and depth must be a combination supported for the screen, or a **BadMatch** error results. The depth need not be the same as the parent, but the parent must not be a window of class **InputOnly,** or a **BadMatch** error results. For an **InputOnly** window, the depth must be zero, and the visual must be one supported by the screen. If either condition is not met, a **BadMatch** error results. The parent window, however, may have any depth and class. If you specify any invalid window attribute for a window, a **BadMatch** error results.

- **Creating simple windows**: The created window is not yet displayed (mapped) on the user's display. To display the window, call **XMapWindow.** The new window initially uses the same cursor as its parent. A new cursor can be defined for the new window by calling **XDefineCursor.** The window will not be visible on the screen unless it and all of its ancestors are mapped and it is not obscured by any of its ancestors.

XCreateWindow can generate **BadAlloc, BadColor, BadCursor, BadMatch, BadPixmap, BadValue,** and **BadWindow** errors.

To create an unmapped **InputOutput** subwindow of a given parent window, use **XCreateSimpleWindow**

```
Window XCreateSimpleWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    unsigned long border,
    unsigned long background
)
```

– **display**: Specifies the connection to the X server.

– **parent**: Specifies the parent window.

– x**, y**: Specify the x and y coordinates, which are the top-left outside corner of the new window's borders and are relative to the inside of the parent window's borders.

- **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.
- **border_width**: Specifies the width of the created window's border in pixels.
- **border**: Specifies the border pixel value of the window.
- **background**: Specifies the background pixel value of the window.

The **XCreateSimpleWindow** function creates an unmapped **InputOutput** subwindow for a specified parent window, returns the window ID of the created window, and causes the X server to generate a **CreateNotify** event. The created window is placed on top in the stacking order with respect to siblings. Any part of the window that extends outside its parent window is clipped. The border_width for an **InputOnly** window must be zero, or a **BadMatch** error results. **XCreateSimpleWindow** inherits its depth, class, and visual from its parent. All other window attributes, except background and border, have their default values.

XCreateSimpleWindow can generate **BadAlloc, BadMatch, BadValue,** and **BadWindow** errors.

- **Destroying Windows**: Xlib provides functions that you can use to destroy a window or destroy all subwindows of a window.

  To destroy a window and all of its subwindows, use **XDestroyWindow.**

```
XDestroyWindow(Display* display, Window w)
```

The **XDestroyWindow** function destroys the specified window as well as all of its subwindows and causes the X server to generate a **DestroyNotify** event for each window. The window should never be referenced again. If the window specified by the w argument is mapped, it is unmapped automatically. The ordering of the **DestroyNotify** events is such that for any given window being destroyed, **DestroyNotify** is generated on any inferiors of the window before being generated on the window itself. The ordering among siblings and across **subhierarchies** is not otherwise constrained. If the window you specified is a root window, no windows are destroyed. Destroying a mapped window will generate Expose events on other windows that were obscured by the window being destroyed.

XDestroyWindow can generate a **BadWindow** error

To destroy all subwindows of a specified window, use **XDestroySubwindows**

```
XDestroySubwindows(Display* display, Window w)
```

The **XDestroySubwindows** function destroys all inferior windows of the **specified** window, in bottom-to-top stacking order. It causes the X server to generate a **DestroyNotify** event for each window. If any mapped subwindows were actually destroyed, **XDestroySubwindows** causes the X server to generate **Expose** events on the specified window. This is much more efficient than deleting many windows one at a time because much of the work need be performed only once for all of the windows, rather than for each window. The subwindows should never be referenced again.

XDestroySubwindows can generate a **BadWindow** error.

- **Window stacking order**: The window stacking order is the front-to-back ordering of sibling windows on the screen.

  A window can have many child windows. Among those children, X keeps an ordering that determines which windows appear above or below the others when they overlap.

  Think of it like a stack of sheets of paper:

  Topmost window
  ↑
  |
  Window C
  Window B
  Window A
  |
  ↓
  Bottommost window.

  If Window C overlaps Window B, and Window C is higher in the stacking order, then Window C visually covers that part of Window B.

  The important point is that stacking order is defined relative to windows with the same parent.

```
Root window
    |-- A
    |-- B
    |-- C
```

  Here, A, B, and C are siblings. Their stacking order determines which one appears above the others when they overlap.

  But, if A has children:

```
Root window
 |--A
 |  |--A1
 |  |--A2
 |--B
 |--C
```

  Then A_1 and A_2 have their own stacking order relative to each other. They are not directly stacked against B and C; they are inside the subtree of A.

  If two sibling windows overlap, the one higher in the stack obscures the lower one.

Top-level application windows are usually children of the root window. So the stacking order of top-level windows is the stacking order of the root window's children.

A window manager manipulates this order constantly. When you focus a window, open a menu, raise a dialog, or move a window to the front, the window manager changes the stacking order.

- **Mapping windows**: A window is considered mapped if an **XMapWindow** call has been made on it. It may not be visible on the screen for one of the following reasons:
  - It is obscured by another opaque window.
  - One of its ancestors is not mapped.
  - It is entirely clipped by an ancestor.

  Expose events are generated for the window when part or all of it becomes visible on the screen. A client receives the Expose events only if it has asked for them. Windows retain their position in the stacking order when they are unmapped.

  A window manager may want to control the placement of subwindows. If **SubstructureRedirectMask** has been selected by a window manager on a parent window (usually a root window), a map request initiated by other clients on a child window is not performed, and the window manager is sent a **MapRequest** event. However, if the override-redirect flag on the child had been set to True (usually only on pop-up menus), the map request is performed.

  A tiling window manager might decide to reposition and resize other clients' windows and then decide to map the window to its final location. A window manager that wants to provide decoration might reparent the child into a frame first. Only a single client at a time can select for **SubstructureRedirectMask.**

  Similarly, a single client can select for **ResizeRedirectMask** on a parent window. Then, any attempt to resize the window by another client is suppressed, and the client receives a **ResizeRequest** event.

  To map a given window, use **XMapWindow**

```
0  XMapWindow(display, w)
```

  The **XMapWindow** function maps the window and all of its subwindows that have had map requests. Mapping a window that has an unmapped ancestor does not display the window but marks it as eligible for display when the ancestor becomes mapped. Such a window is called **unviewable.** When all its ancestors are mapped, the window becomes viewable and will be visible on the screen if it is not obscured by another window. This function has no effect if the window is already mapped.

  If the override-redirect of the window is **False** and if some other client has selected **SubstructureRedirectMask** on the parent window, then the X server generates a **MapRequest** event, and the **XMapWindow** function does not map the window. Otherwise, the window is mapped, and the X server generates a **MapNotify** event.

If the window becomes viewable and no earlier contents for it are remembered, the X server tiles the window with its background. If the window's background is undefined, the existing screen contents are not altered, and the X server generates zero or more **Expose** events. If backing-store was maintained while the window was unmapped, no **Expose** events are generated. If backing-store will now be maintained, a full-window exposure is always generated. Otherwise, only visible regions may be reported. Similar tiling and exposure take place for any newly viewable inferiors.

If the window is an InputOutput window, **XMapWindow** generates Expose events on each InputOutput window that it causes to be displayed. If the client maps and paints the window and if the client begins processing events, the window is painted twice. To avoid this, first ask for Expose events and then map the window, so the client processes input events as usual. The event list will include Expose for each window that has appeared on the screen. The client's normal response to an Expose event should be to repaint the window. This method usually leads to simpler programs and to proper interaction with window managers.

XMapWindow can generate a **BadWindow** error.

To map and raise a window, use **XMapRaised.**

```
XMapRaised (display, w)
```

The **XMapRaised** function essentially is similar to **XMapWindow** in that it maps the window and all of its subwindows that have had map requests. However, it also raises the specified window to the top of the stack.

XMapRaised can generate multiple **BadWindow** errors.

To map all subwindows for a specified window, use **XMapSubwindows.**

```
XMapSubwindows (display, w)
```

The **XMapSubwindows** function maps all subwindows for a specified window in top-to-bottom stacking order. The X server generates Expose events on each newly displayed window. This may be much more efficient than mapping many windows one at a time because the server needs to perform much of the work only once, for all of the windows, rather than for each window.

XMapSubwindows can generate a **BadWindow** error.

- **Unmapping Windows**: Xlib provides functions that you can use to unmap a window or all subwindows.

To unmap a window, use **XUnmapWindow.**

```
XUnmapWindow(display, w)
```

The **XUnmapWindow** function unmaps the specified window and causes the X server to generate an **UnmapNotify** event. If the specified window is already unmapped, **XUnmapWindow** has no effect. Normal exposure processing on formerly obscured windows is performed. Any child window will no longer be visible until another map call is made on the parent. In other words, the subwindows are still mapped but are not visible until the parent is mapped. Unmapping a window will generate Expose events on windows that were formerly obscured by it.

XUnmapWindow can generate a **BadWindow** error.

To unmap all subwindows for a specified window, use **XUnmapSubwindows**

```
XUnmapSubwindows (display, w)
```

The **XUnmapSubwindows** function unmaps all subwindows for the specified window in bottom-to-top stacking order. It causes the X server to generate an **UnmapNotify** event on each subwindow and Expose events on formerly obscured windows. Using this function is much more efficient than unmapping multiple windows one at a time because the server needs to perform much of the work only once, for all of the windows, rather than for each window.

XUnmapSubwindows can generate a **BadWindow** error.

- **Configuring Windows**: Xlib provides functions that you can use to move a window, resize a window, move and resize a window, or change a window's border width. To change one of these parameters, set the appropriate member of the **XWindowChanges** structure and OR in the corresponding value mask in subsequent calls to **XConfigureWindow.** The symbols for the value mask bits and the **XWindowChanges** structure are

```
/* Configure window value mask bits */
#define CWX                 (1<<0)
#define CWY                 (1<<1)
#define CWWidth             (1<<2)
#define CWHeight            (1<<3)
#define CWBorderWidth       (1<<4)
#define CWSibling           (1<<5)
#define CWStackMode         (1<<6)

/* Values */

typedef struct {
    int x, y;
    int width, height;
    int border_width;
    Window sibling;
    int stack_mode;
} XWindowChanges;
```

The x and y members are used to set the window's x and y coordinates, which are relative to the parent's origin and indicate the position of the upper-left outer corner of the window. The width and height members are used to set the inside size of the window, not including the border, and must be nonzero, or a **BadValue** error results. Attempts to configure a root window have no effect.

The border_width member is used to set the width of the border in pixels. Note that setting just the border width leaves the outer-left corner of the window in a fixed position but moves the absolute position of the window's origin. If you attempt to set the border-width attribute of an **InputOnly** window nonzero, a **BadMatch** error results.

Note: Inside the window, $(0, 0)$ excludes the border. But when positioning the window relative to its parent, the position refers to the outer window, including the border.

The sibling member is used to set the sibling window for stacking operations. The stack_mode member is used to set how the window is to be restacked and can be set to

- **Above**: Places the window above another window. Without **sibling** specified, it raises the window to the top. With sibling specified, it places that window above its specified sibling.
- **Below**: Places the window below another window. Without **sibling** specified, it raises the window to the bottom. With sibling specified, it places that window below its specified sibling.
- **TopIf**: Raises the window only if doing so is necessary. Informally: raise this window if it is currently being obscured.

  Without sibling, X checks whether any sibling window is covering part of this window. If so, it raises the window to the top.

  With sibling, X checks the relationship between this window and that sibling. If the sibling obscures this window, then this window is raised.
- **BottomIf**: Lowers the window only if doing so is necessary.

  Without sibling, X checks whether this window is covering any sibling. If so, it lowers the window to the bottom.

  With sibling, X checks whether this window obscures the given sibling. If so, this window is lowered.
- **Opposite**: This is conditional behavior based on whether the window is above or below another window.

  Informally,

  * if this window is being obscured, raise it;
  * if this window is obscuring another window, lower it

  Without sibling, X compares the window against its siblings generally.

  With sibling, X compares the window specifically against that sibling.

So Opposite can act like either **TopIf** or **BottomIf,** depending on the current overlap and stacking relationship.

If you do not specify a sibling, then stacking is relative to the whole sibling stack. For example, specifying **Above** will mean that a window gets raised to the top of its sibling stack.

If you specify sibling, then the operation is relative to that sibling window.

The sibling must actually be a sibling of the window being configured. That means both windows must have the same parent.

If the override-redirect flag of the window is **False** and if some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a ConfigureRequest event, and no further processing is performed. Otherwise, if some other client has selected **ResizeRedirectMask** on the window and the inside width or height of the window is being changed, a **ResizeRequest** event is generated, and the current inside width and height are used instead. Note that the override-redirect flag of the window has no effect on **ResizeRedirectMask** and that **SubstructureRedirectMask** on the parent has precedence over **ResizeRedirectMask** on the window.

When the geometry of the window is changed as specified, the window is restacked among siblings, and a **ConfigureNotify** event is generated if the state of the window actually changes. **GravityNotify** events are generated after **ConfigureNotify** events. If the inside width or height of the window has actually changed, children of the window are affected as specified.

If a window's size actually changes, the window's subwindows move according to their window gravity. Depending on the window's bit gravity, the contents of the window also may be moved.

If regions of the window were obscured but now are not, exposure processing is performed on these formerly obscured windows, including the window itself and its inferiors. As a result of increasing the width or height, exposure processing is also performed on any new regions of the window and any regions where window contents are lost

The restack check (specifically, the computation for BottomIf, TopIf, and Opposite) is performed with respect to the window's final size and position (as controlled by the other arguments of the request), not its initial position. If a sibling is specified without a stack_mode, a BadMatch error results.

Attempts to configure a root window have no effect.

To configure a window's size, location, stacking, or border, use **XConfigureWindow**.

```
XConfigureWindow(Display* display, Window w, unsigned int
↪  value_mask, XWindowChanges* values)
```

– value_mask Specifies which values are to be set using information in the values structure. This mask is the bitwise inclusive OR of the valid configure window values bits.
– values Specifies the **XWindowChanges** structure.

The **XConfigureWindow** function uses the values specified in the **XWindowChanges** structure to reconfigure a window's size, position, border, and stacking order. Values not specified are taken from the existing geometry of the window.

If a sibling is specified without a stack_mode or if the window is not actually a sibling, a BadMatch error results. Note that the computations for **BottomIf, TopIf,** and Opposite are performed with respect to the window's final geometry (as controlled by the other arguments passed to **XConfigureWindow),** not its initial geometry. Any backing store contents of the window, its inferiors, and other newly visible windows are either discarded or changed to reflect the current screen contents (depending on the implementation).

XConfigureWindow can generate **BadMatch, BadValue,** and **BadWindow** errors.

Note: A call to XConfigureWindow does cause the window to move in the stack (if specified in the mask), with respect to the specified configuration.

To move a window without changing its size, use **XMoveWindow.**

```
XMoveWindow(display, w, x, y)
```

– x, y Specify the x and y coordinates, which define the new location of the top-left pixel of the window's border or the window itself if it has no border.

The **XMoveWindow** function moves the specified window to the specified x and y coordinates, but it does not change the window's size, raise the window, or change the mapping state of the window. Moving a mapped window may or may not lose the window's contents depending on if the window is obscured by non-children and if no backing store exists. If the contents of the window are lost, the X server generates Expose events. Moving a mapped window generates Expose events on any formerly obscured windows.

If the override-redirect flag of the window is False and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a ConfigureRequest event, and no further processing is performed. Otherwise, the window is moved.

XMoveWindow can generate a **BadWindow** error.

To change a window's size without changing the upper-left coordinate, use **XResizeWindow.**

```
XResizeWindow(Display* display, Window w, unsigned int
    ↪ width, unsigned int height)
```

– width, height Specify the width and height, which are the interior dimensions of the window after the call completes.

The **XResizeWindow** function changes the inside dimensions of the specified window, not including its borders. This function does not change the window's upper-left coordinate or the origin and does not restack the window. Changing the size of a mapped window may lose its contents and generate **Expose** events. If a mapped window is made smaller, changing its size generates **Expose** events on windows that the mapped window formerly obscured.

If the override-redirect flag of the window is False and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a ConfigureRequest event, and no further processing is performed. If either width or height is zero, a **BadValue** error results. **XResizeWindow** can generate **BadValue** and **BadWindow** errors.

To change the size and location of a window, use **XMoveResizeWindow.**

```
XMoveResizeWindow(Display* display, Window w, int $x$, int
    ↪ y, unsigned int width, unsigned int height)
```

– x, y Specify the x and y coordinates, which define the new position of the window relative to its parent.

– width, height Specify the width and height, which define the interior size of the window.

The **XMoveResizeWindow** function changes the size and location of the specified window without raising it. Moving and resizing a mapped window may generate an **Expose** event on the window. Depending on the new size and location parameters, moving and resizing a window may generate **Expose** events on windows that the window formerly obscured.

If the override-redirect flag of the window is False and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a ConfigureRequest event, and no further processing is performed. Otherwise, the window size and location are changed.

XMoveResizeWindow can generate **BadValue** and **BadWindow** errors.

To change the border width of a given window, use **XSetWindowBorderWidth.**

```
XSetWindowBorderWidth (Display* display, Window w, unsigned
    ↪ int width)
```

The **XSetWindowBorderWidth** function sets the specified window's border width to the specified width.

XSetWindowBorderWidth can generate a **BadWindow** error.

- **Changing Window Stacking Order**: Xlib provides functions that you can use to raise, lower, circulate, or restack windows.

  To raise a window so that no sibling window obscures it, use XRaiseWindow.

```
XRaiseWindow(display, w)
```

  The **XRaiseWindow** function raises the specified window to the top of the stack so that no sibling window obscures it. If the windows are regarded as overlapping sheets of paper stacked on a desk, then raising a window is analogous to moving the sheet to the top of the stack but leaving its x and y location on the desk constant. Raising a mapped window may generate **Expose** events for the window and any mapped subwindows that were formerly obscured.

  If the override-redirect attribute of the window is **False** and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a ConfigureRequest event, and no processing is performed. Otherwise, the window is raised.

  XRaiseWindow can generate a **BadWindow** error.

  To lower a window so that it does not obscure any sibling windows, use **XLowerWindow.**

```
XLowerWindow(display, w)
```

  The **XLowerWindow** function lowers the specified window to the bottom of the stack so that it does not obscure any sibling windows. If the windows are regarded as overlapping sheets of paper stacked on a desk, then lowering a window is analogous to moving the sheet to the bottom of the stack but leaving its x and y location on the desk constant. Lowering a mapped window will generate **Expose** events on any windows it formerly obscured.

  If the override-redirect attribute of the window is **False** and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates a **ConfigureRequest** event, and no processing is performed. Otherwise, the window is lowered to the bottom of the stack.

  XLowerWindow can generate a **BadWindow** error.

  To circulate a subwindow up or down, use **XCirculateSubwindows**

```
XCirculateSubwindows(Display* display, Window w, int
    direction)
```

– direction Specifies the direction (up or down) that you want to circulate the window. You can pass **RaiseLowest** or **LowerHighest.**

The **XCirculateSubwindows** function circulates children of the specified window in the specified direction. If you specify **RaiseLowest, XCirculateSubwindows** raises the lowest mapped child (if any) that is occluded by another child to the top of the stack. If you specify **LowerHighest, XCirculateSubwindows** lowers the highest mapped child (if any) that occludes another child to the bottom of the stack. Exposure processing is then performed on formerly obscured windows. If some other client has selected **SubstructureRedirectMask** on the window, the X server generates a **CirculateRequest** event, and no further processing is performed. If a child is actually restacked, the X server generates a **CirculateNotify** event.

XCirculateSubwindows can generate **BadValue** and **BadWindow** errors.

To raise the lowest mapped child of a window that is partially or completely occluded by another child, use **XCirculateSubwindowsUp.**

```
XCirculateSubwindowsUp (display, w)
```

The **XCirculateSubwindowsUp** function raises the lowest mapped child of the specified window that is partially or completely occluded by another child. Completely unobscured children are not affected. This is a convenience function equivalent to **XCirculateSubwindows** with **RaiseLowest** specified.

XCirculateSubwindowsUp can generate a **BadWindow** error.

To lower the highest mapped child of a window that partially or completely occludes another child, use **XCirculateSubwindowsDown**

```
XCirculateSubwindowsDown (display, w)
```

The **XCirculateSubwindowsDown** function lowers the highest mapped child of the specified window that partially or completely occludes another child. Completely unobscured children are not affected. This is a convenience function equivalent to **XCirculateSubwindows** with **LowerHighest** specified.

XCirculateSubwindowsDown can generate a **BadWindow** error.

To restack a set of windows from top to bottom, use **XRestackWindows.**

```
XRestackWindows (Display* display, Window windows[], int
    ↪  nwindows);
```

- windows Specifies an array containing the windows to be restacked.
- nwindows Specifies the number of windows to be restacked.

The **XRestackWindows** function restacks the windows in the order specified, from top to bottom. The stacking order of the first window in the windows array is unaffected, but the other windows in the array are stacked underneath the first window, in the order of the array. The stacking order of the other windows is not affected. For each window in the window array that is not a child of the specified window, a **BadMatch** error results.

If the override-redirect attribute of a window is False and some other client has selected **SubstructureRedirectMask** on the parent, the X server generates **ConfigureRequest** events for each window whose override-redirect flag is not set, and no further processing is performed. Otherwise, the windows will be restacked in top-to-bottom order.

XRestackWindows can generate a **BadWindow** error.

- **Changing window attributes**: Xlib provides functions that you can use to set window attributes. **XChangeWindowAttributes** is the more general function that allows you to set one or more window attributes provided by the XSetWindowAttributes structure. The other functions described in this section allow you to set one specific window attribute, such as a window's background.

  Recall the **XSetWindowAttributes** structure

```c
/* Window attribute value mask bits */
#define CWBackPixmap        (1L<<0)
#define CWBackPixel         (1L<<1)
#define CWBorderPixmap      (1L<<2)
#define CWBorderPixel       (1L<<3)
#define CWBitGravity        (1L<<4)
#define CWWinGravity        (1L<<5)
#define CWBackingStore      (1L<<6)
#define CWBackingPlanes     (1L<<7)
#define CWBackingPixel      (1L<<8)
#define CWOverrideRedirect  (1L<<9)
#define CWSaveUnder         (1L<<10)
#define CWEventMask         (1L<<11)
#define CWDontPropagate     (1L<<12)
#define CWColormap          (1L<<13)
#define CWCursor            (1L<<14)

typedef struct {
    Pixmap background_pixmap;       /* background, None, or
    ↪  ParentRelative */
    unsigned long background_pixel; /* background pixel */
    Pixmap border_pixmap;           /* border of the window
    ↪  or CopyFromParent */
    unsigned long border_pixel;     /* border pixel value */
    int bit_gravity;                /* one of bit gravity
    ↪  values */
    int win_gravity;                /* one of the window
    ↪  gravity values */
    int backing_store;              /* NotUseful, WhenMapped,
    ↪  Always */
    unsigned long backing_planes;   /* planes to be preserved
    ↪  if possible */
    unsigned long backing_pixel;    /* value to use in
    ↪  restoring planes */
    Bool save_under;                /* should bits under be
    ↪  saved? (popups) */
    long event_mask;                /* set of events that
    ↪  should be saved */
    long do_not_propagate_mask;     /* set of events that
    ↪  should not propagate */
    Bool override_redirect;         /* boolean value for
    ↪  override_redirect */
    Colormap colormap;              /* color map to be
    ↪  associated with window */
    Cursor cursor;                  /* cursor to be displayed
    ↪  (or None) */
} XSetWindowAttributes;
```

To change one or more attributes for a given window, use **XChangeWindowAttributes**

```
XChangeWindowAttributes(Display* display, Window w, unsigned
↪ long valuemask, XSetWindowAttributes* attributes)
```

- valuemask Specifies which window attributes are defined in the attributes argument. This mask is the bitwise inclusive OR of the valid attribute mask bits. If valuemask is zero, the attributes are ignored and are not referenced. The values and restrictions are the same as for XCreateWindow.
- attributes Specifies the structure from which the values (as specified by the value mask) are to be taken. The value mask should have the appropriate bits set to indicate which attributes have been set in the structure

Depending on the valuemask, the **XChangeWindowAttributes** function uses the window attributes in the XSetWindowAttributes structure to change the specified window attributes. Changing the background does not cause the window contents to be changed. To repaint the window and its background, use **XClearWindow.** Setting the border or changing the background such that the border tile origin changes causes the border to be repainted. Changing the background of a root window to None or **ParentRelative** restores the default background pixmap. Changing the border of a root window to **CopyFromParent** restores the default border pixmap. Changing the win-gravity does not affect the current position of the window. Changing the backing-store of an obscured window to **WhenMapped** or Always, or changing the backing-planes, backing-pixel, or save-under of a mapped window may have no immediate effect. Changing the colormap of a window (that is, defining a new map, not changing the contents of the existing map) generates a **ColormapNotify** event. Changing the colormap of a visible window may have no immediate effect on the screen because the map may not be installed (see **XInstallColormap).** Changing the cursor of a root window to None restores the default cursor. Whenever possible, you are encouraged to share colormaps.

Multiple clients can select input on the same window. Their event masks are maintained separately. When an event is generated, it is reported to all interested clients. However, only one client at a time can select for **SubstructureRedirectMask, ResizeRedirectMask,** and **ButtonPressMask.** If a client attempts to select any of these event masks and some other client has already selected one, a **BadAccess** error results. There is only one do-not-propagate-mask for a window, not one per client.

XChangeWindowAttributes can generate **BadAccess, BadColor, BadCursor, BadMatch, BadPixmap, BadValue,** and **BadWindow** errors.

To set the background of a window to a given pixel, use **XSetWindowBackground**

```
XSetWindowBackground (Display* display, Window w, unsigned
↪ long background_pixel)
```

- background_pixel Specifies the pixel that is to be used for the background.

The **XSetWindowBackground** function sets the background of the window to the specified pixel value. Changing the background does not cause the window contents to be changed. **XSetWindowBackground** uses a pixmap of undefined size filled with the pixel value you passed. If you try to change the background of an **InputOnly** window, a **BadMatch** error results.

XSetWindowBackground can generate **BadMatch** and **BadWindow** errors.

To set the background of a window to a given pixmap, use **XSetWindowBackgroundPixmap.**

```
XSetWindowBackgroundPixmap (Display* display, Window w,
↪  Pixmap background_pixmap)
```

The **XSetWindowBackgroundPixmap** function sets the background pixmap of the window to the specified pixmap. The background pixmap can immediately be freed if no further explicit references to it are to be made. If **ParentRelative** is specified, the background pixmap of the window's parent is used, or on the root window, the default background is restored. If you try to change the background of an **InputOnly** window, a **BadMatch** error results. If the background is set to None, the window has no defined background.

XSetWindowBackgroundPixmap can generate **BadMatch, BadPixmap,** and **BadWindow** errors.

Note: **XSetWindowBackground** and **XSetWindowBackgroundPixmap** do not change the current contents of the window.

To change and repaint a window's border to a given pixel, use **XSetWindowBorder.**

```
XSetWindowBorder (display, w, unsigned long border_pixel)
```

– border_pixel Specifies the entry in the colormap.

The **XSetWindowBorder** function sets the border of the window to the pixel value you specify. If you attempt to perform this on an **InputOnly** window, a **BadMatch** error results.

XSetWindowBorder can generate **BadMatch** and **BadWindow** errors.

To change and repaint the border tile of a given window, use XSetWindowBorderPixmap.

```
XSetWindowBorderPixmap (display, w, Pixmap border_pixmap)
```

– border_pixmap Specifies the border pixmap or CopyFromParent.

The **XSetWindowBorderPixmap** function sets the border pixmap of the window to the pixmap you specify. The border pixmap can be freed immediately if no further explicit references to it are to be made. If you specify **CopyFromParent,** a copy of the parent window's border pixmap is used. If you attempt to perform this on an **InputOnly** window, a **BadMatch** error results.

XSetWindowBorderPixmap can generate **BadMatch, BadPixmap,** and **BadWindow** errors.

To set the colormap of a given window, use **XSetWindowColormap.**

```
XSetWindowColormap (display, w, Colormap colormap)
```

The **XSetWindowColormap** function sets the specified colormap of the specified window. The colormap must have the same visual type as the window, or a **BadMatch** error results.

XSetWindowColormap can generate **BadColor, BadMatch,** and **BadWindow** errors.

To define which cursor will be used in a window, use **XDefineCursor**

```
XDefineCursor (display, w, Cursor cursor)
```

– cursor Specifies the cursor that is to be displayed or None.

If a cursor is set, it will be used when the pointer is in the window. If the cursor is None, it is equivalent to **XUndefineCursor. XDefineCursor** can generate BadCursor and **BadWindow** errors.

To undefine the cursor in a given window, use **XUndefineCursor.**

```
XUndefineCursor (display, w)
```

The **XUndefineCursor** function undoes the effect of a previous **XDefineCursor** for this window. When the pointer is in the window, the parent's cursor will now be used. On the root window, the default cursor is restored.

XUndefineCursor can generate a **BadWindow** error.

4. Window information functions

- **Intro**: After you connect the display to the X server and create a window, you can use the Xlib window information functions to:
  - Obtain information about a window
  - Translate screen coordinates
  - Manipulate property lists
  - Obtain and change window properties
  - Manipulate selections
- **Obtaining Window Information**: Xlib provides functions that you can use to obtain information about the window tree, the window's current attributes, the window's current geometry, or the current pointer coordinates. Because they are most frequently used by window managers, these functions all return a status to indicate whether the window still exists.

  To obtain the parent, a list of children, and number of children for a given window, use **XQueryTree.**

```
Status XQueryTree (Display* display, Window w, Window*
    ↪ root_return, Window* parent_return, Window**
    ↪ children_return, unsigned int* nchildren_return)
```

  - root_return Returns the root window.
  - parent_return Returns the parent window.
  - children_return Returns the list of children.
  - nchildren_return Returns the number of children.

  The **XQueryTree** function returns the root ID, the parent window ID, a pointer to the list of children windows (NULL when there are no children), and the number of children in the list for the specified window. The children are listed in current stacking order, from bottom-most (first) to top-most (last). **XQueryTree** returns zero if it fails and nonzero if it succeeds. To free a non-NULL children list when it is no longer needed, use **XFree.**

  XQueryTree can generate a **BadWindow** error.

  To obtain the current attributes of a given window, use **XGetWindowAttributes.**

```
Status XGetWindowAttributes (display, w, XWindowAttributes*
    ↪ window_attributes_return)
```

  - **window_attributes_return**: Returns the specified window's attributes in the XWindowAttributes structure.

  The **XGetWindowAttributes** function returns the current attributes for the specified window to an **XWindowAttributes** structure.

```
typedef struct {
    int x, y; /* location of window */
    int width, height; /* width and height of window */
    int border_width; /* border width of window */
    int depth; /* depth of window */
    Visual *visual; /* the associated visual structure */
    Window root; /* root of screen containing window */
    int class; /* InputOutput, InputOnly*/
    int bit_gravity; /* one of the bit gravity values */
    int win_gravity; /* one of the window gravity values */
    int backing_store; /* NotUseful, WhenMapped, Always */
    unsigned long backing_planes; /* planes to be preserved
    ↪  if possible */
    unsigned long backing_pixel; /* value to be used when
    ↪  restoring planes */
    Bool save_under; /* boolean, should bits under be saved?
    ↪  */
    Colormap colormap; /* color map to be associated with
    ↪  window */
    Bool map_installed; /* boolean, is color map currently
    ↪  installed*/
    int map_state; /* IsUnmapped, IsUnviewable, IsViewable */
    long all_event_masks; /* set of events all people have
    ↪  interest in*/
    long your_event_mask; /* my event mask */
    long do_not_propagate_mask; /* set of events that should
    ↪  not propagate */
    Bool override_redirect; /* boolean value for
    ↪  override-redirect */
    Screen *screen; /* back pointer to correct screen */
} XWindowAttributes;
```

The x and y members are set to the upper-left outer corner relative to the parent window's origin. The width and height members are set to the inside size of the window, not including the border. The border_width member is set to the window's border width in pixels. The depth member is set to the depth of the window (that is, bits per pixel for the object). The visual member is a pointer to the screen's associated **Visual** structure. The root member is set to the root window of the screen containing the window. The class member is set to the window's class and can be either **InputOutput** or **InputOnly.**

The bit_gravity member is set to the window's bit gravity and can be one of the following:

- ForgetGravity
- EastGravity
- NorthWestGravity
- SouthWestGravity
- NorthGravity
- SouthGravity
- NorthEastGravity

- SouthEastGravity
- WestGravity
- StaticGravity
- CenterGravity

The win_gravity member is set to the window's window gravity and can be one of the following:

- UnmapGravity
- EastGravity
- NorthWestGravity
- SouthWestGravity
- NorthGravity
- SouthGravity
- NorthEastGravity
- SouthEastGravity
- WestGravity
- StaticGravity
- CenterGravity

The backing_store member is set to indicate how the X server should maintain the contents of a window and can be **WhenMapped,** Always, or **NotUseful.** The backing_planes member is set to indicate (with bits set to 1) which bit planes of the window hold dynamic data that must be pre-served in backing_stores and during save_unders. The backing_pixel member is set to indicate what values to use for planes not set in backing_planes.

The save_under member is set to True or False. The colormap member is set to the colormap for the specified window and can be a colormap ID or None. The map_installed member is set to indicate whether the colormap is currently installed and can be True or False. The map_state member is set to indicate the state of the window and can be **IsUnmapped, IsUnviewable,** or **IsViewable. IsUnviewable** is used if the window is mapped but some ancestor is unmapped.

The all_event_masks member is set to the bitwise inclusive OR of all event masks selected on the window by all clients. The your_event_mask member is set to the bitwise inclusive OR of all event masks selected by the querying client. The do_not_propagate_mask member is set to the bitwise inclusive OR of the set of events that should not propagate.

The override_redirect member is set to indicate whether this window overrides structure control facilities and can be True or False. Window manager clients should ignore the window if this member is True.

The screen member is set to a screen pointer that gives you a back pointer to the correct screen. This makes it easier to obtain the screen information without having to loop over the root window fields to see which field matches. **XGetWindowAttributes** can generate **BadDrawable** and **BadWindow** errors.

To obtain the current geometry of a given drawable, use **XGetGeometry.**

```
Status XGetGeometry(
    Display* display,
    Drawable d,
    Window* root_return,
    int* x_return, int* y_return,
    unsigned int* width_return, unsigned int* height_return,
    unsigned int* border_width_return,
    unsigned int* depth_return
)
```

- d Specifies the drawable, which can be a window or a pixmap.
- root_return Returns the root window.
- x_return, y_return Return the x and y coordinates that define the location of the drawable. For a window, these coordinates specify the upper-left outer corner relative to its parent's origin. For pixmaps, these coordinates are always zero.
- width_return, height_return Return the drawable's dimensions (width and height). For a window, these dimensions specify the inside size, not including the border.
- border_width_return Returns the border width in pixels. If the drawable is a pixmap, it returns zero.
- depth_return Returns the depth of the drawable (bits per pixel for the object).

The **XGetGeometry** function returns the root window and the current geometry of the drawable. The geometry of the drawable includes the x and y coordinates, width and height, border width, and depth. These are described in the argument list. It is legal to pass to this function a window whose class is **InputOnly.**

XGetGeometry can generate a **BadDrawable** error.

- **Translating Screen Coordinates**: Applications sometimes need to perform a coordinate transformation from the coordinate space of one window to another window or need to determine which window the pointing device is in. **XTranslateCoordinates** and **XQueryPointer** fulfill these needs (and avoid any race conditions) by asking the X server to perform these operations.

  To translate a coordinate in one window to the coordinate space of another window, use **XTranslateCoordinates.**

```
    Bool XTranslateCoordinates (
        Display* display,
        Window src_w, Window dest_w,
        int src_x, int src_y,
        int* dest_x_return, int* dest_y_return,
        Window* child_return
)
```

  - display Specifies the connection to the X server.
  - src_w Specifies the source window.
  - dest_w Specifies the destination window.
  - src_x, src_y Specify the x and y coordinates within the source window.

- dest_x_return, dest_y_return Return the x and y coordinates within the destination window.
- child_return Returns the child if the coordinates are contained in a mapped child of the destination window.

If **XTranslateCoordinates** returns **True,** it takes the src_x and src_y coordinates relative to the source window's origin and returns these coordinates to dest_x_return and dest_y_return relative to the destination window's origin. If **XTranslateCoordinates** returns **False,** src_w and dest_w are on different screens, and dest_x_return and dest_y_return are zero. If the coordinates are contained in a mapped child of dest_w, that child is returned to child_return. Otherwise, child_return is set to **None.**

XTranslateCoordinates can generate a **BadWindow** error.

To obtain the screen coordinates of the pointer or to determine the pointer coordinates relative to a specified window, use **XQueryPointer.**

```
Bool XQueryPointer(
    Display* display,
    Window w,
    Window* root_return, Window* child_return,
    int* root_x_return, int* root_y_return,
    int* win_x_return, int* win_y_return,
    unsigned int* mask_return
)
```

- display Specifies the connection to the X server.
- w Specifies the window.
- root_return Returns the root window that the pointer is in.
- child_return Returns the child window that the pointer is located in, if any.
- root_x_return, root_y_return Return the pointer coordinates relative to the root window's origin.
- win_x_return, win_y_return Return the pointer coordinates relative to the specified window.
- mask_return Returns the current state of the modifier keys and pointer buttons.

The **XQueryPointer** function returns the root window the pointer is logically on and the pointer coordinates relative to the root window's origin. If **XQueryPointer** returns False, the pointer is not on the same screen as the specified window, and **XQueryPointer** returns None to child_return and zero to win_x_return and win_y_return. If **XQueryPointer** returns True, the pointer coordinates returned to win_x_return and win_y_return are relative to the origin of the specified window. In this case, **XQueryPointer** returns the child that contains the pointer, if any, or else None to child_return.

XQueryPointer returns the current logical state of the keyboard buttons and the modifier keys in mask_return. It sets mask_return to the bitwise inclusive OR of one or more of the button or modifier key bitmasks to match the current state of the mouse buttons and the modifier keys. Note that the logical state of a device (as seen through Xlib) may lag the physical state if device event processing is frozen (see section 12.1).

XQueryPointer can generate a **BadWindow** error.

- **Properties**: In Xlib, a property is a piece of data attached to a window.

  Think of a window as having a small key-value database attached to it:

```
Window
  |--- property: WM_NAME         -> "My Terminal"
  |--- property: WM_CLASS        -> "kitty"
  |--- property: WM_HINTS        -> window-manager hints
  |--- property: _NET_WM_STATE   -> fullscreen/maximized/etc.
```

  Each property has three main parts:

  - **name**: what the property is called
  - **type**: what kind of data it stores
  - **data**: the actual bytes stored

  For example,

```
name: WM_NAME
type: STRING
data: "My Window"
```

  An **atom** is just a unique numeric ID for a string name.

```
"WM_NAME"  -> atom 39
"STRING"   -> atom 31
"CARDINAL" -> atom 6
```

  So instead of repeatedly passing the string "WM_NAME", Xlib passes the atom representing "WM_NAME".

- **Properties and atoms**: A property is a collection of named, typed data. The window system has a set of predefined properties (for example, the name of a window, size hints, and so on), and users can define any other arbitrary information and associate it with windows. Each property has a name, which is an ISO Latin-1 string. For each named property, a unique identifier (**atom)** is associated with it. A property also has a type, for example, string or integer. These types are also indicated using atoms, so arbitrary new types can be defined. Data of only one type may be associated with a single property name. Clients can store and retrieve properties associated with windows. For efficiency reasons, an atom is used rather than a character string. **XInternAtom** can be used to obtain the atom for property names.

  A property is also stored in one of several possible formats. The X server can store the information as 8-bit quantities, 16-bit quantities, or 32-bit quantities. This permits the X server to present the data in the byte order that the client expects

Note: If you define further properties of complex type, you must encode and decode them yourself. These functions must be carefully written if they are to be portable

The type of a property is defined by an atom, which allows for arbitrary extension in this type scheme.

Certain property names are predefined in the server for commonly used functions. The atoms for these properties are defined in **<X11/Xatom.h>.** To avoid name clashes with user symbols, the **#define** name for each atom has the **XA_ prefix.** For an explanation of the functions that let you get and set much of the information stored in these predefined properties, see chapter 14.

The core protocol imposes no semantics on these property names, but semantics are specified in other X Consortium standards, such as the Inter-Client Communication Conventions Manual and the X Logical Font Description Conventions.

You can use properties to communicate other information between applications. The functions described in this section let you define new properties and get the unique atom IDs in your applications.

Although any particular atom can have some client interpretation within each of the name spaces, atoms occur in five distinct name spaces within the protocol:

- Selections
- Property names
- Property types
- Font properties
- Type of a **ClientMessage** event (none are built into the X server)

The built-in selection property names are:

```
PRIMARY             SECONDARY
CLIPBOARD
```

The built-in property names are:

```
CUT_BUFFER0         RESOURCE_MANAGER
CUT_BUFFER1         WM_CLASS
CUT_BUFFER2         WM_CLIENT_MACHINE
CUT_BUFFER3         WM_COLORMAP_WINDOWS
CUT_BUFFER4         WM_COMMAND
CUT_BUFFER5         WM_HINTS
CUT_BUFFER6         WM_ICON_NAME
CUT_BUFFER7         WM_ICON_SIZE
RGB_BEST_MAP        WM_NAME
RGB_BLUE_MAP        WM_NORMAL_HINTS
RGB_DEFAULT_MAP     WM_PROTOCOLS
RGB_GRAY_MAP        WM_STATE
RGB_GREEN_MAP       WM_TRANSIENT_FOR
RGB_RED_MAP         WM_ZOOM_HINTS
```

The built-in property types are:

```
ARC                 POINT
ATOM                RGB_COLOR_MAP
BITMAP              RECTANGLE
CARDINAL            STRING
COLORMAP            VISUALID
CURSOR              WINDOW
DRAWABLE            WM_HINTS
FONT                WM_SIZE_HINTS
INTEGER
PIXMAP
```

The built-in font property names are:

```
MIN_SPACE           STRIKEOUT_DESCENT
NORM_SPACE          STRIKEOUT_ASCENT
MAX_SPACE           ITALIC_ANGLE
END_SPACE           X_HEIGHT
SUPERSCRIPT_X       QUAD_WIDTH
SUPERSCRIPT_Y       WEIGHT
SUBSCRIPT_X         POINT_SIZE
SUBSCRIPT_Y         RESOLUTION
UNDERLINE_POSITION  COPYRIGHT
UNDERLINE_THICKNESS NOTICE
FONT_NAME           FAMILY_NAME
FULL_NAME           CAP_HEIGHT
```

```
0  Atom XInternAtom(Display* display, char* atom_name, Bool
   ↪  only_if_exists)
```

- display Specifies the connection to the X server.
- atom_name Specifies the name associated with the atom you want returned.
- only_if_exists Specifies a Boolean value that indicates whether the atom must be created.

The **XInternAtom** function returns the atom identifier associated with the specified atom_name string. If only_if_exists is False, the atom is created if it does not exist. Therefore, **XInternAtom** can return None. If the atom name is not in the Host Portable Character Encoding, the result is implementation-dependent. Uppercase and lowercase matter; the strings ‘‘thing’’, ‘‘Thing’’, and ‘‘thinG’’ all designate different atoms. The atom will remain defined even after the client’s connection closes. It will become undefined only when the last connection to the X server closes.

XInternAtom can generate BadAlloc and **BadValue** errors.

To return atoms for an array of names, use **XInternAtoms.**

```
Status XInternAtoms(Display* display, char** names, int
    ↪  count, Bool only_if_exists, Atom* atoms_return)
```

- display Specifies the connection to the X server.
- names Specifies the array of atom names.
- count Specifies the number of atom names in the array.
- only_if_exists Specifies a Boolean value that indicates whether the atom must be created.
- atoms_return Returns the atoms.

The **XInternAtoms** function returns the atom identifiers associated with the specified names. The atoms are stored in the atoms_return array supplied by the caller. Calling this function is equivalent to calling **XInternAtom** for each of the names in turn with the specified value of only_if_exists, but this function minimizes the number of round-trip protocol exchanges between the client and the X server. This function returns a nonzero status if atoms are returned for all of the names; otherwise, it returns zero.

XInternAtoms can generate BadAlloc and **BadValue** errors.

To return a name for a given atom identifier, use **XGetAtomName.**

```
char *XGetAtomName(Display display, Atom atom)
```

- display Specifies the connection to the X server.
- atom Specifies the atom for the property name you want returned.

The **XGetAtomName** function returns the name associated with the specified atom. If the data returned by the server is in the Latin Portable Character Encoding, then the returned string is in the Host Portable Character Encoding. Otherwise, the result is implementation-dependent. To free the resulting string, call **XFree.**

XGetAtomName can generate a **BadAtom** error.

To return the names for an array of atom identifiers, use **XGetAtomNames.**

```
Status XGetAtomNames(Display* display, Atom* atoms, int
    ↪  count, char** names_return)
```

- display Specifies the connection to the X server.
- atoms Specifies the array of atoms.
- count Specifies the number of atoms in the array.
- names_return Returns the atom names.

The **XGetAtomNames** function returns the names associated with the specified atoms. The names are stored in the names_return array supplied by the caller. Calling this function is equivalent to calling **XGetAtomName** for each of the atoms in turn, but this function minimizes the number of round-trip protocol exchanges between the client and the X server. This function returns a nonzero status if names are returned for all of the atoms; otherwise, it returns zero.

XGetAtomNames can generate a BadAtom error.

Note: Note that

```
typedef unsigned long Atom;
```

- **Obtaining and Changing Window Properties**: You can attach a property list to every window. Each property has a name, a type, and a value. The value is an array of 8-bit, 16-bit, or 32-bit quantities, whose interpretation is left to the clients. The type char is used to represent 8-bit quantities, the type short is used to represent 16-bit quantities, and the type long is used to represent 32-bit quantities.

  Xlib provides functions that you can use to obtain, change, update, or interchange window properties. In addition, Xlib provides other utility functions for inter-client communication (chapter 14)

  To obtain the type, format, and value of a property of a given window, use **XGetWindowProperty**

```
int XGetWindowProperty (
    Display* display,
    Window w,
    Atom property,
    long long_offset, long_length,
    Bool delete,
    Atom req_type,
    Atom* actual_type_return,
    int* actual_format_return,
    unsigned long* nitems_return,
    unsigned long* bytes_after_return,
    unsigned char** prop_return
)
```

  - display Specifies the connection to the X server.
  - *w* Specifies the window whose property you want to obtain.
  - property Specifies the property name.
  - long_offset Specifies the offset in the specified property (in 32-bit quantities) where the data is to be retrieved.
  - long_length Specifies the length in 32-bit multiples of the data to be retrieved.
  - delete Specifies a Boolean value that determines whether the property is deleted.
  - req_type Specifies the atom identifier associated with the property type or **AnyPropertyType**.
  - actual_type_return Returns the atom identifier that defines the actual type of the property.
  - actual_format_return Returns the actual format of the property.
  - nitems_return Returns the actual number of 8-bit, 16-bit, or 32-bit items stored in the prop_return data.

- bytes_after_return Returns the number of bytes remaining to be read in the property if a partial read was performed.
- prop_return Returns the data in the specified format.

The **XGetWindowProperty** function returns the actual type of the property; the actual format of the property; the number of 8-bit, 16-bit, or 32-bit items transferred; the number of bytes remaining to be read in the property; and a pointer to the data actually returned. **XGetWindowProperty** sets the return arguments as follows:

- If the specified property does not exist for the specified window, **XGetWindowProperty** returns **None** to actual_type_return and the value zero to actual_format_return and bytes_after_return. The nitems_return argument is empty. In this case, the delete argument is ignored.
- If the specified property exists but its type does not match the specified type, **XGetWindowProperty** returns the actual property type to actual_type_return, the actual property format (never zero) to actual_format_return, and the property length in bytes (even if the actual_format_return is 16 or 32) to bytes_after_return. It also ignores the delete argument. The nitems_return argument is empty.
- If the specified property exists and either you assign **AnyPropertyType** to the req_type argument or the specified type matches the actual property type, **XGetWindowProperty** returns the actual property type to actual_type_return and the actual property format (never zero) to actual_format_return. It also returns a value to bytes_after_return and nitems_return, by defining the following values:

$$
\begin{aligned}
N &= \text{actual length of the stored property in bytes} \\
I &= 4 \cdot \text{long\_offset} \\
T &= N - I \\
L &= \min(T, 4 \cdot \text{long\_length}) \\
A &= N - (I + L).
\end{aligned}
$$

  The returned value starts at byte index I in the property (indexing from zero), and its length in bytes is L. If the value for long_offset causes L to be negative, a **BadValue** error results. The value of bytes_after_return is A, giving the number of trailing unread bytes in the stored property.

Note: `Bool delete` specifies whether X should delete the property after reading it. It only actually deletes the property if the read succeeds and there are no unread bytes left, meaning `bytes_after_return = 0`.

If you pass a specific type and the property has a different actual type, Xlib tells you the actual type but does not return the property data in the normal way.

For

```
0  int* actual_format_return
```

Xlib writes the property's data form here. The format is usually one of

- 8 $\implies$ data is 8-bit quantities
- 16 $\implies$ data is 16-bit quantities
- 32 $\implies$ data is 32-bit quantities

This tells us how to interpret `prop_return`.

If the returned format is 8, the returned data is represented as a **char** array. If the returned format is 16, the returned data is represented as a **short** array and should be cast to that type to obtain the elements. If the returned format is 32, the returned data is represented as a **long** array and should be cast to that type to obtain the elements.

XGetWindowProperty always allocates one extra byte in prop_return (even if the property is zero length) and sets it to zero so that simple properties consisting of characters do not have to be copied into yet another string before use.

If delete is True and bytes_after_return is zero, **XGetWindowProperty** deletes the property from the window and generates a **PropertyNotify** event on the window.

The function returns **Success** if it executes successfully. To free the resulting data, use **XFree.**

XGetWindowProperty can generate **BadAtom, BadValue,** and **BadWindow** errors.

To obtain a given window's property list, use **XListProperties.**

```
Atom* XListProperties(Display* display, Window w, int*
    num_prop_return)
```

- display Specifies the connection to the X server.
- *w* Specifies the window whose property list you want to obtain.
- num_prop_return Returns the length of the properties array.

The **XListProperties** function returns a pointer to an array of atom properties that are defined for the specified window or returns NULL if no properties were found. To free the memory allocated by this function, use **XFree.**

XListProperties can generate a **BadWindow** error.

To change a property of a given window, use **XChangeProperty**

```
XChangeProperty (
    Display* display,
    Window w,
    Atom property, type,
    int format,
    int mode,
    unsigned char* data,
    int nelements
)
```

- display Specifies the connection to the X server.
- *w* Specifies the window whose property you want to change.

- property Specifies the property name.
- type Specifies the type of the property. The X server does not interpret the type but simply passes it back to an application that later calls **XGetWindowProperty.**
- format Specifies whether the data should be viewed as a list of 8-bit, 16-bit, or 32-bit quantities. Possible values are 8, 16, and 32. This information allows the X server to correctly perform byte-swap operations as necessary. If the format is 16-bit or 32-bit, you must explicitly cast your data pointer to an (unsigned char *) in the call to **XChangeProperty.**
- mode Specifies the mode of the operation. You can pass **PropModeReplace, PropModePrepend,** or **PropModeAppend.**
- data Specifies the property data.
- nelements Specifies the number of elements of the specified data format.

The **XChangeProperty** function alters the property for the specified window and causes the X server to generate a **PropertyNotify** event on that window. **XChangeProperty** performs the following:

- If mode is **PropModeReplace, XChangeProperty** discards the previous property value and stores the new data.
- If mode is **PropModePrepend** or **PropModeAppend, XChangeProperty** inserts the specified data before the beginning of the existing data or onto the end of the existing data, respectively. The type and format must match the existing property value, or a **BadMatch** error results. If the property is undefined, it is treated as defined with the correct type and format with zero-length data.

If the specified format is 8, the property data must be a **char** array. If the specified format is 16, the property data must be a **short** array. If the specified format is 32, the property data must be a **long** array.

The lifetime of a property is not tied to the storing client. Properties remain until explicitly deleted, until the window is destroyed, or until the server resets. The maximum size of a property is server dependent and can vary dynamically depending on the amount of memory the server has available. (If there is insufficient space, a BadAlloc error results.)

XChangeProperty can generate BadAlloc, **BadAtom, BadMatch, BadValue,** and **BadWindow** errors.

To rotate a window's property list, use **XRotateWindowProperties.**

```
XRotateWindowProperties (display, w, properties, num_prop,
↪ npositions)
```

- display Specifies the connection to the X server.
- *w* Specifies the window.
- properties Specifies the array of properties that are to be rotated.
- num_prop Specifies the length of the properties array.
- npositions Specifies the rotation amount.

The **XRotateWindowProperties** function allows you to rotate properties on a window and causes the X server to generate **PropertyNotify** events. If the property names in the properties array are viewed as being numbered starting from zero and if there are num_prop property names in the list, then the value associated with property name I becomes the value associated with property name $(I + \text{npositions}) \text{mod} N$ for all I from zero to $N - 1$. The effect is to rotate the states by npositions places around the virtual ring of property names (right for positive npositions, left for negative npositions). If npositions mod N is nonzero, the X server generates a **PropertyNotify** event for each property in the order that they are listed in the array. If an atom occurs more than once in the list or no property with that name is defined for the window, a **BadMatch** error results. If a **BadAtom** or **BadMatch** error results, no properties are changed.

XRotateWindowProperties can generate **BadAtom, BadMatch,** and **BadWindow** errors.

XRotateWindowProperties is a niche convenience function for swapping property values among several property names on the same window.

It does not rotate the bytes inside one property. It rotates which property name owns which value.

Suppose a window has three properties:

```
P0 = "red"
P1 = "green"
P2 = "blue"
```

and you call

```
Atom props[] = { P0, P1, P2 };
XRotateWindowProperties(display, w, props, 3, 1);
```

With npositions = 1, the value associated with property I moves to property $(I + 1) \text{mod} 3$.

- P0's old value moves to P1
- P1's old value moves to P2
- P2's old value moves to P0

```
P0 = "blue"
P1 = "red"
P2 = "green"
```

To delete a property on a given window, use **XDeleteProperty.**

```
XDeleteProperty (display, w, property)
```

- display Specifies the connection to the X server.
- w Specifies the window whose property you want to delete.
- property Specifies the property name.

The **XDeleteProperty** function deletes the specified property only if the property was defined on the specified window and causes the X server to generate a **PropertyNotify** event on the window unless the property does not exist.

XDeleteProperty can generate **BadAtom** and **BadWindow** errors.

- **Selections**: Selections are one method used by applications to exchange data. By using the property mechanism, applications can exchange data of arbitrary types and can negotiate the type of the data. A selection can be thought of as an indirect property with a dynamic type. That is, rather than having the property stored in the X server, the property is maintained by some client (the owner). A selection is global in nature (considered to belong to the user but be maintained by clients) rather than being private to a particular window subhierarchy or a particular set of clients. Xlib provides functions that you can use to set, get, or request conversion of selections. This allows applications to implement the notion of current selection, which requires that notification be sent to applications when they no longer own the selection. Applications that support selection often highlight the current selection and so must be informed when another application has acquired the selection so that they can unhighlight the selection.

  When a client asks for the contents of a selection, it specifies a selection target type. This target type can be used to control the transmitted representation of the contents. For example, if the selection is ''the last thing the user clicked on'' and that is currently an image, then the target type might specify whether the contents of the image should be sent in XY format or Z format.

  The target type can also be used to control the class of contents transmitted, for example, asking for the ''looks'' (fonts, line spacing, indentation, and so forth) of a paragraph selection, not the text of the paragraph. The target type can also be used for other purposes. The protocol does not constrain the semantics.

  A normal X property is stored on a window:

```
window W
  property WM_NAME = "Terminal"
  type STRING
  format 8
```

  The X server stores that property value. Another client can ask the server for WM_NAME, and the server can directly return the data.

  A selection is different. For example, with the clipboard:

```
selection CLIPBOARD
    owner = some client window
```

The X server usually does not store the clipboard contents itself. It only knows which client currently owns the selection. The actual data is maintained by the owner client. That is why it is called an **indirect property:** you do not directly read the value from the server; you ask the server who owns it, and the owner client provides the value. Xlib describes this as a selection being maintained by a client rather than stored in the X server.

The **dynamic type** part means the requester can ask for the same selection in different forms.

For example, suppose an application owns the **CLIPBOARD** selection and the selected content is rich text. Another client might request it as:

```
UTF8_STRING      // plain UTF-8 text
STRING           // older Latin-1-ish text
text/html        // HTML representation
image/png        // PNG representation, if applicable
TARGETS          // list of supported conversion targets
```

So the "type" is not fixed ahead of time like an ordinary property. The requester specifies a **target type**, and the owner decides whether it can convert the selection into that representation. Xlib says that when a client asks for a selection, it specifies a target type, which controls the transmitted representation.

To set the selection owner, use XSetSelectionOwner.

```
XSetSelectionOwner(Display* display, Atom selection, Window
↪  owner, Time time)
```

- display Specifies the connection to the X server.
- selection Specifies the selection atom.
- owner Specifies the owner of the specified selection atom. You can pass a window or **None.**
- time Specifies the time. You can pass either a timestamp or **CurrentTime.**

The **XSetSelectionOwner** function changes the owner and last-change time for the specified selection and has no effect if the specified time is earlier than the current last-change time of the specified selection or is later than the current X server time. Otherwise, the last-change time is set to the specified time, with **CurrentTime** replaced by the current server time. If the owner window is specified as **None,** then the owner of the selection becomes **None** (that is, no owner). Otherwise, the owner of the selection becomes the client executing the request.

If the new owner (whether a client or None) is not the same as the current owner of the selection and the current owner is not None, the current owner is sent a **SelectionClear** event. If the client that is the owner of a selection is later terminated (that is, its connection is closed) or if the owner window it has specified in the request is later destroyed, the owner of the selection automatically reverts to **None,** but the last-change time is not affected. The selection atom is uninterpreted by the X server. **XGetSelectionOwner** returns the owner window, which is reported in **SelectionRequest** and **SelectionClear** events. Selections are global to the X server.

XSetSelectionOwner can generate **BadAtom** and **BadWindow** errors.

The current owner does not get to veto another client taking ownership. the X server changes the owner of that selection, assuming the timestamp is valid. If the new owner is different from the old owner, the old owner is simply sent a SelectionClear event telling it that it lost ownership. The old owner is not asked for permission.

To return the selection owner, use **XGetSelectionOwner.**

```
0  Window XGetSelectionOwner (Display* display, Atom selection)
```

- display Specifies the connection to the X server.
- selection Specifies the selection atom whose owner you want returned.

The **XGetSelectionOwner** function returns the window ID associated with the window that currently owns the specified selection. If no selection was specified, the function returns the constant None. If None is returned, there is no owner for the selection.

XGetSelectionOwner can generate a **BadAtom** error.

To request conversion of a selection, use **XConvertSelection.**

```
0  XConvertSelection (
1      Display* display,
2      Atom selection,
3      Atom target,
4      Atom property,
5      Window requestor,
6      Time time
7  )
```

- display Specifies the connection to the X server.
- selection Specifies the selection atom.
- target Specifies the target atom.
- property Specifies the property name. You also can pass None.
- requestor Specifies the requestor.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

XConvertSelection requests that the specified selection be converted to the specified target type:

- If the specified selection has an owner, the X server sends a **SelectionRequest** event to that owner.
- If no owner for the specified selection exists, the X server generates a **SelectionNotify** event to the requestor with property **None.**

The arguments are passed on unchanged in either of the events. There are two predefined selection atoms: **PRIMARY** and **SECONDARY.**

XConvertSelection can generate **BadAtom** and **BadWindow** errors.

The owner of the selection is responsible for deciding what data to put into the requestor's property. The flow is

1. requestor calls **XConvertSelection**
2. X server sends **SelectionRequest** to the owner
3. owner examines the request
4. owner writes data into a property on the **requestor's** window
5. owner sends **SelectionNotify** back to the requestor
6. requestor reads the property with **XGetWindowProperty**

5. Pixmap and cursor functions

- Once you have connected to an X server, you can use the Xlib functions to:
  - Create and free pixmaps
  - Create, recolor, and free cursors
- **Drawables**: In Xlib, a Drawable is a generic X resource that you can draw into or copy from.

  The two main drawable resource types are

```
Window
Pixmap
```

  In Xlib headers, they are all just X resource IDs:

```
typedef XID Drawable;
typedef XID Window;
typedef XID Pixmap;
```

  Many drawing functions take a Drawable

```
XDrawLine(Display *display,
    Drawable d,
    GC gc,
    int x1, int y1,
    int x2, int y2
);
```

  The `d` argument may be a window or a pixmap. A Window drawable represents an on-screen drawing destination. If you draw into a window, the result may appear on the display, subject to clipping, exposure, obscuring windows, child windows, and so on.

  A Pixmap drawable represents off-screen storage on the X server. You can draw into it, then later copy it into a window:

- **Creating and Freeing Pixmaps**: Pixmaps can only be used on the screen on which they were created. Pixmaps are off-screen resources that are used for various operations, such as defining cursors as tiling patterns or as the source for certain raster operations. Most graphics requests can operate either on a window or on a pixmap. A bitmap is a single bit-plane pixmap.

To create a pixmap of a given size, use **XCreatePixmap.**

```
Pixmap XCreatePixmap(Display* display, Drawable d, unsigned
    ↪  int width, unsigned int height, unsigned int depth)
```

- display Specifies the connection to the X server.
- d Specifies which screen the pixmap is created on.
- width, height Specify the width and height, which define the dimensions of the pixmap.
- depth Specifies the depth of the pixmap.

The **XCreatePixmap** function creates a pixmap of the width, height, and depth you specified and returns a pixmap ID that identifies it. It is valid to pass an **InputOnly** window to the drawable argument. The width and height arguments must be nonzero, or a **BadValue** error results. The depth argument must be one of the depths supported by the screen of the specified drawable, or a BadValue error results.

The server uses the specified drawable to determine on which screen to create the pixmap. The pixmap can be used only on this screen and only with other drawables of the same depth (see **XCopyPlane** for an exception to this rule). The initial contents of the pixmap are undefined. **XCreatePixmap** can generate BadAlloc, **BadDrawable,** and **BadValue** errors.

The `d` argument is important. You are not drawing into d; you are using it to tell Xlib/X server which screen the pixmap should be created on. Pixmaps can only be used on the screen where they were created.

A Drawable specifies the screen because every drawable resource is already associated with a particular X screen inside the X server.

To free all storage associated with a specified pixmap, use **XFreePixmap.**

```
XFreePixmap (Display* display, Pixmap pixmap)
```

- display Specifies the connection to the X server.
- pixmap Specifies the pixmap.

The **XFreePixmap** function first deletes the association between the pixmap ID and the pixmap. Then, the X server frees the pixmap storage when there are no references to it. The pixmap should never be referenced again.

XFreePixmap can generate a **BadPixmap** error.

- **Xlib cursors**: In Xlib, a cursor is not modeled as a modern full-color image. It is modeled as a small two-color bitmap cursor with optional transparency and a logical "active point."

A cursor has four main parts:

1. source pixmap
2. mask pixmap
3. foreground/background colors
4. hotspot

The **source pixmap** is the actual 1-bit cursor image. Since it has depth 1, each pixel is either 0 or 1, not a full RGB color.

The **colors** tell X how to interpret those bits. Typically:

- source bit 1 -> foreground color
- source bit 0 -> background color

The **mask pixmap** controls which cursor pixels are visible. It must also have depth 1.

- mask bit 0 -> transparent / not part of cursor
- mask bit 1 -> visible cursor pixel

So the mask gives the cursor its outline or shape. Without a mask, the cursor would effectively be a full rectangle of source pixels. With a mask, only selected pixels are drawn as the cursor.

The **hotspot** is the coordinate inside the cursor image that counts as the actual pointer location.

For example, with an arrow cursor, the visible image may be 16 by 16 pixels, but the actual click point is usually the arrow tip:

- **Fonts**: In old X11 terminology, a **font** is a server-side collection of **glyph bitmaps**. A glyph is just a small bitmap shape. Xlib can use those bitmap glyphs not only for drawing characters, but also for constructing cursors.

  X provides a special built-in font named something like the cursor font. It contains predefined cursor glyphs: arrows, crosses, watches, hands, etc. The constants in:

  ```
  0  #include <X11/cursorfont.h>
  ```

  Which contains

  - **XC_X_cursor**: An "X" shaped cursor. Often used as a generic default or placeholder cursor.
  - **XC_arrow**: A generic arrow cursor.
  - **XC_based_arrow_down**: A downward-pointing arrow with a base/bar.
  - **XC_based_arrow_up**: An upward-pointing arrow with a base/bar.
  - **XC_boat**: A small boat-shaped cursor. Mostly decorative/legacy.
  - **XC_bogosity**: A strange "bad/invalid/weird" symbol. Mostly humorous/legacy.

- **XC_bottom_left_corner**: Resize cursor for the bottom-left corner of a window.
- **XC_bottom_right_corner**: Resize cursor for the bottom-right corner of a window.
- **XC_bottom_side**: Resize cursor for the bottom edge of a window.
- **XC_bottom_tee**: A T-shaped cursor associated with the bottom side.
- **XC_box_spiral**: A box/square spiral shape. Mostly decorative.
- **XC_center_ptr**: A pointer arrow whose hotspot is more centered than XC_left_ptr.
- **XC_circle**: A circular cursor.
- **XC_clock**: A clock-shaped cursor, usually indicating waiting/busy state.
- **XC_coffee_mug**: A coffee mug cursor. Mostly decorative/legacy.
- **XC_cross**: A simple cross-shaped cursor.
- **XC_cross_reverse**: A reverse/inverted version of the cross cursor.
- **XC_crosshair**: A precision crosshair cursor, often used for targeting or coordinate selection.
- **XC_diamond_cross**: A cross combined with a diamond-like shape.
- **XC_dot**: A small dot cursor.
- **XC_dotbox**: A dot inside a box. Useful for precise pointing.
- **XC_double_arrow**: A double-ended arrow, generally suggesting resizing or directional adjustment.
- **XC_draft_large**: A large drafting/drawing-style cursor.
- **XC_draft_small**: A smaller drafting/drawing-style cursor.
- **XC_draped_box**: A box-like cursor with a draped/covered appearance. Mostly decorative.
- **XC_exchange**: An exchange/swap cursor, suggesting replacement or swapping.
- **XC_fleur**: A four-direction move cursor, like arrows pointing up/down/left/right.
- **XC_gobbler**: A "gobbler"/Pac-Man-like decorative cursor.
- **XC_gumby**: A Gumby-like figure cursor. Decorative/legacy.
- **XC_hand1**: A hand cursor, usually pointing or selecting.
- **XC_hand2**: Another hand cursor variant. Commonly used for links or clickable things.
- **XC_heart**: A heart-shaped cursor.
- **XC_icon**: An icon-like cursor, historically associated with window/icon manipulation.
- **XC_iron_cross**: A heavy cross shape resembling an iron cross.
- **XC_left_ptr**: The usual left-leaning arrow pointer. This is the common default pointer.
- **XC_left_side**: Resize cursor for the left edge of a window.
- **XC_left_tee**: A T-shaped cursor associated with the left side.
- **XC_leftbutton**: A mouse/button symbol emphasizing the left button.
- **XC_ll_angle**: Lower-left angle/corner shape.
- **XC_lr_angle**: Lower-right angle/corner shape.
- **XC_man**: A small human figure cursor.

- **XC_middlebutton**: A mouse/button symbol emphasizing the middle button.
- **XC_mouse**: A mouse-shaped cursor.
- **XC_pencil**: A pencil cursor, commonly used for drawing/editing.
- **XC_pirate**: A pirate/skull-style cursor, often used for destructive or dangerous actions.
- **XC_plus**: A plus-sign cursor. Often suggests adding, inserting, or selecting.
- **XC_question_arrow**: An arrow with a question mark, commonly used for help/context help.
- **XC_right_ptr**: A right-leaning or right-pointing pointer arrow.
- **XC_right_side**: Resize cursor for the right edge of a window.
- **XC_right_tee**: A T-shaped cursor associated with the right side.
- **XC_rightbutton**: A mouse/button symbol emphasizing the right button.
- **XC_rtl_logo**: A stylized logo cursor. Rarely used in modern programs.
- **XC_sailboat**: A sailboat-shaped cursor. Decorative/legacy.
- **XC_sb_down_arrow**: Scrollbar-style down arrow.
- **XC_sb_h_double_arrow**: Horizontal double arrow, commonly suggesting horizontal resizing or scrolling.
- **XC_sb_left_arrow**: Scrollbar-style left arrow.
- **XC_sb_right_arrow**: Scrollbar-style right arrow.
- **XC_sb_up_arrow**: Scrollbar-style up arrow.
- **XC_sb_v_double_arrow**: Vertical double arrow, commonly suggesting vertical resizing or scrolling.
- **XC_shuttle**: A space-shuttle-shaped cursor. Decorative/legacy.
- **XC_sizing**: Generic sizing/resizing cursor.
- **XC_spider**: Spider-shaped cursor. Decorative/legacy.
- **XC_spraycan**: Spray-can cursor, commonly associated with painting/drawing programs.
- **XC_star**: Star-shaped cursor.
- **XC_target**: Target/bullseye cursor.
- **XC_tcross**: A T-like cross cursor.
- **XC_top_left_arrow**: Arrow pointing toward the upper-left.
- **XC_top_left_corner**: Resize cursor for the top-left corner of a window.
- **XC_top_right_corner**: Resize cursor for the top-right corner of a window.
- **XC_top_side**: Resize cursor for the top edge of a window.
- **XC_top_tee**: A T-shaped cursor associated with the top side.
- **XC_trek**: A Star-Trek-like insignia cursor. Decorative/legacy.
- **XC_ul_angle**: Upper-left angle/corner shape.
- **XC_umbrella**: Umbrella-shaped cursor. Decorative/legacy.
- **XC_ur_angle**: Upper-right angle/corner shape.
- **XC_watch**: Watch cursor, commonly used to indicate busy/waiting.
- **XC_xterm**: Text insertion/I-beam cursor, commonly used over text areas.

The practical / common ones is the subset

Cursor	Description	Common use
XC_left_ptr	Standard arrow pointer, usually angled up-left.	Default cursor for normal pointing and selection.
XC_xterm	Text insertion cursor, usually an I-beam.	Used over editable or selectable text regions.
XC_watch	Watch/busy cursor.	Indicates that the application is busy or waiting.
XC_hand1	Hand-shaped cursor, first variant.	Used for clickable or draggable objects.
XC_hand2	Hand-shaped cursor, second variant.	Commonly used for links or clickable UI elements.
XC_crosshair	Crosshair cursor with horizontal and vertical lines.	Precision pointing, drawing, targeting, coordinate selection.
XC_fleur	Four-direction movement cursor.	Moving windows, panels, objects, or selections.
XC_sb_h_double_arrow	Horizontal double-ended arrow.	Horizontal resizing or horizontal scrollbar interaction.
XC_sb_v_double_arrow	Vertical double-ended arrow.	Vertical resizing or vertical scrollbar interaction.
XC_top_side	Resize cursor for the top edge.	Resizing a window or object from its top border.
XC_bottom_side	Resize cursor for the bottom edge.	Resizing from the bottom border.
XC_left_side	Resize cursor for the left edge.	Resizing from the left border.
XC_right_side	Resize cursor for the right edge.	Resizing from the right border.
XC_top_left_corner	Diagonal resize cursor for the top-left corner.	Resizing from the upper-left corner.
XC_top_right_corner	Diagonal resize cursor for the top-right corner.	Resizing from the upper-right corner.
XC_bottom_left_corner	Diagonal resize cursor for the bottom-left corner.	Resizing from the lower-left corner.
XC_bottom_right_corner	Diagonal resize cursor for the bottom-right corner.	Resizing from the lower-right corner.

- **Font**: In Xlib, Font is an X resource ID referring to a font loaded on the X server.

  It is usually defined conceptually like this:

```
typedef XID font;
```

  So Font is not a C struct containing glyph data directly. It is an integer-like handle that identifies a server-side font resource.

  For normal text drawing, a font contains glyphs for characters. For cursor creation, Xlib can also use those glyph bitmaps as cursor images.

```
Font font = XLoadFont(display, "cursor");
```

This loads the standard X cursor font and returns a Font ID. Then XCreateGlyphCursor can use glyphs from that font as the cursor's source and mask.

- **XColor**: XColor is a structure that represents a color in Xlib. It is commonly defined like this

```
typedef struct {
    unsigned long pixel;
    unsigned short red, green, blue;
    char flags;
    char pad;
} XColor;
```

These are 16-bit color components, not 8-bit components. So their values range from:

```
0      // minimum intensity
65535  // maximum intensity
```

pixel is the actual pixel value used by the X server for a color.

It is not necessarily an RGB value in the ordinary sense. It is the value that can be placed into a drawable or into a GC foreground/background field.

The meaning of pixel depends on the visual and colormap.

- **Creating, Recoloring, and Freeing Cursors**: Each window can have a different cursor defined for it. Whenever the pointer is in a visible window, it is set to the cursor defined for that window. If no cursor was defined for that window, the cursor is the one defined for the parent window.

From X's perspective, a cursor consists of a cursor source, mask, colors, and a hotspot. The mask pixmap determines the shape of the cursor and must be a depth of one. The source pixmap must have a depth of one, and the colors determine the colors of the source. The hotspot defines the point on the cursor that is reported when a pointer event occurs. There may be limitations imposed by the hardware on cursors as to size and whether a mask is implemented. **XQueryBestCursor** can be used to find out what sizes are possible. There is a standard font for creating cursors, but Xlib provides functions that you can use to create cursors from arbitrary fonts or from bitmaps.

To create a cursor from the standard cursor font, use **XCreateFontCursor**.

```
#include <X11/cursorfont.h>

Cursor XCreateFontCursor(Display *display, unsigned int
    ↪  shape);
```

  - *display* specifies the connection to the X server.
  - *shape* specifies the shape of the cursor.

X provides a set of standard cursor shapes in a special font named *cursor*. Applications are encouraged to use this interface for their cursors because they cannot customize for the individual display type. The *shape* argument specifies which glyph of the standard fonts to use.

The hotspot comes from the information stored in the cursor font. The initial colors of a cursor are a black foreground and a white background. See **XRecolorCursor**. For further information about cursor shapes, see appendix B.

XCreateFontCursor can generate **BadAlloc** and **BadValue** errors.

To create a cursor from font glyphs, use **XCreateGlyphCursor**.

To create a cursor from font glyphs, use **XCreateGlyphCursor**.

```
Cursor XCreateGlyphCursor(
    Display *display,
    Font source_font,
    Font mask_font,
    unsigned int source_char,
    unsigned int mask_char,
    XColor *foreground_color,
    XColor *background_color
);
```

- *display* specifies the connection to the X server.
- *source_font* specifies the font for the source glyph.
- *mask_font* specifies the font for the mask glyph, or **None**.
- *source_char* specifies the character glyph for the source.
- *mask_char* specifies the glyph character for the mask.
- *foreground_color* specifies the RGB values for the foreground of the source.
- *background_color* specifies the RGB values for the background of the source.

The **XCreateGlyphCursor** function is similar to **XCreatePixmapCursor** except that the source and mask bitmaps are obtained from the specified font glyphs. The *source_char* must be a defined glyph in *source_font*, or a **BadValue** error results. If *mask_font* is given, *mask_char* must be a defined glyph in *mask_font*, or a **BadValue** error results. The *mask_font* and character are optional. The origins of the *source_char* and *mask_char*, if defined, glyphs are positioned coincidently and define the hotspot. The *source_char* and *mask_char* need not have the same bounding box metrics, and there is no restriction on the placement of the hotspot relative to the bounding boxes. If no *mask_char* is given, all pixels of the source are displayed. You can free the fonts immediately by calling **XFreeFont** if no further explicit references to them are to be made.

For 2-byte matrix fonts, the 16-bit value should be formed with the *byte1* member in the most significant byte and the *byte2* member in the least significant byte.

XCreateGlyphCursor can generate **BadAlloc**, **BadFont**, and **BadValue** errors.

Note: To understand, think of a font as a table of small bitmap drawings. Each drawing in the font is selected by a number. That number is what Xlib calls a "character," even if it is not a normal letter.

```
source_font + source_char
```

means: Use glyph number source_char from source_font as the cursor's source image.

```
mask_font + mask_char
```

means: Use glyph number mask_char from mask_font as the cursor's transparency mask

```
source_font + source_char  -> choose the source glyph
mask_font   + mask_char    -> choose the mask glyph
```

The **source glyph** controls the cursor's black/white pattern. Since it is a 1-bit glyph, each pixel is either 0 or 1

```
source bit 1 -> foreground color
source bit 0 -> background color
```

The mask glyph controls which pixels are visible at all.

```
mask bit 1 -> show this cursor pixel
mask bit 0 -> transparent pixel
```

Essentially,

```
if mask bit is 0:
    draw nothing

if mask bit is 1:
    if source bit is 1:
        draw foreground color
    else:
        draw background color
```

To create a cursor from two bitmaps, use **XCreatePixmapCursor**.

```
Cursor XCreatePixmapCursor(
    Display* display,
    Pixmap source,
    Pixmap mask,
    XColor* foreground_color,
    XColor* background_color,
    unsigned int x, unsigned int y
)
```

display Specifies the connection to the X server.

source Specifies the shape of the source cursor.

mask Specifies the cursor source bits to be displayed or **None**.

foreground_color Specifies the RGB values for the foreground of the source.

background_color Specifies the RGB values for the background of the source.

x, *y* Specify the *x* and *y* coordinates, which indicate the hotspot relative to the source's origin.

The **XCreatePixmapCursor** function creates a cursor and returns the cursor ID associated with it. The foreground and background RGB values must be specified using *foreground_color* and *background_color*, even if the X server only has **StaticGray** or **GrayScale** color. The foreground color is used for the pixels set to 1 in the source, and the background color is used for the pixels set to 0. Both source and mask, if specified, must have depth one, or a **BadMatch** error results, but can have any root. The mask argument defines the shape of the cursor. The pixels set to 1 in the mask define which source pixels are displayed, and the pixels set to 0 define which pixels are ignored. If no mask is given, all pixels of the source are displayed. The mask, if present, must be the same size as the pixmap defined by the source argument, or a **BadMatch** error results. The hotspot must be a point within the source, or a **BadMatch** error results.

The components of the cursor can be transformed arbitrarily to meet display limitations. The pixmaps can be freed immediately if no further explicit references to them are to be made. Subsequent drawing in the source or mask pixmap has an undefined effect on the cursor. The X server might or might not make a copy of the pixmap.

XCreatePixmapCursor can generate **BadAlloc** and **BadPixmap** errors.

To determine useful cursor sizes, use **XQueryBestCursor**.

```
Status XQueryBestCursor(
    Display* display,
    Drawable d,
    unsigned int width, height,
    unsigned int* width_return, height_return
)
```

- *display* Specifies the connection to the X server.
- *d* Specifies the drawable, which indicates the screen.

- *width*, *height* Specify the width and height of the cursor that you want the size information for.
- *width_return*, *height_return* Return the best width and height that is closest to the specified width and height.

Some displays allow larger cursors than other displays. The **XQueryBestCursor** function provides a way to find out what size cursors are actually possible on the display. It returns the largest size that can be displayed. Applications should be prepared to use smaller cursors on displays that cannot support large ones.

XQueryBestCursor can generate a **BadDrawable** error.

To change the color of a given cursor, use **XRecolorCursor.**

```
XRecolorCursor (display, cursor, foreground_color,
↪ background_color)
```

- display Specifies the connection to the X server.
- cursor Specifies the cursor.
- foreground_color Specifies the RGB values for the foreground of the source.
- background_color Specifies the RGB values for the background of the source.

The **XRecolorCursor** function changes the color of the specified cursor, and if the cursor is being displayed on a screen, the change is visible immediately. The pixel members of the **XColor** structures are ignored; only the RGB values are used.

XRecolorCursor can generate a BadCursor error.

To free (destroy) a given cursor, use **XFreeCursor.**

```
XFreeCursor (display, cursor)
```

- display Specifies the connection to the X server.
- cursor Specifies the cursor.

The **XFreeCursor** function deletes the association between the cursor resource ID and the specified cursor. The cursor storage is freed when no other resource references it. The specified cursor ID should not be referred to again.

XFreeCursor can generate a BadCursor error.

6. Color management functions

- **Intro**: Each X window always has an associated colormap that provides a level of indirection between pixel values and colors displayed on the screen. Xlib provides functions that you can use to manipulate a colormap. The X protocol defines colors using values in the RGB color space. The RGB color space is device dependent; rendering an RGB value on differing output devices typically results in different colors. Xlib also provides a means for clients to specify color using device-independent color spaces for consistent results across devices. Xlib supports device-independent color spaces derivable from the CIE XYZ color space. This includes the CIE XYZ, xyY, L*u*v*, and L*a*b* color spaces as well as the TekHVC color space.

  All functions, types, and symbols in this chapter with the prefix ''Xcms'' are defined in <X11/Xcms.h>. The remaining functions and types are defined in <X11/Xlib.h>.

  Functions in this chapter manipulate the representation of color on the screen. For each possible value that a pixel can take in a window, there is a color cell in the colormap. For example, if a window is 4 bits deep, pixel values 0 through 15 are defined. A colormap is a collection of color cells. A color cell consists of a triple of red, green, and blue (RGB) values. The hardware imposes limits on the number of significant bits in these values. As each pixel is read out of display memory, the pixel is looked up in a colormap. The RGB value of the cell determines what color is displayed on the screen. On a grayscale display with a black-and-white monitor, the values are combined to determine the brightness on the screen.

  Typically, an application allocates color cells or sets of color cells to obtain the desired colors. The client can allocate read-only cells. In which case, the pixel values for these colors can be shared among multiple applications, and the RGB value of the cell cannot be changed. If the client allocates read/write cells, they are exclusively owned by the client, and the color associated with the pixel value can be changed at will. Cells must be allocated (and, if read/write, initialized with an RGB value) by a client to obtain desired colors. The use of pixel value for an unallocated cell results in an undefined color.

  Because colormaps are associated with windows, X supports displays with multiple colormaps and, indeed, different types of colormaps. If there are insufficient colormap resources in the display, some windows will display in their true colors, and others will display with incorrect colors. A window manager usually controls which windows are displayed in their true colors if more than one colormap is required for the color resources the applications are using. At any time, there is a set of installed colormaps for a screen. Windows using one of the installed colormaps display with true colors, and windows using other colormaps generally display with incorrect colors. You can control the set of installed colormaps by using **XInstallColormap** and **XUninstall-Colormap**

  Colormaps are local to a particular screen. Screens always have a default colormap, and programs typically allocate cells out of this colormap. Generally, you should not write applications that monopolize color resources. Although some hardware supports multiple colormaps installed at one time, many of the hardware displays built today support only a single installed colormap, so the primitives are written to encourage sharing of colormap entries between applications.

The **DefaultColormap** macro returns the default colormap. The **DefaultVisual** macro returns the default visual type for the specified screen. Possible visual types are **StaticGray, GrayScale, StaticColor, PseudoColor, TrueColor,** or **DirectColor**

- **Color structures**: Functions that operate only on RGB color space values use an **XColor** structure, which contains:

```
typedef struct {
    unsigned long pixel; /* pixel value */
    unsigned short red, green, blue; /* rgb values */
    char flags; /* DoRed, DoGreen, DoBlue */
    char pad;
} XColor;
```

The red, green, and blue values are always in the range 0 to 65535 inclusive, independent of the number of bits actually used in the display hardware. The server scales these values down to the range used by the hardware. Black is represented by (0,0,0), and white is represented by (65535,65535,65535). In some functions, the flags member controls which of the red, green, and blue members is used and can be the inclusive OR of zero or more of **DoRed, DoGreen,** and **DoBlue**

Functions that operate on all color space values use an **XcmsColor** structure. This structure contains a union of substructures, each supporting color specification encoding for a particular color space. Like the **XColor** structure, the **XcmsColor** structure contains pixel and color specification information (the spec member in the XcmsColor structure).

```
typedef unsigned long XcmsColorFormat;/* Color Specification
↪  Format */
```

```
typedef struct {
    union {
        XcmsRGB RGB;
        XcmsRGBi RGBi;
        XcmsCIEXYZ CIEXYZ;
        XcmsCIEuvY CIEuvY;
        XcmsCIExyY CIExyY;
        XcmsCIELab CIELab;
        XcmsCIELuv CIELuv;
        XcmsTekHVC TekHVC;
        XcmsPad Pad;
    } spec;
    unsigned long pixel;
    XcmsColorFormat format;
} XcmsColor; /* Xcms Color Structure */
```

Because the color specification can be encoded for the various color spaces, encoding for the spec member is identified by the format member, which is of type **XcmsColorFormat.** The following macros define standard formats.

```
#define XcmsUndefinedFormat 0x00000000
#define XcmsCIEXYZFormat 0x00000001 /* CIE XYZ */
#define XcmsCIEuvYFormat 0x00000002 /* CIE u'v'Y */
#define XcmsCIExyYFormat 0x00000003 /* CIE xyY */
#define XcmsCIELabFormat 0x00000004 /* CIE L*a*b* */
#define XcmsCIELuvFormat 0x00000005 /* CIE L*u*v* */
#define XcmsTekHVCFormat 0x00000006 /* TekHVC */
#define XcmsRGBFormat 0x80000000 /* RGB Device */
#define XcmsRGBiFormat 0x80000001 /* RGB Intensity */
```

Formats for device-independent color spaces are distinguishable from those for device-dependent spaces by the 32nd bit. If this bit is set, it indicates that the color specification is in a device- dependent form; otherwise, it is in a device-independent form. If the 31st bit is set, this indicates that the color space has been added to Xlib at run time (see section 6.12.4). The format value for a color space added at run time may be different each time the program is executed. If references to such a color space must be made outside the client (for example, storing a color specification in a file), then reference should be made by color space string prefix (see **XcmsFormatOfPrefix** and **XcmsPrefixOfFormat).**

Data types that describe the color specification encoding for the various color spaces are defined as follows:

```
typedef double XcmsFloat;
typedef struct {
    unsigned short red; /* 0x0000 to 0xffff */
    unsigned short green; /* 0x0000 to 0xffff */
    unsigned short blue; /* 0x0000 to 0xffff */
} XcmsRGB; /* RGB Device */

typedef struct {
    XcmsFloat red; /* 0.0 to 1.0 */
    XcmsFloat green; /* 0.0 to 1.0 */
    XcmsFloat blue; /* 0.0 to 1.0 */
} XcmsRGBi; /* RGB Intensity */

typedef struct {
    XcmsFloat X;
    XcmsFloat Y; /* 0.0 to 1.0 */
    XcmsFloat Z;
} XcmsCIEXYZ; /* CIE XYZ */

typedef struct {
    XcmsFloat u_prime; /* 0.0 to ~0.6 */
    XcmsFloat v_prime; /* 0.0 to ~0.6 */
    XcmsFloat Y; /* 0.0 to 1.0 */
} XcmsCIEuvY; /* CIE u'v'Y */
```

```
typedef struct {
    XcmsFloat x; /* 0.0 to ~.75 */
    XcmsFloat y; /* 0.0 to ~.85 */
    XcmsFloat Y; /* 0.0 to 1.0 */
} XcmsCIExyY; /* CIE xyY */

typedef struct {
    XcmsFloat L_star; /* 0.0 to 100.0 */
    XcmsFloat a_star;
    XcmsFloat b_star;
} XcmsCIELab; /* CIE L*a*b* */

typedef struct {
    XcmsFloat L_star; /* 0.0 to 100.0 */
    XcmsFloat u_star;
    XcmsFloat v_star;
} XcmsCIELuv; /* CIE L*u*v* */

typedef struct {
    XcmsFloat H; /* 0.0 to 360.0 */
    XcmsFloat V; /* 0.0 to 100.0 */
    XcmsFloat C; /* 0.0 to 100.0 */
} XcmsTekHVC; /* TekHVC */

typedef struct {
    XcmsFloat pad0;
    XcmsFloat pad1;
    XcmsFloat pad2;
    XcmsFloat pad3;
} XcmsPad; /* four doubles */
```

The device-dependent formats provided allow color specification in:

- **RGB Intensity (XcmsRGBi)**: Red, green, and blue linear intensity values, floating-point values from 0.0 to 1.0, where 1.0 indicates full intensity, 0.5 half intensity, and so on.
- **RGB Device (XcmsRGB)**: Red, green, and blue values appropriate for the specified output device. **XcmsRGB** values are of type unsigned short, scaled from 0 to 65535 inclusive, and are interchangeable with the red, green, and blue values in an **XColor** structure.

It is important to note that RGB Intensity values are not gamma corrected values. In contrast, RGB Device values generated as a result of converting color specifications are always gamma corrected, and RGB Device values acquired as a result of querying a colormap or passed in by the client are assumed by Xlib to be gamma corrected. The term RGB value in this manual always refers to an RGB Device value.

- **Display color gamut**: Specific range of colors a screen can reproduce.
- **Gamma correction**: Gamma correction is the process of adjusting color or brightness values to account for the fact that displays do not produce light linearly.

  Ideally, you might think that if you send a display a value of 50%, it produces 50% brightness. In reality, many displays have a nonlinear response. A value halfway between black and white does not necessarily produce half the physical light output.

So gamma correction compensates for this nonlinear relationship.

Mathematically, it is often modeled like this:

$$\text{output brightness} \;=\; \text{input value}^{\gamma},$$

where γ is the **gamma value**. A common display gamma is around

$$\gamma \approx 2.2.$$

That means displays often make raw input values appear darker than a linear brightness model would suggest.

For example, suppose a pixel channel is normalized between 0 and 1:

$$0.5^{2.2} \approx 0.218.$$

So a numeric value of 0.5 may produce only about 21.8% of maximum physical light output, not 50%.

Gamma correction adjusts the stored or transmitted values so the displayed result looks perceptually correct.

In color systems, this matters because RGB values are usually not pure physical light intensities. They are often encoded values meant to look correct on a display. For example, an RGB value like:

```
R = 128, G = 128, B = 128
```

is not necessarily "half the light" of white. It is a gamma-encoded middle gray intended to look visually halfway between black and white.

Gamma correcting an RGB value means changing the numeric channel values so that the final displayed brightness matches the intended perceived or physical brightness.

- **Color strings**: Xlib provides a mechanism for using string names for colors. A color string may either contain an abstract color name or a numerical color specification. Color strings are case-insensitive.

  Color strings are used in the following functions:

  - XAllocNamedColor
  - XcmsAllocNamedColor
  - XLookupColor
  - XcmsLookupColor
  - XParseColor
  - XStoreNamedColor

  Xlib supports the use of abstract color names, for example, red or blue. A value for this abstract name is obtained by searching one or more color name databases. Xlib first searches zero or more client-side databases; the number, location, and content of these databases is implementation-dependent and might depend on the current locale. If the name is not found, Xlib then looks for the color in the X server's database. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent.

A numerical color specification consists of a color space name and a set of values in the following syntax:

```
<color_space_name>:<value>/.../<value>
```

The following are examples of valid color strings.

- "CIEXYZ:0.3227/0.28133/0.2493"
- "RGBi:1.0/0.0/0.0"
- "rgb:00/ff/00"
- "CIELuv:50.0/0.0/0.0"

The syntax and semantics of numerical specifications are given for each standard color space in the following sections.

- **RGB Device String Specification**: An RGB Device specification is identified by the prefix ‘‘rgb:’’ and conforms to the following syntax:

```
rgb:<red>/<green>/<blue>

<red>, <green>, <blue> := h  hh  hhh | hhhh
h := single hexadecimal digits (case insignificant)
```

Note that *h* indicates the value scaled in 4 bits, *hh* the value scaled in 8 bits, *hhh* the value scaled in 12 bits, and hhhh the value scaled in 16 bits, respectively.

For backward compatibility, an older syntax for RGB Device is supported, but its continued use is not encouraged. The syntax is an initial sharp sign character followed by a numeric specification, in one of the following formats:

```
#RGB (4 bits each)
#RRGGBB (8 bits each)
#RRRGGGBBB (12 bits each)
#RRRRGGGGBBBB (16 bits each)
```

The R, G, and B represent single hexadecimal digits. When fewer than 16 bits each are specified, they represent the most significant bits of the value (unlike the ‘‘rgb:’’ syntax, in which values are scaled). For example, the string ‘‘#3a7’’ is the same as ‘‘#3000a0007000’’.

- **RGB Intensity String Specification**: An RGB intensity specification is identified by the prefix ‘‘rgbi:’’ and conforms to the following syntax:

```
rgbi:<red>/<green>/<blue>
```

Note that red, green, and blue are floating-point values between 0.0 and 1.0, inclusive. The input format for these values is an optional sign, a string of numbers possibly containing a decimal point, and an optional exponent field containing an E or e followed by a possibly signed integer string.

- **Device-Independent String Specifications**: The standard device-independent string specifications have the following syntax:

```
CIEXYZ:<X>/<Y>/<Z>
CIEuvY:<u>/<v>/<Y>
CIExyY:<x>/<y>/<Y>
CIELab:<L>/<a>/<b>
CIELuv:<L>/<u>/<v>
TekHVC:<H>/<V>/<C>
```

All of the values (C, H, V, X, Y, Z, a, b, u, v, y, x) are floating-point values. The syntax for these values is an optional plus or minus sign, a string of digits possibly containing a decimal point, and an optional exponent field consisting of an ''E'' or ''e'' followed by an optional plus or minus followed by a string of digits.

- **Color Conversion Contexts and Gamut Mapping**: When Xlib converts device-independent color specifications into device-dependent specifications and vice versa, it uses knowledge about the color limitations of the screen hardware. This information, typically called the device profile, is available in a Color Conversion Context (CCC).

Because a specified color may be outside the color gamut of the target screen and the white point associated with the color specification may differ from the white point inherent to the screen, Xlib applies gamut mapping when it encounters certain conditions:

  - Gamut compression occurs when conversion of device-independent color specifications to device-dependent color specifications results in a color out of the target screen's gamut.
  - White adjustment occurs when the inherent white point of the screen differs from the white point assumed by the client.

Gamut handling methods are stored as callbacks in the CCC, which in turn are used by the color space conversion routines. Client data is also stored in the CCC for each callback. The CCC also contains the white point the client assumes to be associated with color specifications (that is, the Client White Point). The client can specify the gamut handling callbacks and client data as well as the Client White Point. Xlib does not preclude the X client from performing other forms of gamut handling (for example, gamut expansion); however, Xlib does not provide direct support for gamut handling other than white adjustment and gamut compression.

Associated with each colormap is an initial CCC transparently generated by Xlib. Therefore, when you specify a colormap as an argument to an Xlib function, you are indirectly specifying a CCC. There is a default CCC associated with each screen. Newly created CCCs inherit attributes from the default CCC, so the default CCC attributes can be modified to affect new CCCs.

Xcms functions in which gamut mapping can occur return **Status** and have specific status values defined for them, as follows:

  - **XcmsFailure** indicates that the function failed.
  - **XcmsSuccess** indicates that the function succeeded. In addition, if the function performed any color conversion, the colors did not need to be compressed.
  - **XcmsSuccessWithCompression** indicates the function performed color conversion and at least one of the colors needed to be compressed. The gamut compression method is determined by the gamut compression procedure in the CCC that is specified directly as a function argument or in the CCC indirectly specified by means of the colormap argument.

- **Creating, Copying, and Destroying Colormaps**:

  To create a colormap for a screen, use **XCreateColormap**.

```
Colormap XCreateColormap(
    Display *display,
    Window w,
    Visual *visual,
    int alloc
)
```

  - **display**: Specifies the connection to the X server.
  - **w**: Specifies the window on whose screen you want to create a colormap.
  - **visual**: Specifies a visual type supported on the screen. If the visual type is not one supported by the screen, a **BadMatch** error results.
  - **alloc**: Specifies the colormap entries to be allocated. You can pass **AllocNone** or **AllocAll**.

  The **XCreateColormap** function creates a colormap of the specified visual type for the screen on which the specified window resides and returns the colormap ID associated with it. Note that the specified window is only used to determine the screen.

  The initial values of the colormap entries are undefined for the visual classes **GrayScale**, **PseudoColor**, and **DirectColor**. For **StaticGray**, **StaticColor**, and **TrueColor**, the entries have defined values, but those values are specific to the visual and are not defined by X. For **StaticGray**, **StaticColor**, and **TrueColor**, **alloc** must be **AllocNone**, or a **BadMatch** error results. For the other visual classes, if **alloc** is **AllocNone**, the colormap initially has no allocated entries, and clients can allocate them.

  If **alloc** is **AllocAll**, the entire colormap is allocated writable. The initial values of all allocated entries are undefined. For **GrayScale** and **PseudoColor**, the effect is as if an **XAllocColorCells** call returned all pixel values from zero to $N - 1$, where N is the colormap entries value in the specified visual. For **DirectColor**, the effect is as if an **XAllocColorPlanes** call returned a pixel value of zero and `red_mask`, `green_mask`, and `blue_mask` containing the same bits as the corresponding masks in the specified visual. However, in all cases, none of these entries can be freed by using **XFreeColors**.

XCreateColormap can generate **BadAlloc**, **BadMatch**, **BadValue**, and **BadWindow** errors.

To create a new colormap when the allocation out of a previously shared colormap has failed because of resource exhaustion, use **XCopyColormapAndFree**.

```
Colormap XCopyColormapAndFree(
    Display* display,
    Colormap colormap
)
```

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.

The **XCopyColormapAndFree** function creates a colormap of the same visual type and for the same screen as the specified colormap and returns the new colormap ID. It also moves all of the client's existing allocation from the specified colormap to the new colormap with their color values intact and their read-only or writable characteristics intact and frees those entries in the specified colormap. Color values in other entries in the new colormap are undefined.

If the specified colormap was created by the client with **alloc** not set to **AllocAll**, the new colormap is also created with **AllocAll**, all color values for all entries are copied from the specified colormap, and then all entries in the specified colormap are freed.

If the specified colormap was not created by the client with **AllocAll**, the allocations to be moved are all those pixels and planes that have been allocated by the client using **XAllocColor**, **XAllocNamedColor**, **XAllocColorCells**, or **XAllocColorPlanes** and that have not been freed since they were allocated.

XCopyColormapAndFree can generate **BadAlloc** and **BadColor** errors.

Note: A colormap can be "exhausted" because it has a finite number of color cells.

If the visual has 8 colormap bits, there may be only 256 entries. Multiple clients may be sharing the same default colormap. If enough clients allocate entries, there may be no suitable entries left.

The "AndFree" part means it frees your color allocations in the old colormap. It does not necessarily destroy the old colormap object itself. To destroy a colormap resource, you use:

If the old colormap was created by your client with:

```
AllocAll
```

then the client owns the entire colormap. In that case, the new colormap is also treated like an **AllocAll** colormap. All entries are copied, and the old entries are freed.

If the old colormap was not created with **AllocAll,** then only the colors/cells that your client explicitly allocated are moved. That includes allocations made by functions such as:

XCopyColormapAndFree creates a new colormap with the same screen, same visual, and same number of entries as the old one. It does not increase the size of the colormap.

For example, if the old colormap had 256 entries, the new colormap also has 256 entries.

The benefit is that the new colormap does not carry over everyone else's allocations. It mainly preserves your client's allocated colors and leaves the rest available/undefined in the new colormap.

To destroy a colormap, use **XFreeColormap**.

```
XFreeColormap(Display *display, Colormap colormap)
```

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap that you want to destroy.

The **XFreeColormap** function deletes the association between the colormap resource ID and the colormap and frees the colormap storage. However, this function has no effect on the default colormap for a screen. If the specified colormap is an installed map for a screen, it is uninstalled. See **XUninstallColormap**.

If the specified colormap is defined as the colormap for a window by **XCreateWindow**, **XSetWindowColormap**, or **XChangeWindowAttributes**, **XFreeColormap** changes the colormap associated with the window to **None** and generates a **ColormapNotify** event. X does not define the colors displayed for a window with a colormap of **None**.

XFreeColormap can generate a **BadColor** error.

- **Mapping Color Names to Values**: To map a color name to an RGB value, use **XLookupColor**

```
Status XLookupColor(
    Display *display,
    Colormap colormap,
    char *color_name,
    XColor *exact_def_return,
    XColor *screen_def_return
)
```

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.

- **color_name**: Specifies the color name string, for example, `red`, whose color definition structure you want returned.
- **exact_def_return**: Returns the exact RGB values.
- **screen_def_return**: Returns the closest RGB values provided by the hardware.

The **XLookupColor** function looks up the string name of a color with respect to the screen associated with the specified colormap. It returns both the exact color values and the closest values provided by the screen with respect to the visual type of the specified colormap. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter. **XLookupColor** returns nonzero if the name is resolved; otherwise, it returns zero.

XLookupColor can generate a **BadColor** error.

To map a color name to the exact RGB value, use **XParseColor**.

```
0  Status XParseColor(Display *display, Colormap colormap, char
   ↪  *spec,
1  XColor *exact_def_return)
```

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.
- **spec**: Specifies the color name string; case is ignored.
- **exact_def_return**: Returns the exact color value for later use and sets the **DoRed**, **DoGreen**, and **DoBlue** flags.

The **XParseColor** function looks up the string name of a color with respect to the screen associated with the specified colormap. It returns the exact color value. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter. **XParseColor** returns nonzero if the name is resolved; otherwise, it returns zero.

XParseColor can generate a **BadColor** error.

To map a color name to a value in an arbitrary color space, use **XcmsLookupColor**.

```
0  Status XcmsLookupColor(
1      Display *display,
2      Colormap colormap,
3      char *color_string,
4      XcmsColor *color_exact_return,
5      XcmsColor *color_screen_return,
6      XcmsColorFormat result_format
7  )
```

- **display**: Specifies the connection to the X server.

- **colormap**: Specifies the colormap.
- **color_string**: Specifies the color string.
- **color_exact_return**: Returns the color specification parsed from the color string or parsed from the corresponding string found in a color-name database.
- **color_screen_return**: Returns the color that can be reproduced on the screen.
- **result_format**: Specifies the color format for the returned color specifications, that is, the `color_screen_return` and `color_exact_return` arguments. If the format is **XcmsUndefinedFormat** and the color string contains a numerical color specification, the specification is returned in the format used in that numerical color specification. If the format is **XcmsUndefinedFormat** and the color string contains a color name, the specification is returned in the format used to store the color in the database.

The **XcmsLookupColor** function looks up the string name of a color with respect to the screen associated with the specified colormap. It returns both the exact color values and the closest values provided by the screen with respect to the visual type of the specified colormap. The values are returned in the format specified by `result_format`. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter. **XcmsLookupColor** returns **XcmsSuccess** or **XcmsSuccessWithCompression** if the name is resolved; otherwise, it returns **XcmsFailure**. If **XcmsSuccessWithCompression** is returned, the color specification returned in `color_screen_return` is the result of gamut compression.

- **Clearing up confusion**: The colormap is stored and managed by the X server, not by the client.

  The client only holds an XID, a numeric handle, that refers to the server-side colormap object.

  This is why multiple clients can share the same colormap. They are all referring to the same server-side resource by XID.

  For a client to allocate a read-only color cell means: The client asks the X server for a colormap entry representing some RGB color, and the server gives the client a pixel value it may use, but not modify.

  The important part is that the client has allocated permission to use that pixel value, not permission to change the RGB contents of the colormap cell.

  The word allocate here means the server records that this client is using that shared color cell. It increments an internal count so the entry is not freed while clients are still using it.

  When clients call **XFreeColors,** the server decrements that count. The cell is actually available for reuse only after all allocations have been freed.

- **Allocating and Freeing Color Cells**: There are two ways of allocating color cells: explicitly as read-only entries, one pixel value at a time, or read/write, where you can allocate a number of color cells and planes simultaneously. A read-only cell has its RGB value set by the server. Read/write cells do not have defined colors initially; functions described in the next section must be used to store values into them. Although it is possible for any client to store values into a read/write cell allocated by another client, read/write cells normally should be considered private to the client that allocated them.

Read-only colormap cells are shared among clients. The server counts each allocation and freeing of the cell by clients. When the last client frees a shared cell, the cell is finally deallocated. If a single client allocates the same read-only cell multiple times, the server counts each such allocation, not just the first one.

To allocate a read-only color cell with an RGB value, use **XAllocColor**

```
Status XAllocColor(
    Display *display,
    Colormap colormap,
    XColor *screen_in_out
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *screen_in_out*: Specifies and returns the values actually used in the colormap.

The **XAllocColor** function allocates a read-only colormap entry corresponding to the closest RGB value supported by the hardware. **XAllocColor** returns the pixel value of the closest to the specified RGB elements supported by the hardware and returns the RGB value actually used. The corresponding colormap cell is read-only. In addition, **XAllocColor** returns nonzero if it succeeded or zero if it failed. Multiple clients that request the same effective RGB value can be assigned the same read-only entry, thus allowing entries to be shared. When the last client deallocates a shared cell, it is deallocated. **XAllocColor** does not use or affect the flags in the **XColor** structure.

XAllocColor can generate a **BadColor** error.

To allocate a read-only color cell with a color in arbitrary format, use **XcmsAllocColor**.

```
Status XcmsAllocColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color_in_out,
    XcmsColorFormat result_format
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_in_out*: Specifies the color to allocate and returns the pixel and color that is actually used in the colormap.
- *result_format*: Specifies the color format for the returned color specification.

The **XcmsAllocColor** function is similar to **XAllocColor** except the color can be specified in any format. The **XcmsAllocColor** function ultimately calls **XAllocColor** to allocate a read-only color cell, or colormap entry, with the specified color. **XcmsAllocColor** first converts the color specified in *color_in_out* to an RGB value

and then passes it to **XAllocColor**. **XcmsAllocColor** returns the pixel value of the color cell and the color specification actually allocated. This returned color specification is the result of converting the RGB value returned by **XAllocColor** into the format specified with the *result_format* argument. If there is no interest in a returned color specification, unnecessary computation can be bypassed if *result_format* is set to **XcmsRGBFormat**. The corresponding colormap cell is read-only. If this routine returns **XcmsFailure**, the *color_in_out* color specification is left unchanged.

XcmsAllocColor can generate a **BadColor** error.

To allocate a read-only color cell using a color name and return the closest color supported by the hardware in RGB format, use **XAllocNamedColor**.

```
Status XAllocNamedColor(
    Display *display,
    Colormap colormap,
    char *color_name,
    XColor *screen_def_return,
    XColor *exact_def_return
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_name*: Specifies the color name string, for example, `red`, whose color definition structure you want returned.
- *screen_def_return*: Returns the closest RGB values provided by the hardware.
- *exact_def_return*: Returns the exact RGB values.

The **XAllocNamedColor** function looks up the named color with respect to the screen that is associated with the specified colormap. It returns both the exact database definition and the closest color supported by the screen. The allocated color cell is read-only. The pixel value is returned in *screen_def_return*. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter. If *screen_def_return* and *exact_def_return* point to the same structure, the pixel field will be set correctly, but the color values are undefined. **XAllocNamedColor** returns nonzero if a cell is allocated; otherwise, it returns zero.

XAllocNamedColor can generate a **BadColor** error.

To allocate a read-only color cell using a color name and return the closest color supported by the hardware in an arbitrary format, use **XcmsAllocNamedColor**.

```
Status XcmsAllocNamedColor(
    Display *display,
    Colormap colormap,
    char *color_string,
    XcmsColor *color_screen_return,
    XcmsColor *color_exact_return,
    XcmsColorFormat result_format
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_string*: Specifies the color string whose color definition structure is to be returned.
- *color_screen_return*: Returns the pixel value of the color cell and color specification that actually is stored for that cell.
- *color_exact_return*: Returns the color specification parsed from the color string or parsed from the corresponding string found in a color-name database.
- *result_format*: Specifies the color format for the returned color specifications, namely the *color_screen_return* and *color_exact_return* arguments. If the format is **XcmsUndefinedFormat** and the color string contains a numerical color specification, the specification is returned in the format used in that numerical color specification. If the format is **XcmsUndefinedFormat** and the color string contains a color name, the specification is returned in the format used to store the color in the database.

The **XcmsAllocNamedColor** function is similar to **XAllocNamedColor** except that the color returned can be in any format specified. This function ultimately calls **XAllocColor** to allocate a read-only color cell with the color specified by a color string. The color string is parsed into an **XcmsColor** structure, converted to an RGB value, and finally passed to **XAllocColor**. If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter.

This function returns both the color specification as a result of parsing, called the exact specification, and the actual color specification stored, called the screen specification. This screen specification is the result of converting the RGB value returned by **XAllocColor** into the format specified in *result_format*. If there is no interest in a returned color specification, unnecessary computation can be bypassed if *result_format* is set to **XcmsRGBFormat**. If *color_screen_return* and *color_exact_return* point to the same structure, the pixel field will be set correctly, but the color values are undefined.

XcmsAllocNamedColor can generate a **BadColor** error.

To allocate read/write color cell and color plane combinations for a **PseudoColor** model, use **XAllocColorCells**.

```
Status XAllocColorCells(
    Display *display,
    Colormap colormap,
    Bool contig,
    unsigned long plane_masks_return[],
    unsigned int nplanes,
    unsigned long pixels_return[],
    unsigned int npixels
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.

– *contig*: Specifies a Boolean value that indicates whether the planes must be contiguous.
– *plane_masks_return*: Returns an array of plane masks.
– *nplanes*: Specifies the number of plane masks that are to be returned in the plane masks array.
– *pixels_return*: Returns an array of pixel values.
– *npixels*: Specifies the number of pixel values that are to be returned in the *pixels_return* array.

The **XAllocColorCells** function allocates read/write color cells. The number of colors must be positive and the number of planes nonnegative, or a **BadValue** error results. If *npixels* and *nplanes* are requested, then *npixels* pixels and *nplanes* plane masks are returned. No mask will have any bits set to 1 in common with any other mask or with any of the pixels. By ORing together each pixel with zero or more masks, $npixels \times 2^{nplanes}$ distinct pixels can be produced. All of these are allocated writable by the request.

For **GrayScale** or **PseudoColor**, each mask has exactly one bit set to 1. For **DirectColor**, each has exactly three bits set to 1. If *contig* is **True** and if all masks are ORed together, a single contiguous set of bits set to 1 will be formed for **GrayScale** or **PseudoColor**, and three contiguous sets of bits set to 1, one within each pixel subfield, for **DirectColor**. The RGB values of the allocated entries are undefined. **XAllocColorCells** returns nonzero if it succeeded or zero if it failed.

XAllocColorCells can generate **BadColor** and **BadValue** errors.

To allocate read/write color resources for a **DirectColor** model, use **XAllocColorPlanes**.

The server guarantees that the returned masks do not overlap with each other or with the base pixels. That guarantee matters because otherwise ORing the masks together could accidentally collide with other allocated pixel values.

```
Status XAllocColorPlanes(
    Display *display,
    Colormap colormap,
    Bool contig,
    unsigned long pixels_return[],
    int ncolors,
    int nreds,
    int ngreens,
    int nblues,
    unsigned long *rmask_return,
    unsigned long *gmask_return,
    unsigned long *bmask_return
)
```

– *display*: Specifies the connection to the X server.
– *colormap*: Specifies the colormap.

- *contig*: Specifies a Boolean value that indicates whether the planes must be contiguous.
- *pixels_return*: Returns an array of pixel values. **XAllocColorPlanes** returns the pixel values in this array.
- *ncolors*: Specifies the number of pixel values that are to be returned in the *pixels_return* array.
- *nreds*: Specifies the number of red planes. The value you pass must be nonnegative.
- *ngreens*: Specifies the number of green planes. The value you pass must be nonnegative.
- *nblues*: Specifies the number of blue planes. The value you pass must be nonnegative.
- *rmask_return*: Returns the bit mask for the red planes.
- *gmask_return*: Returns the bit mask for the green planes.
- *bmask_return*: Returns the bit mask for the blue planes.

The specified *ncolors* must be positive, and *nreds*, *ngreens*, and *nblues* must be nonnegative, or a **BadValue** error results. If *ncolors* colors, *nreds* reds, *ngreens* greens, and *nblues* blues are requested, *ncolors* pixels are returned, and the masks have *nreds*, *ngreens*, and *nblues* bits set to 1, respectively. If *contig* is **True**, each mask will have a contiguous set of bits set to 1. No mask will have any bits set to 1 in common with any other mask or with any of the pixels.

For **DirectColor**, each mask will lie within the corresponding pixel subfield. By ORing together subsets of masks with each pixel value, $ncolors \times 2^{(nreds+ngreens+nblues)}$ distinct pixel values can be produced. All of these are allocated by the request.

However, in the colormap, there are only $ncolors \times 2^{nreds}$ independent red entries, $ncolors \times 2^{ngreens}$ independent green entries, and $ncolors \times 2^{nblues}$ independent blue entries. This is true even for **PseudoColor**. When the colormap entry of a pixel value is changed using **XStoreColors**, **XStoreColor**, or **XStoreNamedColor**, the pixel is decomposed according to the masks, and the corresponding independent entries are updated. **XAllocColorPlanes** returns nonzero if it succeeded or zero if it failed.

XAllocColorPlanes can generate **BadColor** and **BadValue** errors.

Note: The contig argument controls the shape of those masks. If contig is False, the server may return masks using any non-overlapping bit positions.

```
mask0 = 0b00000001
mask1 = 0b00001000
mask2 = 0b00100000
```

These masks are valid because they do not overlap, but the set of mask bits is not contiguous.

If contig is True, the server must return masks whose set bits form a contiguous range.

```
mask0 = 0b00000100
mask1 = 0b00001000
mask2 = 0b00010000
```

Together, these occupy adjacent bit positions:

```
0b00011100
```

For DirectColor, each mask affects the red, green, and blue subfields separately. So contig = True means there is a contiguous set of bits within each color subfield.

To free colormap cells, use **XFreeColors**.

```
int XFreeColors(
    Display *display,
    Colormap colormap,
    unsigned long pixels[],
    int npixels,
    unsigned long planes
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *pixels*: Specifies an array of pixel values that map to the cells in the specified colormap.
- *npixels*: Specifies the number of pixels.
- *planes*: Specifies the planes you want to free.

The **XFreeColors** function frees the cells represented by pixels whose values are in the *pixels* array. The *planes* argument should not have any bits set to 1 in common with any of the pixels. The set of all pixels is produced by ORing together subsets of the *planes* argument with the pixels.

The request frees all of these pixels that were allocated by the client using **XAllocColor**, **XAllocNamedColor**, **XAllocColorCells**, and **XAllocColorPlanes**. Note that freeing an individual pixel obtained from **XAllocColorPlanes** may not actually allow it to be reused until all of its related pixels are also freed. Similarly, a read-only entry is not actually freed until it has been freed by all clients, and if a client allocates the same read-only entry multiple times, it must free the entry that many times before the entry is actually freed.

All specified pixels that are allocated by the client in the colormap are freed, even if one or more pixels produce an error. If a specified pixel is not a valid index into the colormap, a **BadValue** error results. If a specified pixel is not allocated by the client, that is, if it is unallocated or is only allocated by another client, or if the colormap was created with all entries writable by passing **AllocAll** to **XCreateColormap**, a **BadAccess** error results. If more than one pixel is in error, the one that gets reported is arbitrary.

XFreeColors can generate **BadAccess**, **BadColor**, and **BadValue** errors.

- **Modifying and Querying Colormap Cells**: To store an RGB value in a single colormap cell, use **XStoreColor**.

```
int XStoreColor(
    Display *display,
    Colormap colormap,
    XColor *color
)
```

  - *display*: Specifies the connection to the X server.
  - *colormap*: Specifies the colormap.
  - *color*: Specifies the pixel and RGB values.

The **XStoreColor** function changes the colormap entry of the pixel value specified in the *pixel* member of the **XColor** structure. This pixel value must be a read/write cell and a valid index into the colormap. If a specified pixel is not a valid index into the colormap, a **BadValue** error results.

XStoreColor also changes the red, green, and/or blue color components. You specify which color components are to be changed by setting **DoRed**, **DoGreen**, and/or **DoBlue** in the *flags* member of the **XColor** structure. If the colormap is an installed map for its screen, the changes are visible immediately.

XStoreColor can generate **BadAccess**, **BadColor**, and **BadValue** errors.

To store multiple RGB values in multiple colormap cells, use **XStoreColors**.

```
int XStoreColors(
    Display *display,
    Colormap colormap,
    XColor color[],
    int ncolors
)
```

  - *display*: Specifies the connection to the X server.
  - *colormap*: Specifies the colormap.
  - *color*: Specifies an array of color definition structures to be stored.
  - *ncolors*: Specifies the number of **XColor** structures in the color definition array.

The **XStoreColors** function changes the colormap entries of the pixel values specified in the *pixel* members of the **XColor** structures. You specify which color components are to be changed by setting **DoRed**, **DoGreen**, and/or **DoBlue** in the *flags* member of the **XColor** structures. If the colormap is an installed map for its screen, the changes are visible immediately.

XStoreColors changes the specified pixels if they are allocated writable in the colormap by any client, even if one or more pixels generates an error. If a specified pixel is not a valid index into the colormap, a **BadValue** error results. If a specified pixel either is unallocated or is allocated read-only, a **BadAccess** error results. If more than one pixel is in error, the one that gets reported is arbitrary.

XStoreColors can generate **BadAccess**, **BadColor**, and **BadValue** errors.

To store a color of arbitrary format in a single colormap cell, use **XcmsStoreColor**.

```
Status XcmsStoreColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies the color cell and the color to store. Values specified in this **XcmsColor** structure remain unchanged on return.

The **XcmsStoreColor** function converts the color specified in the **XcmsColor** structure into RGB values. It then uses this RGB specification in an **XColor** structure, whose three flags, **DoRed**, **DoGreen**, and **DoBlue**, are set, in a call to **XStoreColor** to change the color cell specified by the *pixel* member of the **XcmsColor** structure.

This pixel value must be a valid index for the specified colormap, and the color cell specified by the pixel value must be a read/write cell. If the pixel value is not a valid index, a **BadValue** error results. If the color cell is unallocated or is allocated read-only, a **BadAccess** error results. If the colormap is an installed map for its screen, the changes are visible immediately.

Note that **XStoreColor** has no return value; therefore, an **XcmsSuccess** return value from this function indicates that the conversion to RGB succeeded and the call to **XStoreColor** was made. To obtain the actual color stored, use **XcmsQueryColor**. Because of the screen's hardware limitations or gamut compression, the color stored in the colormap may not be identical to the color specified.

XcmsStoreColor can generate **BadAccess**, **BadColor**, and **BadValue** errors.

To store multiple colors of arbitrary format in multiple colormap cells, use **XcmsStoreColors**.

```
Status XcmsStoreColors(
    Display *display,
    Colormap colormap,
    XcmsColor colors[],
    int ncolors,
    Bool compression_flags_return[]
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *colors*: Specifies the color specification array of **XcmsColor** structures, each specifying a color cell and the color to store in that cell. Values specified in the array remain unchanged upon return.
- *ncolors*: Specifies the number of **XcmsColor** structures in the color-specification array.
- *compression_flags_return*: Returns an array of Boolean values indicating compression status. If a non-**NULL** pointer is supplied, each element of the array is set to **True** if the corresponding color was compressed and **False** otherwise. Pass **NULL** if the compression status is not useful.

The **XcmsStoreColors** function converts the colors specified in the array of **XcmsColor** structures into RGB values and then uses these RGB specifications in **XColor** structures, whose three flags, **DoRed**, **DoGreen**, and **DoBlue**, are set, in a call to **XStoreColors** to change the color cells specified by the *pixel* member of the corresponding **XcmsColor** structure.

Each pixel value must be a valid index for the specified colormap, and the color cell specified by each pixel value must be a read/write cell. If a pixel value is not a valid index, a **BadValue** error results. If a color cell is unallocated or is allocated read-only, a **BadAccess** error results. If more than one pixel is in error, the one that gets reported is arbitrary. If the colormap is an installed map for its screen, the changes are visible immediately.

Note that **XStoreColors** has no return value; therefore, an **XcmsSuccess** return value from this function indicates that conversions to RGB succeeded and the call to **XStoreColors** was made. To obtain the actual colors stored, use **XcmsQueryColors**. Because of the screen's hardware limitations or gamut compression, the colors stored in the colormap may not be identical to the colors specified.

XcmsStoreColors can generate **BadAccess**, **BadColor**, and **BadValue** errors.

To store a color specified by name in a single colormap cell, use **XStoreNamedColor**.

```
int XStoreNamedColor(
    Display *display,
    Colormap colormap,
    char *color,
    unsigned long pixel,
    int flags
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies the color name string, for example, `red`.
- *pixel*: Specifies the entry in the colormap.
- *flags*: Specifies which red, green, and blue components are set.

The **XStoreNamedColor** function looks up the named color with respect to the screen associated with the colormap and stores the result in the specified colormap. The *pixel* argument determines the entry in the colormap. The *flags* argument determines which of the red, green, and blue components are set. You can set this member to the bitwise inclusive OR of the bits **DoRed**, **DoGreen**, and **DoBlue**.

If the color name is not in the Host Portable Character Encoding, the result is implementation-dependent. Use of uppercase or lowercase does not matter. If the specified pixel is not a valid index into the colormap, a **BadValue** error results. If the specified pixel either is unallocated or is allocated read-only, a **BadAccess** error results.

XStoreNamedColor can generate **BadAccess**, **BadColor**, **BadName**, and **BadValue** errors.

The **XQueryColor** and **XQueryColors** functions take pixel values in the *pixel* member of **XColor** structures and store in the structures the RGB values for those pixels from the specified colormap. The values returned for an unallocated entry are undefined. These functions also set the *flags* member in the **XColor** structure to all three colors. If a pixel is not a valid index into the specified colormap, a **BadValue** error results. If more than one pixel is in error, the one that gets reported is arbitrary.

To query the RGB value of a single colormap cell, use **XQueryColor**.

```
int XQueryColor(
    Display *display,
    Colormap colormap,
    XColor *def_in_out
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *def_in_out*: Specifies and returns the RGB values for the pixel specified in the structure.

The **XQueryColor** function returns the current RGB value for the pixel in the **XColor** structure and sets the **DoRed**, **DoGreen**, and **DoBlue** flags.

XQueryColor can generate **BadColor** and **BadValue** errors.

To query the RGB values of multiple colormap cells, use **XQueryColors**.

```
int XQueryColors(
    Display *display,
    Colormap colormap,
    XColor defs_in_out[],
    int ncolors
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *defs_in_out*: Specifies and returns an array of color definition structures for the pixels specified in the structures.
- *ncolors*: Specifies the number of **XColor** structures in the color definition array.

The **XQueryColors** function returns the RGB value for each pixel in each **XColor** structure and sets the **DoRed**, **DoGreen**, and **DoBlue** flags in each structure.

XQueryColors can generate **BadColor** and **BadValue** errors.

To query the color of a single colormap cell in an arbitrary format, use **XcmsQueryColor**.

```
Status XcmsQueryColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color_in_out,
    XcmsColorFormat result_format
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_in_out*: Specifies the *pixel* member that indicates the color cell to query. The color specification stored for the color cell is returned in this **XcmsColor** structure.
- *result_format*: Specifies the color format for the returned color specification.

The **XcmsQueryColor** function obtains the RGB value for the pixel value in the *pixel* member of the specified **XcmsColor** structure and then converts the value to the target format as specified by the *result_format* argument. If the pixel is not a valid index in the specified colormap, a **BadValue** error results.

XcmsQueryColor can generate **BadColor** and **BadValue** errors.

To query the color of multiple colormap cells in an arbitrary format, use **XcmsQueryColors**.

```
Status XcmsQueryColors(
    Display *display,
    Colormap colormap,
    XcmsColor colors_in_out[],
    unsigned int ncolors,
    XcmsColorFormat result_format
)
```

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.

- *colors_in_out*: Specifies an array of **XcmsColor** structures, each *pixel* member indicating the color cell to query. The color specifications for the color cells are returned in these structures.
- *ncolors*: Specifies the number of **XcmsColor** structures in the color-specification array.
- *result_format*: Specifies the color format for the returned color specification.

The **XcmsQueryColors** function obtains the RGB values for pixel values in the *pixel* members of **XcmsColor** structures and then converts the values to the target format as specified by the *result_format* argument.

If a pixel is not a valid index into the specified colormap, a **BadValue** error results. If more than one pixel is in error, the one that gets reported is arbitrary.

XcmsQueryColors can generate **BadColor** and **BadValue** errors.

7. Graphics context functions

- **Intro**: A number of resources are used when performing graphics operations in X. Most information about performing graphics (for example, foreground color, background color, line style, and so on) is stored in resources called graphics contexts (GCs). Most graphics operations (see chapter 8) take a GC as an argument. Although in theory the X protocol permits sharing of GCs between applications, it is expected that applications will use their own GCs when performing operations. Sharing of GCs is highly discouraged because the library may cache GC state

  Graphics operations can be performed to either windows or pixmaps, which collectively are called drawables. Each drawable exists on a single screen. A GC is created for a specific screen and drawable depth and can only be used with drawables of matching screen and depth.

- **Write-back cache**: A write-back cache is a caching technique where the system writes incoming data directly to the fast cache and instantly confirms the write is complete, deferring the update to the slower primary storage (like a hard drive or database) until a later time

- **Manipulating Graphics Context/State**: Most attributes of graphics operations are stored in GCs. These include line width, line style, plane mask, foreground, background, tile, stipple, clipping region, end style, join style, and so on. Graphics operations (for example, drawing lines) use these values to determine the actual drawing operation.

  Xlib implements a write-back cache for all elements of a GC that are not resource IDs to allow Xlib to implement the transparent coalescing of changes to GCs. For example, a call to **XSetForeground** of a GC followed by a call to **XSetLineAttributes** results in only a single-change GC protocol request to the server. GCs are neither expected nor encouraged to be shared between client applications, so this write-back caching should present no problems. Applications cannot share GCs without external synchronization. Therefore, sharing GCs between applications is highly discouraged.

  To set an attribute of a GC, set the appropriate member of the **XGCValues** structure and OR in the corresponding value bitmask in your subsequent calls to **XCreateGC.** The symbols for the value mask bits and the **XGCValues** structure are:

```
/* GC attribute value mask bits */
#define GCFunction (1L<<0)
#define GCPlaneMask (1L<<1)
#define GCForeground (1L<<2)
#define GCBackground (1L<<3)
#define GCLineWidth (1L<<4)
#define GCLineStyle (1L<<5)
#define GCCapStyle (1L<<6)
#define GCJoinStyle (1L<<7)
#define GCFillStyle (1L<<8)
#define GCFillRule (1L<<9)
#define GCTile (1L<<10)
#define GCStipple (1L<<11)
#define GCTileStipXOrigin (1L<<12)
#define GCTileStipYOrigin (1L<<13)
#define GCFont (1L<<14)
#define GCSubwindowMode (1L<<15)
#define GCGraphicsExposures (1L<<16)
#define GCClipXOrigin (1L<<17)
#define GCClipYOrigin (1L<<18)
#define GCClipMask (1L<<19)
#define GCDashOffset (1L<<20)
#define GCDashList (1L<<21)
#define GCArcMode (1L<<22)
```

```
/* Values */
typedef struct {
    int function;                   /* logical operation */
    unsigned long plane_mask;       /* plane mask */
    unsigned long foreground;       /* foreground pixel */
    unsigned long background;       /* background pixel */
    int line_width;                 /* line width (in pixels)
    ↪ */
    int line_style;                 /* LineSolid,
    ↪ LineOnOffDash, LineDoubleDash */
    int cap_style;                  /* CapNotLast, CapButt,
    ↪ CapRound, CapProjecting */
    int join_style;                 /* JoinMiter, JoinRound,
    ↪ JoinBevel */
    int fill_style;                 /* FillSolid, FillTiled,
    ↪ FillStippled FillOpaqueStippled*/
    int fill_rule;                  /* EvenOddRule,
    ↪ WindingRule */
    int arc_mode;                   /* ArcChord, ArcPieSlice
    ↪ */
    Pixmap tile;                    /* tile pixmap for tiling
    ↪ operations */
    Pixmap stipple;                 /* stipple 1 plane pixmap
    ↪ for stippling */
    int ts_x_origin;                /* offset for tile or
    ↪ stipple operations */
    int ts_y_origin;
    Font font;                      /* default text font for
    ↪ text operations */
    int subwindow_mode;             /* ClipByChildren,
    ↪ IncludeInferiors */
    Bool graphics_exposures;        /* boolean, should
    ↪ exposures be generated */
    int clip_x_origin;              /* origin for clipping */
    int clip_y_origin;
    Pixmap clip_mask;               /* bitmap clipping; other
    ↪ calls for rects */
    int dash_offset;                /* patterned/dashed line
    ↪ information */
    char dashes;
} XGCValues;
```

8. Graphics functions

9. Window and session manager functions

10. Events

- **Events**: In X11, events are messages from the X server to clients.

  Your program is an X client. The X server owns the display, windows, keyboard, mouse, screen resources, and window tree. When something happens, the server may place an event into your client's event queue.

  - A key was pressed.
  - The pointer moved.
  - A window was created, mapped, unmapped, destroyed, resized, or moved.
  - A window needs repainting.
  - A client changed a property.
  - A client asked the window manager for something.
  - The input focus changed.
  - The pointer entered or left a window

- **Events are not automatically delivered to everyone**: The X server does not send every event to every client. That would be wasteful and ambiguous. Instead, each client declares interest in certain event categories on certain windows. This is done through event masks. An event mask says "For this window, send me these kinds of events."

- **Event types**: An event is data generated asynchronously by the X server as a result of some device activity or as side effects of a request sent by an Xlib function. Device-related events propagate from the source window to ancestor windows until some client application has selected that event type or until the event is explicitly discarded. The X server generally sends an event to a client application only if the client has specifically asked to be informed of that event type, typically by setting the event-mask attribute of the window. The mask can also be set when you create a window or by changing the window's event-mask. You can also mask out events that would propagate to ancestor windows by manipulating the do-not-propagate mask of the window's attributes. However, **MappingNotify** events are always sent to all clients.

  n event type describes a specific event generated by the X server. For each event type, a corresponding constant name is defined in <X11/X.h>, which is used when referring to an event type. The following table lists the event category and its associated event type or types.

Event Category	Event Type
Keyboard events	KeyPress, KeyRelease
Pointer events	ButtonPress, ButtonRelease, MotionNotify
Window crossing events	EnterNotify, LeaveNotify
Input focus events	FocusIn, FocusOut
Keymap state notification event	KeymapNotify
Exposure events	Expose, GraphicsExpose, NoExpose
Structure control events	CirculateRequest, ConfigureRequest, MapRequest, ResizeRequest
Window state notification events	CirculateNotify, ConfigureNotify, CreateNotify, DestroyNotify, GravityNotify, MapNotify, MappingNotify, ReparentNotify, UnmapNotify, VisibilityNotify
Colormap state notification event	ColormapNotify
Client communication events	ClientMessage, PropertyNotify, SelectionClear, SelectionNotify, SelectionRequest

- **Event structures**: For each event type, a corresponding structure is declared in <X11/Xlib.h>. All the event structures have the following common members:

```
typedef struct {
    int type;
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;
} XAnyEvent;
```

The type member is set to the event type constant name that uniquely identifies it. For example, when the X server reports a GraphicsExpose event to a client application, it sends an XGraphicsExposeEvent structure with the type member set to GraphicsExpose. The display member is set to a pointer to the display the event was read on. The send_event member is set to True if the event came from a SendEvent protocol request. The serial member is set from the serial number reported in the protocol but expanded from the 16-bit least-significant bits to a full 32-bit value. The window member is set to the window that is most useful to toolkit dispatchers.

The X server can send events at any time in the input stream. Xlib stores any events received while waiting for a reply in an event queue for later use. Xlib also provides functions that allow you to check events in the event queue

In addition to the individual structures declared for each event type, the XEvent structure is a union of the individual structures declared for each event type. Depending on the type, you should access members of each event by using the XEvent union.

```
typedef union _XEvent {
    int type;           /* must not be changed */
    XAnyEvent xany;
    XKeyEvent xkey;
    XButtonEvent xbutton;
    XMotionEvent xmotion;
    XCrossingEvent xcrossing;
    XFocusChangeEvent xfocus;
    XExposeEvent xexpose;
    XGraphicsExposeEvent xgraphicsexpose;
    XNoExposeEvent xnoexpose;
    XVisibilityEvent xvisibility;
    XCreateWindowEvent xcreatewindow;
    XDestroyWindowEvent xdestroywindow;
    XUnmapEvent xunmap;
    XMapEvent xmap;
    XMapRequestEvent xmaprequest;
    XReparentEvent xreparent;
    XConfigureEvent xconfigure;
    XGravityEvent xgravity;
    XResizeRequestEvent xresizerequest;
    XConfigureRequestEvent xconfigurerequest;
    XCirculateEvent xcirculate;
    XCirculateRequestEvent xcirculaterequest;
    XPropertyEvent xproperty;
    XSelectionClearEvent xselectionclear;
    XSelectionRequestEvent xselectionrequest;
    XSelectionEvent xselection;
    XColormapEvent xcolormap;
    XClientMessageEvent xclient;
    XMappingEvent xmapping;
    XErrorEvent xerror;
    XKeymapEvent xkeymap;
    long pad[24];
} XEvent;
```

An XEvent structure's first entry always is the type member, which is set to the event type. The second member always is the serial number of the protocol request that generated the event. The third member always is send_event, which is a Bool that indicates if the event was sent by a different client. The fourth member always is a display, which is the display that the event was read from. Except for keymap events, the fifth member always is a window, which has been carefully selected to be useful to toolkit dispatchers. To avoid breaking toolkits, the order of these first five entries is not to change. Most events also contain a time member, which is the time at which an event occurred. In addition, a pointer to the generic event must be cast before it is used to access any other information in the structure.

- **Event Masks**: Clients select event reporting of most events relative to a window. To do this, pass an event mask to an Xlib event-handling function that takes an event_mask argument. The bits of the event mask are defined in <X11/X.h>. Each bit in the event mask maps to an event mask name, which describes the event or events you want the X server to return to a client application.

  Unless the client has specifically asked for them, most events are not reported to

clients when they are generated. Unless the client suppresses them by setting graphics-exposures in the GC to False, GraphicsExpose and NoExpose are reported by default as a result of XCopyPlane and XCopyArea. SelectionClear, SelectionRequest, SelectionNotify, or ClientMessage cannot be masked. Selection-related events are only sent to clients cooperating with selections. When the keyboard or pointer mapping is changed, MappingNotify is always sent to clients.

Event Mask	**Circumstances**
NoEventMask	No events wanted
KeyPressMask	Keyboard down events wanted
KeyReleaseMask	Keyboard up events wanted
ButtonPressMask	Pointer button down events wanted
ButtonReleaseMask	Pointer button up events wanted
EnterWindowMask	Pointer window entry events wanted
LeaveWindowMask	Pointer window leave events wanted
PointerMotionMask	Pointer motion events wanted
PointerMotionHintMask	Pointer motion hints wanted
Button1MotionMask	Pointer motion while button 1 down
Button2MotionMask	Pointer motion while button 2 down
Button3MotionMask	Pointer motion while button 3 down
Button4MotionMask	Pointer motion while button 4 down
Button5MotionMask	Pointer motion while button 5 down
ButtonMotionMask	Pointer motion while any button down
KeymapStateMask	Keyboard state wanted at window entry and focus in
ExposureMask	Any exposure wanted
VisibilityChangeMask	Any change in visibility wanted
StructureNotifyMask	Any change in window structure wanted
ResizeRedirectMask	Redirect resize of this window
SubstructureNotifyMask	Substructure notification wanted
SubstructureRedirectMask	Redirect structure requests on children
FocusChangeMask	Any change in input focus wanted
PropertyChangeMask	Any change in property wanted
ColormapChangeMask	Any change in colormap wanted
OwnerGrabButtonMask	Automatic grabs should activate with owner_events set to True

- **Event processing overview**: The event reported to a client application during event processing depends on which event masks you provide as the event-mask attribute for a window. For some event masks, there is a one-to- one correspondence between the event mask constant and the event type constant. For example, if you pass the event mask ButtonPressMask, the X server sends back only ButtonPress events. Most events contain a time member, which is the time at which an event occurred.

  In other cases, one event mask constant can map to several event type constants. For example, if you pass the event mask SubstructureNotifyMask, the X server can send back CirculateNotify, ConfigureNotify, CreateNotify, DestroyNotify, GravityNotify, MapNotify, ReparentNotify, or UnmapNotify events.

  In another case, two event masks can map to one event type. For example, if you pass either PointerMotionMask or ButtonMotionMask, the X server sends back a MotionNotify event.

  The following table lists the event mask, its associated event type or types, and the structure name associated with the event type. Some of these structures actually are typedefs to a generic structure that is shared between two event types. Note that N.A. appears in columns for which the information is not applicable

Event Mask	**Event Type**	**Structure**	**Generic Structure**
ButtonMotionMask	MotionNotify	XPointerMovedEvent	XMotionEvent
Button1MotionMask			
Button2MotionMask			
Button3MotionMask			
Button4MotionMask			
Button5MotionMask			
ButtonPressMask	ButtonPress	XButtonPressedEvent	XButtonEvent
ButtonReleaseMask	ButtonRelease	XButtonReleasedEvent	XButtonEvent
ColormapChangeMask	ColormapNotify	XColormapEvent	
EnterWindowMask	EnterNotify	XEnterWindowEvent	XCrossingEvent
LeaveWindowMask	LeaveNotify	XLeaveWindowEvent	XCrossingEvent
ExposureMask	Expose	XExposeEvent	
GCGraphicsExposures in GC	GraphicsExpose	XGraphicsExposeEvent	
	NoExpose	XNoExposeEvent	

Event Mask	Event Type	Structure	Generic Structure
FocusChangeMask	FocusIn	XFocusInEvent	XFocusChangeEvent
	FocusOut	XFocusOutEvent	XFocusChangeEvent
KeymapStateMask	KeymapNotify	XKeymapEvent	
KeyPressMask	KeyPress	XKeyPressedEvent	XKeyEvent
KeyReleaseMask	KeyRelease	XKeyReleasedEvent	XKeyEvent
OwnerGrabButton-Mask	N.A.	N.A.	
PointerMotionMask	MotionNotify	XPointerMovedEvent	XMotionEvent
PointerMotionHint-Mask	N.A.	N.A.	
PropertyChangeMask	PropertyNotify	XPropertyEvent	
ResizeRedirectMask	ResizeRequest	XResizeRequestEvent	
StructureNotifyMask	CirculateNotify	XCirculateEvent	
	ConfigureNotify	XConfigureEvent	
	DestroyNotify	XDestroyWindow-Event	
	GravityNotify	XGravityEvent	
	MapNotify	XMapEvent	
	ReparentNotify	XReparentEvent	
	UnmapNotify	XUnmapEvent	
SubstructureNotify-Mask	CirculateNotify	XCirculateEvent	
	ConfigureNotify	XConfigureEvent	
	CreateNotify	XCreateWindowEvent	
	DestroyNotify	XDestroyWindow-Event	
	GravityNotify	XGravityEvent	
	MapNotify	XMapEvent	
	ReparentNotify	XReparentEvent	
	UnmapNotify	XUnmapEvent	

Event Mask	Event Type	Structure	Generic Structure
SubstructureRedirect-Mask	CirculateRequest	XCircu-lateRequestEvent	
	ConfigureRequest	XConfigur-eRequestEvent	
	MapRequest	XMapRequestEvent	
N.A.	ClientMessage	XClientMessageEvent	
N.A.	MappingNotify	XMappingEvent	
N.A.	SelectionClear	XSelectionClearEvent	
N.A.	SelectionNotify	XSelectionEvent	
N.A.	SelectionRequest	XSelectionRe-questEvent	
VisibilityChangeMask	VisibilityNotify	XVisibilityEvent	

The sections that follow describe the processing that occurs when you select the different event masks. The sections are organized according to these processing categories:

- Keyboard and pointer events
- Window crossing events
- Input focus events
- Keymap state notification events
- Exposure events
- Window state notification events
- Structure control events
- Colormap state notification events
- Client communication events

- **Pointer button events**: The following describes the event processing that occurs when a pointer button press is processed with the pointer in some window w and when no active pointer grab is in progress.

  The X server searches the ancestors of w from the root down, looking for a passive grab to activate. If no matching passive grab on the button exists, the X server automatically starts an active grab for the client receiving the event and sets the last-pointer-grab time to the current server time. The effect is essentially equivalent to an XGrabButton with these client passed arguments:

  - w The event window
  - event_mask The client's selected pointer events on the event window
  - pointer_mode GrabModeAsync
  - keyboard_mode GrabModeAsync
  - owner_events True, if the client has selected OwnerGrabButtonMask on the event window, otherwise False
  - confine_to None
  - cursor None

The active grab is automatically terminated when the logical state of the pointer has all buttons released. Clients can modify the active grab by calling XUngrabPointer and XChangeActivePointerGrab.

- **Keyboard and pointer events**: The X server reports KeyPress or KeyRelease events to clients wanting information about keys that logically change state. Note that these events are generated for all keys, even those mapped to modifier bits. The X server reports ButtonPress or ButtonRelease events to clients wanting information about buttons that logically change state.

  The X server reports MotionNotify events to clients wanting information about when the pointer logically moves. The X server generates this event whenever the pointer is moved and the pointer motion begins and ends in the window. The granularity of MotionNotify events is not guaranteed, but a client that selects this event type is guaranteed to receive at least one event when the pointer moves and then rests.

  The generation of the logical changes lags the physical changes if device event processing is frozen.

  To receive KeyPress, KeyRelease, ButtonPress, and ButtonRelease events, set KeyPressMask, KeyReleaseMask, ButtonPressMask, and ButtonReleaseMask bits in the event-mask attribute of the window.

  To receive MotionNotify events, set one or more of the following event masks bits in the eventmask attribute of the window

  - **Button1MotionMask — Button5MotionMask**: The client application receives MotionNotify events only when one or more of the specified buttons is pressed.
  - **ButtonMotionMask**: The client application receives MotionNotify events only when at least one button is pressed.
  - **PointerMotionMask**: The client application receives MotionNotify events independent of the state of the pointer buttons.
  - **PointerMotionHintMask**: If PointerMotionHintMask is selected in combination with one or more of the above masks, the X server is free to send only one MotionNotify event (with the is_hint member of the XPointerMovedEvent structure set to NotifyHint) to the client for the event window, until either the key or button state changes, the pointer leaves the event window, or the client calls XQueryPointer or XGetMotionEvents. The server still may send MotionNotify events without is_hint set to NotifyHint

  The source of the event is the viewable window that the pointer is in. The window used by the X server to report these events depends on the window's position in the window hierarchy and whether any intervening window prohibits the generation of these events. Starting with the source window, the X server searches up the window hierarchy until it locates the first window specified by a client as having an interest in these events. If one of the intervening windows has its do-not-propagate-mask set to prohibit generation of the event type, the events of those types will be suppressed. Clients can modify the actual window used for reporting by performing active grabs and, in the case of keyboard events, by using the focus window

  The structures for these event types contain:

```
typedef struct {
    int type;               /* ButtonPress or ButtonRelease
    ↪  */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window it is
    ↪  reported relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    unsigned int state;     /* key or button mask */
    unsigned int button;    /* detail */
    Bool same_screen;       /* same screen flag */
} XButtonEvent;
typedef XButtonEvent XButtonPressedEvent;
typedef XButtonEvent XButtonReleasedEvent;
```

```
typedef struct {
    int type;               /* KeyPress or KeyRelease */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window it is
    ↪  reported relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    unsigned int state;     /* key or button mask */
    unsigned int keycode;   /* detail */
    Bool same_screen;       /* same screen flag */
} XKeyEvent;
typedef XKeyEvent XKeyPressedEvent;
typedef XKeyEvent XKeyReleasedEvent;
```

```
typedef struct {
    int type;               /* MotionNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;          /* ‘‘event’’ window reported
    ↪ relative to */
    Window root;            /* root window that the event
    ↪ occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪ event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪ */
    unsigned int state;     /* key or button mask */
    char is_hint;           /* detail */
    Bool same_screen;       /* same screen flag */
} XMotionEvent;
typedef XMotionEvent XPointerMovedEvent;
```

These structures have the following common members: window, root, subwindow, time, x, y, x_root, y_root, state, and same_screen. The window member is set to the window on which the event was generated and is referred to as the event window. As long as the conditions previously discussed are met, this is the window used by the X server to report the event. The root member is set to the source window's root window. The x_root and y_root members are set to the pointer's coordinates relative to the root window's origin at the time of the event

The same_screen member is set to indicate whether the event window is on the same screen as the root window and can be either True or False. If True, the event and root windows are on the same screen. If False, the event and root windows are not on the same screen.

If the source window is an inferior of the event window, the subwindow member of the structure is set to the child of the event window that is the source window or the child of the event window that is an ancestor of the source window. Otherwise, the X server sets the subwindow member to None. The time member is set to the time when the event was generated and is expressed in milliseconds.

Subwindow is not trying to store "the deepest real source window." It stores the immediate child of the event window on the path toward the source window. So if the source window is an inferior of the event window, the source is somewhere below it in the window tree.

- subwindow = the immediate child of the event window containing the event/source

It identifies which branch below the event window the event came from.

If the event window is on the same screen as the root window, the x and y members are set to the coordinates relative to the event window's origin. Otherwise, these members are set to zero.

The state member is set to indicate the logical state of the pointer buttons and modifier keys just prior to the event, which is the bitwise inclusive OR of one or more of the button or modifier key masks: Button1Mask, Button2Mask, Button3Mask, Button4Mask, Button5Mask, ShiftMask, LockMask, ControlMask, Mod1Mask, Mod2Mask, Mod3Mask, Mod4Mask, and Mod5Mask

Each of these structures also has a member that indicates the detail. For the XKeyPressedEvent and XKeyReleasedEvent structures, this member is called a keycode. It is set to a number that represents a physical key on the keyboard. The keycode is an arbitrary representation for any key on the keyboard

For the XButtonPressedEvent and XButtonReleasedEvent structures, this member is called button. It represents the pointer button that changed state and can be the Button1, Button2, Button3, Button4, or Button5 value. For the XPointerMovedEvent structure, this member is called is_hint. It can be set to NotifyNormal or NotifyHint

Some of the symbols mentioned in this section have fixed values, as follows:

Symbol	Value
Button1MotionMask	(1L«8)
Button2MotionMask	(1L«9)
Button3MotionMask	(1L«10)
Button4MotionMask	(1L«11)
Button5MotionMask	(1L«12)
Button1Mask	(1«8)
Button2Mask	(1«9)
Button3Mask	(1«10)
Button4Mask	(1«11)
Button5Mask	(1«12)
ShiftMask	(1«0)
LockMask	(1«1)
ControlMaUsk	(1«2)
Mod1Mask U	(1«3)
Mod2Mask U	(1«4)
Mod3Mask U	(1«5)
Mod4Mask U	(1«6)
Mod5Mask U	(1«7)
Button1	1
Button2	2
Button3	3
Button4	4
Button5	5

  - source window = where the event logically originated
  - event window = the window to which this particular event was delivered

  Reminder: The do-not-propagate mask is a per-window attribute that says: "If this kind of input event starts here, and nobody selected it here, do not let it bubble upward to ancestor windows."

- **Window Entry/Exit Events**: This section describes the processing that occurs for the window crossing events EnterNotify and LeaveNotify. If a pointer motion or a window hierarchy change causes the pointer to be in a different window than before, the X server reports EnterNotify or LeaveNotify events to clients who have selected for these events. All EnterNotify and LeaveNotify events caused by a hierarchy change are generated after any hierarchy event (UnmapNotify, MapNotify, ConfigureNotify,

GravityNotify, CirculateNotify) caused by that change; however, the X protocol does not constrain the ordering of EnterNotify and LeaveNotify events with respect to FocusOut, VisibilityNotify, and Expose events.

This contrasts with MotionNotify events, which are also generated when the pointer moves but only when the pointer motion begins and ends in a single window. An EnterNotify or LeaveNotify event also can be generated when some client application calls XGrabPointer and XUngrabPointer.

To receive EnterNotify or LeaveNotify events, set the EnterWindowMask or LeaveWindowMask bits of the event-mask attribute of the window.

The structure for these event types contains:

```
typedef struct {
    int type;               /* EnterNotify or LeaveNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window reported
    ↪  relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    int mode;               /* NotifyNormal, NotifyGrab,
    ↪  NotifyUngrab */
    int detail;
                            /*
                            * NotifyAncestor, NotifyVirtual,
                            ↪  NotifyInferior,
                            * NotifyNonlinear,NotifyNonlinea⌋
                            ↪  rVirtual
                            */
    Bool same_screen;       /* same screen flag */
    Bool focus;             /* boolean focus */
    unsigned int state;     /* key or button mask */
} XCrossingEvent;
typedef XCrossingEvent XEnterWindowEvent;
typedef XCrossingEvent XLeaveWindowEvent;
```

The window member is set to the window on which the EnterNotify or LeaveNotify event was generated and is referred to as the event window. This is the window used by the X server to report the event, and is relative to the root window on which the event occurred. The root member is set to the root window of the screen on which the event occurred.

For a LeaveNotify event, if a child of the event window contains the initial position of the pointer, the subwindow component is set to that child. Otherwise, the X server sets the subwindow member to None. For an EnterNotify event, if a child of the event window contains the final pointer position, the subwindow component is set to that child or None

The time member is set to the time when the event was generated and is expressed in milliseconds. The and y members are set to the coordinates of the pointer position in the event window. This position is always the pointer's final position, not its initial position. If the event window is on the same screen as the root window, x and y are the pointer coordinates relative to the event window's origin. Otherwise, x and y are set to zero. The x_root and y_root members are set to the pointer's coordinates relative to the root window's origin at the time of the event.

The same_screen member is set to indicate whether the event window is on the same screen as the root window and can be either True or False. If True, the event and root windows are on the same screen. If False, the event and root windows are not on the same screen.

The focus member is set to indicate whether the event window is the focus window or an inferior of the focus window. The X server can set this member to either True or False. If True, the event window is the focus window or an inferior of the focus window. If False, the event window is not the focus window or an inferior of the focus window.

The state member is set to indicate the state of the pointer buttons and modifier keys just prior to the event. The X server can set this member to the bitwise inclusive OR of one or more of the button or modifier key masks: Button1Mask, Button2Mask, Button3Mask, Button4Mask, Button5Mask, ShiftMask, LockMask, ControlMask, Mod1Mask, Mod2Mask, Mod3Mask, Mod4Mask, Mod5Mask

The mode member is set to indicate whether the events are normal events, pseudo-motion events when a grab activates, or pseudo-motion events when a grab deactivates. The X server can set this member to NotifyNormal, NotifyGrab, or NotifyUngrab

The detail member is set to indicate the notify detail and can be NotifyAncestor, NotifyVirtual, NotifyInferior, NotifyNonlinear, or NotifyNonlinearVirtual.

  - **NotifyAncestor**: This means the crossing is between a window and one of its ancestors.
  - **NotifyInferior**: This means the crossing is between a window and one of its descendants.
  - **NotifyVirtual**: NotifyVirtual is used for intermediate windows between the old pointer window and the new pointer window when the movement is along a straight ancestor/descendant chain.
  - **NotifyNonlinear**: NotifyNonlinear means the old and new pointer windows are not ancestors of each other.
  - **NotifyNonlinearVirtual**: NotifyNonlinearVirtual is like NotifyVirtual, but for a nonlinear crossing.

- **Normal Entry/Exit Events**: EnterNotify and LeaveNotify events are generated when the pointer moves from one window to another window. Normal events are identified by XEnterWindowEvent or XLeaveWindowEvent structures whose mode member is set to NotifyNormal.

- When the pointer moves from window A to window B and A is an inferior of B, the X server does the following:
  - It generates a LeaveNotify event on window A, with the detail member of the XLeaveWindowEvent structure set to NotifyAncestor.
  - It generates a LeaveNotify event on each window between window A and window B, exclusive, with the detail member of each XLeaveWindowEvent structure set to NotifyVirtual.
  - It generates an EnterNotify event on window B, with the detail member of the XEnterWindowEvent structure set to NotifyInferior.
- When the pointer moves from window A to window B and B is an inferior of A, the X server does the following:
  - It generates a LeaveNotify event on window A, with the detail member of the XLeaveWindowEvent structure set to NotifyInferior.
  - It generates an EnterNotify event on each window between window A and window B, exclusive, with the detail member of each XEnterWindowEvent structure set to NotifyVirtual.
  - It generates an EnterNotify event on window B, with the detail member of the XEnterWindowEvent structure set to NotifyAncestor.
- When the pointer moves from window A to window B and window C is their least common ancestor, the X server does the following:
  - It generates a LeaveNotify event on window A, with the detail member of the XLeaveWindowEvent structure set to NotifyNonlinear.
  - It generates a LeaveNotify event on each window between window A and window C, exclusive, with the detail member of each XLeaveWindowEvent structure set to NotifyNonlinearVirtual.
  - It generates an EnterNotify event on each window between window C and window B, exclusive, with the detail member of each XEnterWindowEvent structure set to NotifyNonlinearVirtual.
  - It generates an EnterNotify event on window B, with the detail member of the XEnterWindowEvent structure set to NotifyNonlinear.
- When the pointer moves from window A to window B on different screens, the X server does the following:
  - It generates a LeaveNotify event on window A, with the detail member of the XLeaveWindowEvent structure set to NotifyNonlinear.
  - If window A is not a root window, it generates a LeaveNotify event on each window above window A up to and including its root, with the detail member of each XLeaveWindowEvent structure set to NotifyNonlinearVirtual.
  - If window B is not a root window, it generates an EnterNotify event on each window from window B's root down to but not including window B, with the detail member of each XEnterWindowEvent structure set to NotifyNonlinearVirtual.
  - It generates an EnterNotify event on window B, with the detail member of the XEnterWindowEvent structure set to NotifyNonlinear

- **Grab and Ungrab Entry/Exit Events**: Pseudo-motion mode EnterNotify and LeaveNotify events are generated when a pointer grab activates or deactivates. Events in which the pointer grab activates are identified by XEnterWindowEvent or XLeaveWindowEvent structures whose mode member is set to NotifyGrab. Events in which the pointer grab deactivates are identified by XEnterWindowEvent or XLeaveWindowEvent structures whose mode member is set to NotifyUngrab

- **Input Focus Events**: This section describes the processing that occurs for the input focus events FocusIn and FocusOut. The X server can report FocusIn or FocusOut events to clients wanting information about when the input focus changes. The keyboard is always attached to some window (typically, the root window or a top-level window), which is called the focus window. The focus window and the position of the pointer determine the window that receives keyboard input. Clients may need to know when the input focus changes to control highlighting of areas on the screen.

  To receive FocusIn or FocusOut events, set the FocusChangeMask bit in the event-mask attribute of the window.

  The structure for these event types contains:

```
typedef struct {
    int type;               /* FocusIn or FocusOut */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* window of event */
    int mode;               /* NotifyNormal, NotifyGrab,
    ↪  NotifyUngrab */
    int detail;
    /*
    * NotifyAncestor, NotifyVirtual, NotifyInferior,
    * NotifyNonlinear,NotifyNonlinearVirtual, NotifyPointer,
    * NotifyPointerRoot, NotifyDetailNone
    */
} XFocusChangeEvent;
typedef XFocusChangeEvent XFocusInEvent;
typedef XFocusChangeEvent XFocusOutEvent;
```

  The window member is set to the window on which the FocusIn or FocusOut event was generated. This is the window used by the X server to report the event. The mode member is set to indicate whether the focus events are normal focus events, focus events while grabbed, focus events when a grab activates, or focus events when a grab deactivates. The X server can set the mode member to NotifyNormal, NotifyWhileGrabbed, NotifyGrab, or NotifyUngrab.

  All FocusOut events caused by a window unmap are generated after any UnmapNotify event; however, the X protocol does not constrain the ordering of FocusOut events with respect to generated EnterNotify, LeaveNotify, VisibilityNotify, and Expose events.

  Depending on the event mode, the detail member is set to indicate the notify detail and can be NotifyAncestor, NotifyVirtual, NotifyInferior, NotifyNonlinear, NotifyNonlinearVirtual, NotifyPointer, NotifyPointerRoot, or NotifyDetailNone.

- **PointerRoot**: PointerRoot is a special value that can be used as the input focus.

If focus is PointerRoot, then keyboard focus follows the pointer / root context. PointerRoot is a special value that can be used as the input focus.

- **NotifyPointer**: Means This focus event is being generated for a window because of the pointer's position.
- **NotifyPointerRoot**: This focus event is related to the special PointerRoot focus state.
- **Normal Focus Events and Focus Events While Grabbed**: Normal focus events are identified by XFocusInEvent or XFocusOutEvent structures whose mode member is set to NotifyNormal. Focus events while grabbed are identified by XFocusInEvent or XFocusOutEvent structures whose mode member is set to NotifyWhileGrabbed
- **Focus Events Generated by Grabs**: Focus events in which the keyboard grab activates are identified by XFocusInEvent or XFocusOutEvent structures whose mode member is set to NotifyGrab. Focus events in which the keyboard grab deactivates are identified by XFocusInEvent or XFocusOutEvent structures whose mode member is set to NotifyUngrab (see XGrabKeyboard
- **Key Map State Notification Events**: The X server can report KeymapNotify events to clients that want information about changes in their keyboard state.

  To receive KeymapNotify events, set the KeymapStateMask bit in the event-mask attribute of the window. The X server generates this event immediately after every EnterNotify and FocusIn event.

  The structure for this event type contains:

```
0  /* generated on EnterWindow and FocusIn when KeymapState
   ↪  selected */
1  typedef struct {
2      int type;                   /* KeymapNotify */
3      unsigned long serial;       /* # of last request
       ↪  processed by server */
4      Bool send_event;            /* true if this came from a
       ↪  SendEvent request */
5      Display *display;           /* Display the event was read
       ↪  from */
6      Window window;
7      char key_vector[32];
8  } XKeymapEvent;
```

  The window member is not used but is present to aid some toolkits. The key_vector member is set to the bit vector of the keyboard. Each bit set to 1 indicates that the corresponding key is currently pressed. The vector is represented as 32 bytes. Byte N (from 0) contains the bits for keys $8N$ to $8N + 7$ with the least significant bit in the byte representing key 8N.

  Note: KeymapNotify is an X event that gives a snapshot of the current keyboard state.

  It does not mean "a key was pressed." It means: "here is the current keymap state right now." In practice, it tells you which physical keycodes are currently down.

The X server generates KeymapNotify immediately after an EnterNotify or FocusIn event when KeymapStateMask is selected. So it commonly appears when the pointer enters a window or when the keyboard focus moves to that window.

- **Exposure Events**: The X protocol does not guarantee to preserve the contents of window regions when the windows are obscured or reconfigured. Some implementations may preserve the contents of windows. Other implementations are free to destroy the contents of windows when exposed. X expects client applications to assume the responsibility for restoring the contents of an exposed window region. (An exposed window region describes a formerly obscured window whose region becomes visible.) Therefore, the X server sends Expose events describing the window and the region of the window that has been exposed. A naive client application usually redraws the entire window. A more sophisticated client application redraws only the exposed region.

- **Expose events**: The X server can report Expose events to clients wanting information about when the contents of window regions have been lost. The circumstances in which the X server generates Expose events are not as definite as those for other events. However, the X server never generates Expose events on windows whose class you specified as InputOnly. The X server can generate Expose events when no valid contents are available for regions of a window and either the regions are visible, the regions are viewable and the server is (perhaps newly) maintaining backing store on the window, or the window is not viewable but the server is (perhaps newly) honorin

  the window's backing-store attribute of Always or WhenMapped. The regions decompose into an (arbitrary) set of rectangles, and an Expose event is generated for each rectangle. For any given window, the X server guarantees to report contiguously all of the regions exposed by some action that causes Expose events, such as raising a window.

  To receive Expose events, set the ExposureMask bit in the event-mask attribute of the window.

  The structure for this event type contains:

```
typedef struct {
    int type;               /* Expose */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;
    int x, y;
    int width, height;
    int count;              /* if nonzero, at least this many
    ↪  more */
} XExposeEvent;
```

The window member is set to the exposed (damaged) window. The x and y members are set to the coordinates relative to the window's origin and indicate the upper-left corner of the rectangle. The width and height members are set to the size (extent) of the rectangle. The count member is set to the number of Expose events that are to follow. If count is zero, no more Expose events follow for this window. However, if count is nonzero, at least that number of Expose events (and possibly more) follow for this window. Simple applications that do not want to optimize redisplay by distinguishing between subareas of its window can just ignore all Expose events with nonzero counts and perform full redisplays on events with zero counts.

Note: Expose is mainly for windows or regions that come into visibility, not for regions that go out of visibility. Expose means: some part of this window is now visible, and its pixels may need to be redrawn.

- **GraphicsExpose and NoExpose Events**: The X server can report GraphicsExpose events to clients wanting information about when a destination region could not be computed during certain graphics requests: XCopyArea or XCopyPlane. The X server generates this event whenever a destination region could not be computed because of an obscured or out-of-bounds source region. In addition, the X server guarantees to report contiguously all of the regions exposed by some graphics request (for example, copying an area of a drawable to a destination drawable)

  The X server generates a NoExpose event whenever a graphics request that might produce a GraphicsExpose event does not produce any. In other words, the client is really asking for a GraphicsExpose event but instead receives a NoExpose event.

  To receive GraphicsExpose or NoExpose events, you must first set the graphics-exposure attribute of the graphics context to True. You also can set the graphics-expose attribute when creating a graphics context using XCreateGC or by calling XSetGraphicsExposures

  The structures for these event types contain:

```
typedef struct {
    int type;               /* GraphicsExpose */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Drawable drawable;
    int x, y;
    int width, height;
    int count;              /* if nonzero, at least this many
    ↪ more */
    int major_code;         /* core is CopyArea or CopyPlane
    ↪ */
    int minor_code;         /* not defined in the core */
} XGraphicsExposeEvent;

typedef struct {
    int type;               /* NoExpose */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Drawable drawable;
    int major_code;         /* core is CopyArea or CopyPlane
    ↪ */
    int minor_code;         /* not defined in the core */
} XNoExposeEvent;
```

Both structures have these common members: drawable, major_code, and minor_code. The drawable member is set to the drawable of the destination region on which the graphics request was to be performed. The major_code member is set to the graphics request initiated by the client and can be either X_CopyArea or X_CopyPlane. If it is X_CopyArea, a call to XCopyArea initiated the request. If it is X_CopyPlane, a call to XCopyPlane initiated the request. These constants are defined in <X11/Xproto.h>. The minor_code member, like the major_code member, indicates which graphics request was initiated by the client. However, the minor_code member is not defined by the core X protocol and will be zero in these cases, although it may be used by an extension.

The XGraphicsExposeEvent structure has these additional members: x, y, width, height, and count. The x and y members are set to the coordinates relative to the drawable's origin and indicate the upper-left corner of the rectangle. The width and height members are set to the size (extent) of the rectangle. The count member is set to the number of GraphicsExpose events to follow. If count is zero, no more GraphicsExpose events follow for this window. However, if count is nonzero, at least that number of GraphicsExpose events (and possibly more) are to follow for this window.

- **Window State Change Events**: The following sections discuss:

- CirculateNotify events
- ConfigureNotify events
- CreateNotify events
- DestroyNotify events
- GravityNotify events
- MapNotify events
- MappingNotify events
- ReparentNotify events
- UnmapNotify events
- VisibilityNotify events

- **CirculateNotify Events**: The X server can report CirculateNotify events to clients wanting information about when a window changes its position in the stack. The X server generates this event type whenever a window is actually restacked as a result of a client application calling XCirculateSubwindows, XCirculateSubwindowsUp, or XCirculateSubwindowsDown

  To receive CirculateNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, circulating any child generates an event).

  The structure for this event type contains:

```
typedef struct {
    int type;               /* CirculateNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window event;
    Window window;
    int place;              /* PlaceOnTop, PlaceOnBottom */
} XCirculateEvent;
```

  The event member is set either to the restacked window or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the window that was restacked. The place member is set to the window's position after the restack occurs and is either PlaceOnTop or PlaceOnBottom. If it is PlaceOnTop, the window is now on top of all siblings. If it is PlaceOnBottom, the window is now below all siblings.

- **ConfigureNotify Events**: The X server can report ConfigureNotify events to clients wanting information about actual changes to a window's state, such as size, position, border, and stacking order. The X server generates this event type whenever one of the following configure window requests made by a client application actually completes:

  - A window's size, position, border, and/or stacking order is reconfigured by calling XConfigureWindow.

- The window's position in the stacking order is changed by calling XLowerWindow, XRaiseWindow, or XRestackWindows.
- A window is moved by calling XMoveWindow.
- A window's size is changed by calling XResizeWindow.
- A window's size and location is changed by calling XMoveResizeWindow.
- A window is mapped and its position in the stacking order is changed by calling XMapRaised.
- A window's border width is changed by calling XSetWindowBorderWidth.

To receive ConfigureNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, configuring any child generates an event).

The structure for this event type contains:

```
typedef struct {
    int type;               /* ConfigureNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window event;
    Window window;
    int x, y;
    int width, height;
    int border_width;
    Window above;
    Bool override_redirect;
} XConfigureEvent;
```

The event member is set either to the reconfigured window or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the window whose size, position, border, and/or stacking order was changed.

The x and y members are set to the coordinates relative to the parent window's origin and indicate the position of the upper-left outside corner of the window. The width and height members are set to the inside size of the window, not including the border. The border_width member is set to the width of the window's border, in pixels.

The above member is set to the sibling window and is used for stacking operations. If the X server sets this member to None, the window whose state was changed is on the bottom of the stack with respect to sibling windows. However, if this member is set to a sibling window, the window whose state was changed is placed on top of this sibling window.

The override_redirect member is set to the override-redirect attribute of the window. Window manager clients normally should ignore this window if the override_redirect member is True

- **CreateNotify Events**: The X server can report CreateNotify events to clients wanting information about creation of windows. The X server generates this event whenever a client application creates a window by calling XCreateWindow or XCreateSimpleWindow

  To receive CreateNotify events, set the SubstructureNotifyMask bit in the event-mask attribute of the window. Creating any children then generates an event

  The structure for the event type contains:

```
typedef struct {
    int type;                   /* CreateNotify */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window parent;              /* parent of the window */
    Window window;              /* window id of window
    ↪ created */
    int x, y;                   /* window location */
    int width, height;          /* size of window */
    int border_width;           /* border width */
    Bool override_redirect;     /* creation should be
    ↪ overridden */
} XCreateWindowEvent;
```

  The parent member is set to the created window's parent. The window member specifies the created window. The x and y members are set to the created window's coordinates relative to the parent window's origin and indicate the position of the upper-left outside corner of the created window. The width and height members are set to the inside size of the created window (not including the border) and are always nonzero. The border_width member is set to the width of the created window's border, in pixels. The override_redirect member is set to the override-redirect attribute of the window. Window manager clients normally should ignore this window if the override_redirect member is True.

- **DestroyNotify Events**: The X server can report DestroyNotify events to clients wanting information about which windows are destroyed. The X server generates this event whenever a client application destroys a window by calling XDestroyWindow or XDestroySubwindows.

  The ordering of the DestroyNotify events is such that for any given window, DestroyNotify is generated on all inferiors of the window before being generated on the window itself. The X protocol does not constrain the ordering among siblings and across subhierarchies.

To receive DestroyNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, destroying any child generates an event).

The structure for this event type contains:

```
typedef struct {
    int type; /* DestroyNotify */
    unsigned long serial; /* # of last request processed by
    ↪ server */
    Bool send_event; /* true if this came from a SendEvent
    ↪ request */
    Display *display; /* Display the event was read from */
    Window event;
    Window window;
} XDestroyWindowEvent;
```

The event member is set either to the destroyed window or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the window that is destroyed.

- **GravityNotify Events**: The X server can report GravityNotify events to clients wanting information about when a window is moved because of a change in the size of its parent. The X server generates this event whenever a client application actually moves a child window as a result of resizing its parent by calling XConfigureWindow, XMoveResizeWindow, or XResizeWindow

To receive GravityNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, any child that is moved because its parent has been resized generates an event).

The structure for this event type contains:

```
typedef struct {
    int type;               /* GravityNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    int x, y;
} XGravityEvent;
```

The event member is set either to the window that was moved or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the child window that was moved. The x and y members are set to the coordinates relative to the new parent window's origin and indicate the position of the upper-left outside corner of the window.

- **MapNotify Events**: The X server can report MapNotify events to clients wanting information about which windows are mapped. The X server generates this event type whenever a client application changes the window's state from unmapped to mapped by calling XMapWindow, XMapRaised, XMapSubwindows, XReparentWindow, or as a result of save-set processing

  To receive MapNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, mapping any child generates an event).

  The structure for this event type contains:

```
typedef struct {
    int type;                   /* MapNotify */
    unsigned long serial;       /* # of last request
    ↪  processed by server */
    Bool send_event;            /* true if this came from a
    ↪  SendEvent request */
    Display *display;           /* Display the event was read
    ↪  from */
    Window event;
    Window window;
    Bool override_redirect;     /* boolean, is override
    ↪  set... */
} XMapEvent;
```

  The event member is set either to the window that was mapped or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the window that was mapped. The override_redirect member is set to the override-redirect attribute of the window. Window manager clients normally should ignore this window if the override-redirect attribute is True, because these events usually are generated from pop-ups, which override structure control

- **UnmapNotify Events**: The X server can report UnmapNotify events to clients wanting information about which windows are unmapped. The X server generates this event type whenever a client application changes the window's state from mapped to unmapped.

  To receive UnmapNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of the parent window (in which case, unmapping any child window generates an event).

  The structure for this event type contains:

```
typedef struct {
    int type;               /* UnmapNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    Bool from_configure;
} XUnmapEvent;
```

The event member is set either to the unmapped window or to its parent, depending on whether StructureNotify or SubstructureNotify was selected. This is the window used by the X server to report the event. The window member is set to the window that was unmapped. The from_configure member is set to True if the event was generated as a result of a resizing of the window's parent when the window itself had a win_gravity of UnmapGravity

- **MappingNotify Events**: The X server reports MappingNotify events to all clients. There is no mechanism to express dis- interest in this event. The X server generates this event type whenever a client application successfully calls:
  - XSetModifierMapping to indicate which KeyCodes are to be used as modifiers
  - XChangeKeyboardMapping to change the keyboard mapping
  - XSetPointerMapping to set the pointer mapping

  The structure for this event type contains:

```
typedef struct {
    int type;               /* MappingNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;          /* unused */
    int request;            /* one of MappingModifier,
    ↪ MappingKeyboard,
    MappingPointer */
    int first_keycode;      /* first keycode */
    int count;              /* defines range of change w.
    ↪ first_keycode*/
} XMappingEvent;
```

The request member is set to indicate the kind of mapping change that occurred and can be MappingModifier, MappingKeyboard, or MappingPointer. If it is Mapping-Modifier, the modifier mapping was changed. If it is MappingKeyboard, the keyboard mapping was changed. If it is MappingPointer, the pointer button mapping was

changed. The first_keycode and count members are set only if the request member was set to MappingKeyboard. The number in first_keycode represents the first number in the range of the altered mapping, and count represents the number of keycodes altered.

To update the client application's knowledge of the keyboard, you should call XRefreshKeyboardMapping

- **ReparentNotify Events**: The X server can report ReparentNotify events to clients wanting information about changing a window's parent. The X server generates this event whenever a client application calls XReparentWindow and the window is actually reparented.

  To receive ReparentNotify events, set the StructureNotifyMask bit in the event-mask attribute of the window or the SubstructureNotifyMask bit in the event-mask attribute of either the old or the new parent window (in which case, reparenting any child generates an event).

  The structure for this event type contains

```
typedef struct {
    int type;               /* ReparentNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window event;
    Window window;
    Window parent;
    int x, y;
    Bool override_redirect;
} XReparentEvent;
```

  The event member is set either to the reparented window or to the old or the new parent, depending on whether StructureNotify or SubstructureNotify was selected. The window member is set to the window that was reparented. The parent member is set to the new parent window. The x and y members are set to the reparented window's coordinates relative to the new parent window's origin and define the upper-left outer corner of the reparented window. The override_redirect member is set to the override-redirect attribute of the window specified by the window member. Window manager clients normally should ignore this window if the override_redirect member is True

- **VisibilityNotify Events**: The X server can report VisibilityNotify events to clients wanting any change in the visibility of the specified window. A region of a window is visible if someone looking at the screen can actually see it. The X server generates this event whenever the visibility changes state. However, this event is never generated for windows whose class is InputOnly.

All VisibilityNotify events caused by a hierarchy change are generated after any hierarchy event (UnmapNotify, MapNotify, ConfigureNotify, GravityNotify, CirculateNotify) caused by that change. Any VisibilityNotify event on a given window is generated before any Expose events on that window, but it is not required that all VisibilityNotify events on all windows be generated before all Expose events on all windows. The X protocol does not constrain the ordering of VisibilityNotify events with respect to FocusOut, EnterNotify, and LeaveNotify events.

To receive VisibilityNotify events, set the VisibilityChangeMask bit in the event-mask attribute of the window.

The structure for this event type contains

```
typedef struct {
    int type;               /* VisibilityNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;
    int state;
} XVisibilityEvent;
```

The window member is set to the window whose visibility state changes. The state member is set to the state of the window's visibility and can be VisibilityUnobscured, VisibilityPartiallyObscured, or VisibilityFullyObscured. The X server ignores all of a window's subwindows when determining the visibility state of the window and processes VisibilityNotify events according to the following:

- When the window changes state from partially obscured, fully obscured, or not viewable to viewable and completely unobscured, the X server generates the event with the state member of the XVisibilityEvent structure set to VisibilityUnobscured.
- When the window changes state from viewable and completely unobscured or not viewable to viewable and partially obscured, the X server generates the event with the state member of the XVisibilityEvent structure set to VisibilityPartiallyObscured.
- When the window changes state from viewable and completely unobscured, viewable and partially obscured, or not viewable to viewable and fully obscured, the X server generates the event with the state member of the XVisibilityEvent structure set to VisibilityFully- Obscured

- **CirculateRequest Events**: The X server can report CirculateRequest events to clients wanting information about when another client initiates a circulate window request on a specified window. The X server generates this event type whenever a client initiates a circulate window request on a window and a subwindow actually needs to be restacked. The client initiates a circulate window request on the window by calling XCirculateSubwindows, XCirculateSubwindowsUp, or XCirculateSubwindowsDown.

To receive CirculateRequest events, set the SubstructureRedirectMask in the event-mask attribute of the window. Then, in the future, the circulate window request for the specified window is not executed, and thus, any subwindow's position in the stack is not changed. For example, suppose a client application calls XCirculateSubwindowsUp to raise a subwindow to the top of the stack. If you had selected SubstructureRedirectMask on the window, the X server reports to you a CirculateRequest event and does not raise the subwindow to the top of the stack

The structure for this event type contains:

```
typedef struct {
    int type;                   /* CirculateRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window parent;
    Window window;
    int place;                  /* PlaceOnTop, PlaceOnBottom
    ↪ */
} XCirculateRequestEvent;
```

The parent member is set to the parent window. The window member is set to the subwindow to be restacked. The place member is set to what the new position in the stacking order should be and is either PlaceOnTop or PlaceOnBottom. If it is PlaceOnTop, the subwindow should be on top of all siblings. If it is PlaceOnBottom, the subwindow should be below all siblings.

- **ConfigureRequest Events**: The X server can report ConfigureRequest events to clients wanting information about when a different client initiates a configure window request on any child of a specified window. The con- figure window request attempts to reconfigure a window's size, position, border, and stacking order. The X server generates this event whenever a different client initiates a configure window request on a window by calling XConfigureWindow, XLowerWindow, XRaiseWindow, XMapRaised, XMoveResizeWindow, XMoveWindow, XResizeWindow, XRestackWindows, or XSetWindowBorderWidth.

  To receive ConfigureRequest events, set the SubstructureRedirectMask bit in the event-mask attribute of the window. ConfigureRequest events are generated when a ConfigureWindow protocol request is issued on a child window by another client. For example, suppose a client application calls XLowerWindow to lower a window. If you had selected SubstructureRedirectMask on the parent window and if the override-redirect attribute of the window is set to False, the X server reports a ConfigureRequest event to you and does not lower the specified window.

  The structure for this event type contains

```
typedef struct {
    int type;                   /* ConfigureRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window parent;
    Window window;
    int x, y;
    int width, height;
    int border_width;
    Window above;
    int detail; /* Above, Below, TopIf, BottomIf, Opposite */
    unsigned long value_mask;
} XConfigureRequestEvent;
```

The parent member is set to the parent window. The window member is set to the window whose size, position, border width, and/or stacking order is to be reconfigured. The value_mask member indicates which components were specified in the ConfigureWindow protocol request. The cor- responding values are reported as given in the request. The remaining values are filled in from the current geometry of the window, except in the case of above (sibling) and detail (stack-mode), which are reported as None and Above, respectively, if they are not given in the request.

- **MapRequest Events**: The X server can report MapRequest events to clients wanting information about a different client's desire to map windows. A window is considered mapped when a map window request completes. The X server generates this event whenever a different client initiates a map window request on an unmapped window whose override_redirect member is set to False. Clients initiate map window requests by calling XMapWindow, XMapRaised, or XMapSubwindows

  To receive MapRequest events, set the SubstructureRedirectMask bit in the event-mask attribute of the window. This means another client's attempts to map a child window by calling one of the map window request functions is intercepted, and you are sent a MapRequest instead. For example, suppose a client application calls XMapWindow to map a window. If you (usually a window manager) had selected SubstructureRedirectMask on the parent window and if the override-redirect attribute of the window is set to False, the X server reports a MapRequest event to you and does not map the specified window. Thus, this event gives your window manager client the ability to control the placement of subwindows.

  The structure for this event type contains:

```
typedef struct {
    int type; /* MapRequest */
    unsigned long serial; /* # of last request processed by
    ↪ server */
    Bool send_event; /* true if this came from a SendEvent
    ↪ request */
    Display *display; /* Display the event was read from */
    Window parent;
    Window window;
} XMapRequestEvent;
```

The parent member is set to the parent window. The window member is set to the window to be mapped.

- **ResizeRequest Events**: The X server can report ResizeRequest events to clients wanting information about another client's attempts to change the size of a window. The X server generates this event whenever some other client attempts to change the size of the specified window by calling XConfigureWindow, XResizeWindow, or XMoveResizeWindow

  To receive ResizeRequest events, set the ResizeRedirect bit in the event-mask attribute of the window. Any attempts to change the size by other clients are then redirected.

  The structure for this event type contains:

```
typedef struct {
    int type;                   /* ResizeRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    int width, height;
} XResizeRequestEvent;
```

  The window member is set to the window whose size another client attempted to change. The width and height members are set to the inside size of the window, excluding the border.

- **Colormap State Change Events**: The X server can report ColormapNotify events to clients wanting information about when the colormap changes and when a colormap is installed or uninstalled. The X server generates this event type whenever a client application
  - Changes the colormap member of the XSetWindowAttributes structure by calling XChangeWindowAttributes, XFreeColormap, or XSetWindowColormap
  - Installs or uninstalls the colormap by calling XInstallColormap or XUninstallColormap

  To receive ColormapNotify events, set the ColormapChangeMask bit in the event-mask attribute of the window.

The structure for this event type contains:

```
typedef struct {
    int type;                   /* ColormapNotify */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    Colormap colormap;          /* colormap or None */
    Bool new;
    int state;                  /* ColormapInstalled,
    ↪ ColormapUninstalled */
} XColormapEvent;
```

The window member is set to the window whose associated colormap is changed, installed, or uninstalled. For a colormap that is changed, installed, or uninstalled, the colormap member is set to the colormap associated with the window. For a colormap that is changed by a call to XFreeColormap, the colormap member is set to None. The new member is set to indicate whether the colormap for the specified window was changed or installed or uninstalled and can be True or False. If it is True, the colormap was changed. If it is False, the colormap was installed or uninstalled. The state member is always set to indicate whether the colormap is installed or uninstalled and can be ColormapInstalled or ColormapUninstalled

- **Client Communication Events**: This section discusses:
  - ClientMessage events
  - PropertyNotify events
  - SelectionClear events
  - SelectionNotify events
  - SelectionRequest events
- **ClientMessage Events**: The X server generates ClientMessage events only when a client calls the function XSendEvent.

The structure for this event type contains:

```
typedef struct {
    int type;                   /* ClientMessage */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    Atom message_type;
    int format;
    union {
        char b[20];
        short s[10];
        long l[5];
    } data;
} XClientMessageEvent;
```

The message_type member is set to an atom that indicates how the data should be interpreted by the receiving client. The format member is set to 8, 16, or 32 and specifies whether the data should be viewed as a list of bytes, shorts, or longs. The data member is a union that contains the members b, s, and l. The b, s, and l members represent data of twenty 8-bit values, ten 16-bit values, and five 32-bit values. Particular message types might not make use of all these values. The X server places no interpretation on the values in the window, message_type, or data members.

- **PropertyNotify Events**: The X server can report PropertyNotify events to clients wanting information about property changes for a specified window.

  To receive PropertyNotify events, set the PropertyChangeMask bit in the event-mask attribute of the window.

  The structure for this event type contains:

```
typedef struct {
    int type;               /* PropertyNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;
    Atom atom;
    Time time;
    int state;              /* PropertyNewValue or
    ↪ PropertyDelete */
} XPropertyEvent;
```

The window member is set to the window whose associated property was changed. The atom member is set to the property's atom and indicates which property was changed or desired. The time member is set to the server time when the property was changed. The state member is set to indicate whether the property was changed to a new value or deleted and can be PropertyNew- Value or PropertyDelete. The state member is set to PropertyNewValue when a property of the window is changed using XChangeProperty or XRotateWindowProperties (even when adding zero-length data using XChangeProperty) and when replacing all or part of a property with identical data using XChangeProperty or XRotateWindowProperties. The state member is set to PropertyDelete when a property of the window is deleted using XDeleteProperty or, if the delete argument is True, XGetWindowProperty

- **SelectionClear Events**: The X server reports SelectionClear events to the client losing ownership of a selection. The X server generates this event type when another client asserts ownership of the selection by calling XSetSelectionOwner.

  The structure for this event type contains:

```
typedef struct {
    int type;                   /* SelectionClear */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    Atom selection;
    Time time;
} XSelectionClearEvent;
```

  The selection member is set to the selection atom. The time member is set to the last change time recorded for the selection. The window member is the window that was specified by the current owner (the owner losing the selection) in its XSetSelectionOwner call.

- **SelectionRequest Events**: The X server reports SelectionRequest events to the owner of a selection. The X server gener- ates this event whenever a client requests a selection conversion by calling XConvertSelection for the owned selection.

  The structure for this event type contains:

```
typedef struct {
    int type;                   /* SelectionRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window owner;
    Window requestor;
    Atom selection;
    Atom target;
    Atom property;
    Time time;
} XSelectionRequestEvent;
```

The owner member is set to the window that was specified by the current owner in its XSetSelectionOwner call. The requestor member is set to the window requesting the selection. The selection member is set to the atom that names the selection. For example, PRIMARY is used to indicate the primary selection. The target member is set to the atom that indicates the type the selection is desired in. The property member can be a property name or None. The time member is set to the timestamp or CurrentTime value from the ConvertSelection request.

The owner should convert the selection based on the specified target type and send a Selection- Notify event back to the requestor. A complete specification for using selections is given in the X Consortium standard Inter-Client Communication Conventions Manual.

- **SelectionNotify Events**: This event is generated by the X server in response to a ConvertSelection protocol request when there is no owner for the selection. When there is an owner, it should be generated by the owner of the selection by using XSendEvent. The owner of a selection should send this event to a requestor when a selection has been converted and stored as a property or when a selection con- version could not be performed (which is indicated by setting the property member to None).

  If None is specified as the property in the ConvertSelection protocol request, the owner should choose a property name, store the result as that property on the requestor window, and then send a SelectionNotify giving that actual property name.

  The structure for this event type contains:

```
typedef struct {
    int type;               /* SelectionNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window requestor;
    Atom selection;
    Atom target;
    Atom property;          /* atom or None */
    Time time;
} XSelectionEvent;
```

The requestor member is set to the window associated with the requestor of the selection. The selection member is set to the atom that indicates the selection. For example, PRIMARY is used for the primary selection. The target member is set to the atom that indicates the converted type.

For example, PIXMAP is used for a pixmap. The property member is set to the atom that indi- cates which property the result was stored on. If the conversion failed, the property member is set to None. The time member is set to the time the conversion took place and can be a timestamp or CurrentTime

11. Event handling functions

12. Input device functions